

Apostila de Estrutura de Dados

Luis A. A. Meira

Faculdade de Tecnologia

Universidade Estadual de Campinas (Unicamp)

13 de agosto de 2026

Sumário

Prefácio	5
1 Aula 1	7
1.1 Uso do <i>debugger</i>	7
1.2 Revisão da linguagem C	8
1.3 Exercícios	14
1.4 Notação assintótica	17
2 Aula 2	25
2.1 Notação assintótica e tempo de execução	25
3 Aula 3	41
3.1 Tempo de execução de algoritmos	41
3.2 Tipo abstrato de dados	47
4 Aula 4	59
4.1 Pilhas	59
4.1.1 Pilhas Usando Vetores	59
4.1.2 Pilhas Usando Lista Ligada	68
5 Aula 5	85
5.1 Fila Usando Lista Ligada	87
5.2 Fila Usando Vetores	101

6	Aula 6	111
6.1	Revisão de apontadores	111
6.2	Listas Ligadas	112
6.3	Listas Ligadas	112
7	Aula 7	129
8	Aula 8	141
8.1	Ordenação $O(n^2)$	141
9	Aula 9	157
9.1	Ordenação em $O(n \lg n)$	157
9.2	MergeSort	158
9.3	QuickSort	166
10	Aula 10	171
10.1	Busca Sequencial	171
10.2	Busca Binária	177
10.3	Intersecção de Conjunto em $O(n \lg n)$	183
11	Aula 11	185
11.1	Definição de Árvores	185
11.2	Implementação Árvore Binária	188
11.3	Imprimindo uma Árvore Binária	197
11.4	Método de Ordenação Baseado em Árvores de Binárias de Busca	204
12	Aula 12	207
12.1	Busca em uma árvore binária de busca	207
12.2	Percursos em uma árvore	209
12.2.1	Calculando o número de elementos da árvore	216
12.2.2	Calculando a altura de uma árvore	218
12.3	Exercícios de implementação	221

Prefácio

Esta apostila foi criada na intenção de compilar o material do curso de SI201 - Estrutura de Dados I ministrada na Faculdade de Tecnologia da Unicamp. Este material não deve ser divulgado, pois encontra-se em fase de escrita e revisão.

Capítulo 1

Aula 1

Este curso supõe que você conhece linguagem C de maneira aprofundada. Indico o livro [Damas, 2006] caso o aluno precise reforçar os conceitos de C antes de começar a ler esta apostila. O conteúdo do livro é o pré-requisito para o curso corrente. Comece o curso fazendo uma boa revisão da linguagem C.

Outra opção para revisar o conteúdo de linguagem C é a apostila de C da UFMG¹ e apostila de C da Cenapad².

A disciplina de programação em C é um pré-requisito para estrutura de dados. No link a seguir temos um curso completo de C que pode ser feito antes de se fazer a leitura desta apostila: <https://t.ly/oBdZT>.

1.1 Uso do *debugger*

O aluno deve buscar uma IDE que possua *debugger*. Outra opção é usar um compilador online. Minha sugestão é o onlineGdb³ que possui uma boa interface e possui *debugger*. Todo mundo que deseja um dia ser um bom programador precisa utilizar uma ferramenta de depuração, também chamado *debugger*.

No *debugger* existe um conceito chamado *breakpoint*. O programa executa até encontrar o primeiro *breakpoint*. Outros comandos do *debugger* são:

¹<https://www.google.com/search?q=apostila+c+ufmg>

²<https://www.google.com/search?q=apostila+c+cenapad>

³<https://www.onlinegdb.com>

1. *step over*: Executa a linha corrente sem entrar em funções e procedimentos.
2. *step into*: Executa a linha corrente, entrando em funções e procedimentos.
3. *continue*: Executa várias linhas, até encontrar o próximo *breakpoint*.

Também é possível acompanhar o conteúdo das variáveis e, até mesmo, mudar o conteúdo de uma variável durante a programação. Note que, para executar um teste de mesa, ou seja, uma execução passo a passo de seu programa, é necessário executar o código linha a linha e ter também o conteúdo das variáveis que estão em memória.

Uma coisa muito comum, principalmente entre programadores sem muita experiência, é fazer um código que, ao ser executado, faz algo diferente do que era esperado. Neste ponto se faz necessário descobrir em que momento o código começa a fazer algo fora do esperado. Uma maneira é tentar cercar o erro com *printfs* até descobrir o ponto em que ocorre a primeira divergência. A maneira mais eficiente costuma ser a o uso do *debugger*. Coloca-se um *breakpoint* no início do código e executa-se o programa linha a linha, acompanhando os conteúdos das variáveis até o momento em que ocorre a primeira divergência entre o código e o que se desejou programar.

A partir da primeira divergência entre o código e a intenção do programador, tudo que vem pela frente pode ser inconsistente. Deve-se fazer a correção e repetir o processo até que todos os erros tenham sido eliminados.

Um programa não fazer aquilo para o qual foi programado é algo muito comum e muitos alunos costumam me procurar com este tipo de problema. Em geral, os alunos que me procuram não dominam o uso de *debugger*.

1.2 Revisão da linguagem C

Considere o programa abaixo:

```
1 #include <stdio.h>
2
3 int main(){
4     int x;
```

```
5     printf("Um int tem %d bits.\n",8*(int)sizeof(int));  
6 }
```

A saída deste programa será

Um int tem 32 bits.

O comando *sizeof* devolve o número de bytes da variável. É necessário multiplicar este valor por 8 para se obter o número de bits.

Agora eu gostaria de fazer um programa para descobrir o maior inteiro que pode ser armazenado por uma variável do tipo `int`. Veja o código abaixo:

```
1 #include <stdio.h>  
2  
3 int main(){  
4     int x=1,y=2;  
5     while(x<y){  
6         x++;  
7         y=x+1;  
8     }  
9     printf("Max int: %d\n",x);  
10    x=1;  
11    y=0;  
12    while(y<x){  
13        x--;  
14        y=x-1;  
15    }  
16    printf("Min int: %d\n",x);  
17 }
```

Este código vai aumentar o valor de x até que ele valha o maior inteiro possível. Neste momento $x+1$ vai dar um *overflow* e vai passar a valer uma valor menor que x . De maneira análoga vai ser encontrado o menor inteiro. Veja o que o programa imprime:

```
1 Max int: 2147483647
2 Min int: -2147483648
```

Dos 32 bits utilizados para representar o `int`, um bit vai para o sinal, que pode ser $\{+, -\}$. Temos 2^{31} números positivos, que vão de 0 a $2^{31} - 1$ e 2^{31} números negativos, que vão de -1 até -2^{31} .

O maior inteiro é 2.147.483.647, ou seja, pouco mais de 2 bilhões. Veja que este número é insuficiente para se guardar um CPF, por exemplo.

Se trabalharmos com `long`, temos 64 bits de representação. O maior `long` é $2^{63} - 1$ e o menor `long` representável é -2^{63} . Em notação de engenharia, o maior `long` será $\approx 9.2e + 18$, ou seja $\approx 9.2 \times 10^{18}$. Isso significa que `long` pode ser usado para representar números na base decimal com até 18 dígitos. O `long` pode ser usado para guardar um CPF, mas não pode ser usado para guardar o número de um boleto ou de uma conta de luz.

Sendo assim, por que trabalhar com `int`? `Int` é mais rápido? Sim, mas na maior parte dos nossos programas, o tempo é muito baixo e o uso de `long` ou `int` é irrelevante em questões de tempo. Só que o `long` vai usar uma representação desnecessariamente grande. O que também é irrelevante para programas pequenos.

Temos em C variáveis do tipo `char`, que representa um caracter. Variáveis do tipo `float` e `double` que representam ponto flutuantes, ou seja, número com vírgulas. Temos também vetores e matrizes. Um vetor `int v[5]` tem 5 posições, iniciando no zero e indo até quatro. Uma matriz `int m[3][3]` é uma matriz bidimensional 3×3 . Podemos ter um cubo, `int cubo[3][3][3]`. Este cubo possui 27 posições. Saindo de `cubo[0][0][0]` até `cubo[2][2][2]`.

Finalmente, temos a `string` que guarda uma sequência de caracteres. Uma `string` é um vetor de caracteres que termina com `\0`.

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4     char w[100];
5     //atribui
```

```
6     strcpy(w,"Ola");
7     //concatena
8     strcat(w," Mundo");
9     printf("%s\n",w);
10 }
```

Esses são os principais tipos básicos para se trabalhar em C.

Existe também em C dados complexos, que são criados por meio de um struct.

```
1 #include <stdio.h>
2
3 typedef struct Coordenada{
4     int x,y;
5 }Coordenada;
6
7 int main(){
8     Coordenada co;
9     co.x = 10;
10    co.y = 20;
11 }
```

No exemplo anterior criamos uma coordenada bidimensional, composta de dois inteiros.

Existem operadores importantes da linguagem C.

1. Operadores Condicionais:

if-else;

switch-case;

operador ternário: $(a > b)?a : b$.

2. Operadores de repetição

for: $for(i = 0; i < n; i++)$;

while: $while(a \neq b)\{\}$;

do-while: $do\{ \}while(a == b)$.

Os operadores de repetição tem a mesma capacidade de representar repetição, porém em alguns casos o *for* se encaixa melhor, em outro o *while* se encaixa melhor.

1. Operadores Aritméticos:

soma: $a + b$;

subtração: $a - b$;

multiplicação: $a * b$;

divisão: a/b ;

resto: $a \% b$;

2. Operadores Lógicos:

ou: $X || Y$

e: $X \&\& Y$

not: $!Y$

3. Operadores relacionais:

maior: $a > b$;

maior igual: $a >= b$;

menor: $a < b$;

menor igual: $a <= b$;

igual: $a == b$;

diferente: $a \neq b$;

Outros elementos importantes da linguagem C são funções e procedimentos. Veja exemplos de assinaturas de funções e procedimentos abaixo:

```
1 #include <stdio.h>
2
3 double seno(double alpha){
4     return 1.0;
5 }
6
7 double areaTriangulo(double l1, double l2, double l3){
8     return 1.0;
9 }
10
11 void recalculaSalario(double taxa){
12
13 }
14 void ordena(int v[], int n){
15
16 }
17 int main(){
18     double res = seno(3.14);
19     double res2 = area(3.0, 4.0, 5.0);
20     recalculaSalario(1.03);
21     int v[] = {1, 3, 2};
22     ordena(v, 3);
23 }
```

Todas as funções e procedimentos acima não estão implementados. Suas implementações são deixadas como exercício. A função seno tem um parâmetro (alpha) e a função área tem três parâmetros (l1, l2 e l3).

Se não devolver nada, temos um procedimento. Por exemplo, podemos fazer um procedimento chamado recalcula salários. Este procedimento recebe a taxa de aumento e trabalha

em variáveis internas do programa. Ao final, todos os salários são recalculados e o procedimento não devolve nada. Também temos o procedimento *ordena*, que recebe um vetor e trabalha sobre ele, sem devolver nada.

Uma função pode chamar outra função. Uma função também pode chamar a si mesma. Neste último caso, temos uma função recursiva.

1.3 Exercícios

Nesta seção existe um conjunto de exercícios de revisão da linguagem C.

Questão 1) (Laços de Repetição) Considere um vetor de floats com n posições, de 0 até $n - 1$. A média de um vetor é dada pela seguinte fórmula:

$$media(v, n) = \sum_{i=0}^{n-1} \frac{v[i]}{n}.$$

E o desvio padrão é dado por

$$dp(v, n) = \sqrt{\frac{1}{n-1} \sum_{i=0}^{n-1} (v[i] - media(v, n))^2}.$$

Faça um programa que calcula a média e outro programa que calcula o desvio padrão de um vetor. Você não precisa ler nada, o vetor e o tamanho são parâmetros das funções. Você não precisa escrever nada. A média e o desvio padrão são devolvidos pela função. Seu programa deve ser eficiente, ou seja, não desperdiçar processamento desnecessariamente.

Questão 2) (Debug) Diga o que há de errado neste programa. O programa calcula 20^{20} e imprime este valor.

```
int main ()
{
    int i,r;
    r=1;
    for(i=0;i<20;i++){
        r*=20;
    }
```

```

    printf("20^20=%d\n",r);
    return 0;
}

```

Questão 3) (Impressão na tela) Considere o código abaixo:

```

int main(){
    float salario, imposto;
    int idade;
    printf("Digite: salario, idade, imposto");
    scanf("%f %d %f",&salario,&idade,&imposto);
    if((salario>1000||
    (salario<1000 && idade <18))
    &&(0.1*salario<imposto)){
        printf("AAAA\n");
    }else{
        printf("BBBB\n");
    }
    return 0;
}

```

Complete a tabela

Usuário Digitou	Impressão na tela
900 17 120	
900 17 80	
900 19 80	
900 19 120	
1100 17 120	
1100 17 80	
1100 19 80	
1100 19 120	

Questão 4) (Recursão) A função abaixo está escrita de forma iterativa. Converta para uma função recursiva. A função devolver verdadeiro se existir um elemento em v que vale k , falso caso contrário. O vetor é indexado da posição 0 até a posição $n-1$.

```
int pertence(int v[], int n, int k){
    int i;
    for(i=0;i<n;i++){
        if(v[i]==k){
            return 1;
        }
    }
    return 0;
}
```

Questão 5) (String) Crie um algoritmo que remove um determinado caracter de uma string.

Suponha que $v = \text{"abxabxxcdx"}$ e que $r = \text{"x"}$. Após a execução de seu programa, o vetor deve ser igual a $v = \text{"ababcd"}$. Você não deve nenhuma biblioteca pronta. Esta questão pode ser implementada sem o uso de um vetor adicional.

Questão 6) (String) Existe uma função que concatena duas strings na biblioteca *string.h*. A função chama-se *concat(a,b)* para a e b strings. Por exemplo, se $a = \text{"abc"}$ e $b = \text{"xyz"}$, após a execução de *concat(a,b)*, o primeiro vetor passa a valer $a = \text{"abcxyz"}$. Suponha que você não pode usar a biblioteca *string.h*. Reimplemente a função *concat* que concatena duas strings.

Questão 7) (Matriz) Faça um programa que soma 3 matrizes quadradas. As matrizes de entrada são A, B e C . Você deve somá-las e guardar o resultado em R . A dimensão da matriz é $n \times n$.

Questão 8) (Estruturas)

Considere as seguintes estruturas:

```
struct circulo{
```

```

    float r;
};
struct cilindro{
    float h;
    struct circulo base;
};

```

Crie uma função que recebe como parâmetro um círculo e calcula a área do círculo. Faça uma função que recebe como parâmetro um cilindro e calcula o volume de um cilindro. O volume do cilindro deve usar a função da área do círculo.

Dicas) A área do círculo πr^2 . O volume do cilindro é *area_circulo * altura*

1.4 Notação assintótica

Uma boa referência para notação assintótica é o livro [Cormen et al., 2002], Seção 3.1, crescimento de funções, notação assintótica. Fica também como sugestão a leitura dos capítulos I e II deste livro.

Existem três principais notações assintóticas: O (lê-se ó) Θ (lê-se teta) e Ω (lê-se omega).

E do que se trata notação assintótica. Considere uma função $f(n) > 0$. Note que a função é sempre maior que zero. A teoria de notação assintótica só vale para funções positivas. Temos também uma segunda função $g(n) > 0$. Temos também que $n > 0$. Existem estas duas funções positivas.

Mas que funções são estas? Em geral, a função $f(n)$ é o tempo de execução do algoritmo ou o número de operações do algoritmo, onde n é, em geral, o tamanho da instância. Por exemplo, n pode representar o número de elementos em um vetor e $f(n)$ o tempo para se ordenar o vetor.

Se eu tiver um grafo, n pode ser o número de vértices ou o número de arestas do grafo.

Temos que $f(n) = O(g(n))$ se e somente se existirem duas constantes positivas c e n_0 tal que $0 \leq f(n) \leq cg(n), \forall n > n_0$.

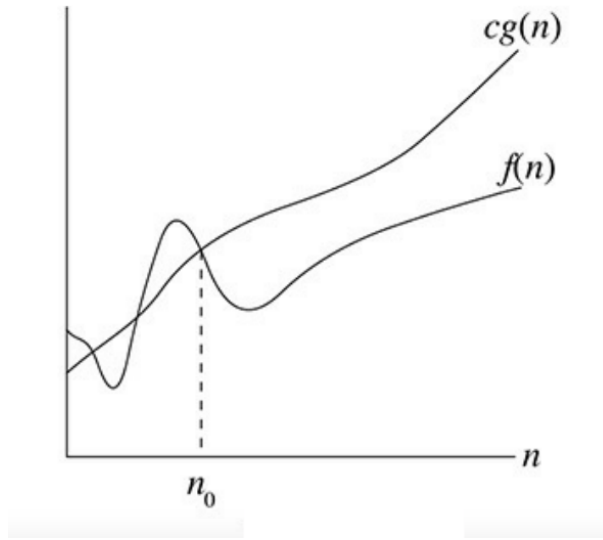


Figura 1.1: Notação assintótica $f(n) = O(g(n))$. Fonte: [Cormen et al., 2002]

Podemos usar tanto $f(n) \in O(g(n))$ quanto $f(n) = O(g(n))$. Pela definição, para que $f(n)$ seja $O(g(n))$, podemos desconsiderar valores pequenos de n . Podemos iniciar em um valor grande, porém constante n_0 . Somente a partir de n_0 é que $f(n)$ precisa ser limitada superiormente por $cg(n)$. Outra coisa que podemos fazer é aumentar a função $g(n)$ tanto quanto for necessário, multiplicando-a por uma constante c .

Veja a Figura 1.1 para exemplificar a função O . A notação O diz respeito a um limitante superior.

Vamos ver um exemplo de como isso funciona. Seja $f(n) = 10n^2 + 5n + 5$ e $g(n) = n^2$. Mostre que $10n^2 + 5n + 5 = O(n^2)$.

Vou fazer uma tentativa e usar a constante igual a $c = 10$.

$$10n^2 + 5n + 5 \leq 10n^2, \forall n > 0 \text{ (False)} \quad (1.1)$$

A inequação acima está errada, pois o lado esquerdo é maior que o lado direito. Suponha agora que $n > 1$. Nestas circunstâncias $n \leq n^2$ e $1 \leq n^2$.

$$10n^2 + 5n + 5 \leq 10n^2 + 5n^2 + 5n^2, \forall n > 1 \quad (1.2)$$

$$10n^2 + 5n + 5 \leq 20n^2, \forall n > 1. \quad (1.3)$$

Neste momento termina a prova. Temos que $10n^2 + 5n + 5 = O(n^2)$ pois existem constantes positivas $c = 20$ e $n_0 = 1$ tal que $10n^2 + 5n + 5 \leq 20n^2$, para todo $n > 1$. Note que eu poderia usar outras constantes como $c = 30$ ou $c = 40$ e poderia usar $n > 10$ ou $n > 100$ que a afirmação continuaria verdadeira.

A função O diz o seguinte. Eu posso multiplicar minha função n^2 por uma constante. Será que eu consigo fazer n^2 passar $10n^2 + 5n + 5$? Eu também posso desprezar valores pequenos de n . No caso, todo $n \leq 1$ é desprezado.

O que esta notação assintótica quer dizer? Todas as funções a seguir também são $O(n^2)$:

- $100n^2$
- $5n^2 + 5n$
- $5n^2 + \log n$

Eu agrupo várias funções e digo que todas elas são $O(n^2)$. todas estas funções são limitadas superiormente por n^2 . No caso $5n^2 + 5n$ eu olho apenas para o termo mais importante para valores grandes de n , que é o $5n^2$. Para valores grande de n , $5n^2$ vai ser muito maior que $5n$ e assim o $5n$ pode ser desprezado. Eu também posso desprezar constantes multiplicativas, e olhar apenas para o n^2 . Notação assintótica é uma teoria para grandes números. Ao ignorar-se as constantes, $5n^2$ e $1000000n^2$ tem a mesma complexidade $O(n^2)$.

Sendo assim temos classes de algoritmos: algoritmos $O(n^2)$, que são chamados quadráticos; algoritmos $O(n)$ que são chamados lineares; algoritmos $O(\log n)$ que são logarítmicos e assim por diante.

É claro que $n = O(n^2)$ mas neste caso temos folga. Temos que $n = O(n)$, e nesse caso temos uma notação justa. Temos também, de maneira imediata que $n^2 = O(n^3)$, $n^2 = O(n^4)$ e assim por diante. Nestes casos também temos folga.

Quando é que a notação O dá errado? Vamos tentar provar que $n = O(\sqrt{n})$, ou seja, que n é limitada superiormente por $\sqrt{n}$. De fato não é, mas vamos tentar provar e ver o que acontece.

Vamos considerar a função $\sqrt{n}$. Vamos multiplicar $\sqrt{n}$ por uma constante bem expressiva, como 10^6 . Será que $n \leq 10^6 \sqrt{n}$ para todo $n \geq n_0$?

Não estamos interessados em valores pequenos de n . Podemos desconsiderar $n < n_0$. Independente da constante que eu use, n sempre vai ultrapassar $\sqrt{n}$. Mas vamos tentar.

$$n \leq c\sqrt{n}, n \geq n_0 \quad (1.4)$$

$$n^1 \leq cn^{0.5}, n \geq n_0 \quad (1.5)$$

$$n^{1-0.5} \leq c, n \geq n_0 \quad (1.6)$$

$$n^{0.5} \leq c, n \geq n_0 \quad (1.7)$$

$$\sqrt{n} \leq c, n \geq n_0 \quad (1.8)$$

Após alguns cálculos chegamos a uma contradição. Nossa constante c precisa ser maior que $\sqrt{n}$ e este valor vai para infinito. Não existe constante nestas condições. Sendo assim, $n \neq O(\sqrt{n})$.

Algo análogo acontece ao se tentar provar que $n^3 = O(n^2)$. Temos que, para ser O temos que $n^3 \leq cn^2, \forall n > n_0$. Chegamos a contradição que $c \geq n$. Qualquer constante oferecida, existe um n que inviabiliza esta constante.

Considere o código abaixo:

```

1 #include <stdio.h>
2 int main(){
3     int n=100;
4     int x = 0;
5     for(int i=0;i<n;i++){
6         x = x+3;
7     }
8 }
```

Neste caso eu pergunto quantas operações são gastas pelo algoritmo em função de n . Suponha que o valor de n possa ser mudado. Temos o seguinte:

1. declaração e atribuição de n : 2 operações;
2. declaração e atribuição de x : 2 operações;

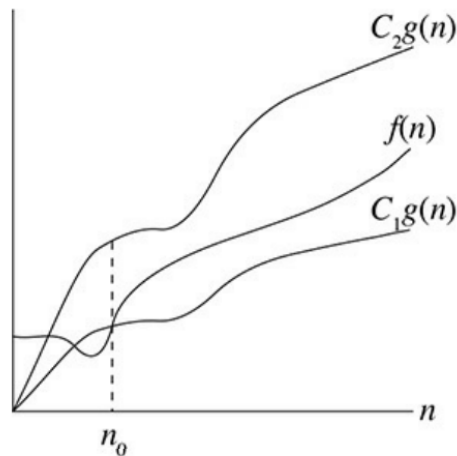


Figura 1.2: Notação assintótica $f(n) = \Theta(g(n))$. Fonte: [Cormen et al., 2002]

3. declaração e atribuição de i : 2 operações;
4. comparação $i < n$: $n+1$ operações;
5. $i++$: n operações;
6. $x = x + 3$: n operações;

O número total de operações é $T(n) = 3n + 7$. Nestes casos não estamos interessados no número exato de operações. O que é importante é que o número de operações é $T(n) = O(n)$. Ou seja, o algoritmo é linear.

A outra notação assintótica que é de nosso interesse é a Θ (lê-se teta).

Temos que $f(n) = \Theta(g(n))$ se e somente se existirem constantes positivas c_1 , c_2 e n_0 tais que $0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$ para todo $n > n_0$.

Novamente não estou interessado em valores pequenos. A partir de um certo ponto a função $f(n)$ está numa espécie de sanduíche. Temos $g(n)$ multiplicada por uma constante c_1 que derruba $g(n)$ e a deixa sempre menor que $f(n)$ para $n > n_0$ e c_2 que levanta a função $g(n)$ e a deixa sempre maior que $f(n)$ para $n > n_0$. $f(n)$ fica no meio de $g(n)$ a partir de um certo ponto e até o infinito. Veja Figura 1.2.

Seja $f(n) = 100n^2 + 100n + 100$. Eu gostaria de mostrar que $f(n) = \Theta(n^2)$. Vou assumir

que $n > 1$. Nestas circunstâncias eu tenho que:

$$100n^2 + 100n + 100 \leq 100n^2 + 100n^2 + 100n^2, \forall n > 1 \quad (1.9)$$

$$100n^2 + 100n + 100 \leq 300n^2, \forall n > 1 \quad (1.10)$$

$$(1.11)$$

Esta parte é idêntica aos cálculos feitos para O . Agora temos que mostrar o outro lado:

$$n^2 \leq 100n^2 + 100n^2 + 100n^2, \forall n > 1 \quad (1.12)$$

Esta parte também é verdadeira.

Sendo assim, $f(n) = \Theta(n^2)$ pois existem constantes positivas $c_1 = 1$, $c_2 = 300$ e $n_0 = 1$ tal que $0 \leq n^2 \leq f(n) \leq 300n^2, \forall n \geq 1$.

Exercícios:

Questão 9) Mostre que $100n^2 + 200n^2$ é $\Theta(n^2)$.

Questão 10) Mostre que $n \log n = O(n^2)$.

Questão 11) Mostre que, dado um polinômio de grau k , com o seguinte formato $P(x) = a_0x^0 + a_1x^2 + \dots + a_kx^k$ onde os valores a_i são constantes. Mostre que $P(x) = \Theta(x^k)$.

Questão 12) Justifique ou refute as seguintes afirmativas:

1. Se $f(n) = O(g(n))$ então $f(n) = \Theta(g(n))$.
2. Se $f(n) = \Theta(g(n))$ então $f(n) = O(g(n))$.
3. Se $f(n) = \Theta(g(n))$ então $g(n) = \Theta(f(n))$.

Questão 13) Responda “sim” ou “não”. Justifique sua resposta com uma prova matemática.

- $10n^2 + 10n + 100 = O(n^2)$?
- $n^2 = O(10n^2 + 10n + 100)$?
- $100n^3 = O(n^4)$?

- $n^4 = O(n^3)$?
- $2^n = O(3^n)$?
- $3^n = O(2^n)$?
- $\log_{10}(n) = O(\log_2 n)$?
- $10n^2 + 10n + 100 = \Theta(n^2)$?
- $n^2 = \Theta(10n^2 + 10n + 100)$?
- $100n^3 = \Theta(n^4)$?
- $2^n = \Theta(3^n)$?
- $3^n = \Theta(2^n)$?
- $\log_{10}(n) = \Theta(\log_2 n)$?

Capítulo 2

Aula 2

Nesta aula vamos dar continuidade ao tema notação assintótica.

2.1 Notação assintótica e tempo de execução

Continuando, temos que existem cinco notações assintóticas: $O, o, \Theta, \Omega, \omega$. Lê-se ó, ozinho, teta, ômega e omegazinho. No nosso curso não iremos trabalhar com o e ω . As mais importantes são O e Θ .

Às vezes dizemos que um pesquisador é mais focado no $O(n)$. Isso significa que tal pesquisador está mais preocupado com a complexidade do algoritmo do que em detalhes de eficiência e programação.

A complexidade de um programa é descrito por meio de notação assintótica. Por exemplo, um algoritmo tem complexidade $O(n^2)$ ou tem complexidade $O(n^3)$. O programa de ordenação, por exemplo, pode ter complexidade $O(n \lg n)$.

Note que $\log n$ significa $\log_{10}$ enquanto $\lg n$ significa $\log_2$. Em termos de notação assintótica, não faz diferença usar $\log n$ e $\lg n$ pois a diferença entre eles é apenas uma constante. Ou seja, $\log n = \Theta(\lg n)$.

Temos também complexidades exponenciais. Um algoritmo pode ser $\Theta(2^n)$ ou $\Theta(n!)$. Algoritmos exponenciais são os piores e tendem a executar em tempo hábil apenas para valores muito pequenos de n .

Todo programa está associado a um tempo e este tempo é descrito por meio de notação

assintótica. Veja o código abaixo:

```

1 for(int i=0;i<n;i++){
2     for(int j=0;j<n;j++){
3         v[i] = i+j;
4     }
5 }

```

Quantas operações o programa gasta para fazer dois *for*s encadeados? O laço mais interno vai gastar $laco_j(n) = 1 + (n + 1) + n + n = 3n + 2$ pois

- $int\ j = 0$: 1 operação ;
- $j < n$: $n + 1$ operações;
- $j++$: n operações;
- $v[i] = i + j$: n operações.

O laço mais externo vai gastar algo semelhante:

$$T(n) = 1 + (n + 1) + n + n(laco_j(n)) \quad (2.1)$$

$$T(n) = 2n + 2 + n(3n + 2) \quad (2.2)$$

$$T(n) = 3n^2 + 4n + 2 \quad (2.3)$$

Não estamos preocupados com valores pequenos de n . Gostaríamos de saber $T(n)$ para $n = 1.000$ ou $n = 1.000.000$. Quando n é pequeno, todo programa é rápido.

Para $n = 1000$, $T(n) = 3 * 1000 * 1000 + 4 * 1000 + 2$. Perto de 3.000.000 tanto o 4.000 quanto o 2 são muito pequenos. Podemos assumir que $T(n) = 3 * 1000 * 1000$ inserindo um erro de apenas 0.1%. Podemos jogar fora inclusive a constante 3 em $3n^2$ e deixar apenas $T(n) = \Theta(n^2)$.

A notação assintótica é uma maneira de simplificar o tempo de execução ou número de operações de algoritmos, jogando fora parte da informação e simplificando o resultado. No fim dizemos apenas que o tempo de execução do algoritmo é $\Theta(n^2)$.

Vamos dizer agora que o um programa gasta tempo $T_1(n) = n10^{-6}s$. A constante 10^{-6} varia de computador para computador. Um outro algoritmo gasta $T_2(n) = n^210^{-6}s$. Se n for igual a 1 milhão, quanto tempo gastam os programas 1 e 2?

Para o programa 1 temos:

$$T_1(n) = n10^{-6}s \quad (2.4)$$

$$T_1(10^6) = 10^610^{-6}s \quad (2.5)$$

$$= 1s \quad (2.6)$$

Já para o programa 2 temos:

$$T_2(n) = n^210^{-6}s \quad (2.7)$$

$$T_2(10^6) = (10^6)^210^{-6}s \quad (2.8)$$

$$= 1.000.000s \quad (2.9)$$

$$\approx 11dias. \quad (2.10)$$

Enquanto o primeiro programa gasta 1s, o segundo programa gasta 1 milhão de segundos. Se o programador cometeu um erro e, ao invés de fazer um programa $\Theta(n)$ fez um programa $\Theta(n^2)$, o programa fica 1 milhão de vezes mais lento para $n = 1.000.000$. Para exemplificar, vamos analisar uma questão que foi deixada como exercício, o cálculo da média e do desvio padrão.

O código abaixo está correto, porém intencionalmente ineficiente:

```

1 double media(double v[], int n){
2     double sum=0;
3     for(int i=0;i<n;i++){
4         sum+=v[i];
5     }
6     return sum/n;
7 }
8 double dvp(double v[],int n){
9     double sum=0,aux;
```

```
10     for(int i=0;i<n;i++){
11         aux=v[i]-media(v,n);
12         sum+=aux*aux;
13     }
14     sum = sum /(n-1);
15     return sqrt(sum);
16 }
```

O calculo da média é $\Theta(n)$. O cálculo de desvio padrão é um for que repete n vezes alguns cálculos, entre eles o cálculo da média. Como o calculo da média é repetido n vezes, a complexidade final fica em $\Theta(n^2)$.

Para deixar o cálculo do desvio padrão $\Theta(n)$ basta salvar a média em uma variável que é calculada uma única vez, conforme mostrado abaixo:

```
1
2 double dvp(double v[],int n){
3     double soma=0,aux,med;
4     med = media(v,n);
5     for(int i=0;i<n;i++){
6         aux=v[i]-med;
7         sum+=aux*aux;
8     }
9     sum = sum /(n-1);
10    return sqrt(sum);
11 }
```

Este tipo de erro é bastante comum, principalmente para programadores iniciantes. Programadores que não dominam estrutura de dados também estão propensos criar versões não otimizadas de algoritmos.

A partir da análise destes programas fica claro que não interessa muito a constante. Se um programa gasta 1s e outro gasta 1 milhão de segundos, mesmo que a constante do mais

lento muda de 10^{-6} para 3×10^{-6} a diferença continua muito grande. A diferença entre o $\Theta(n)$ e o $\Theta(n^2)$ é tão grande que a relevância das constantes de tempo se torna pequena.

Já para valores pequenos de n , como $n = 10$, a diferença de tempos torna-se insignificante, a não ser que o algoritmo seja executado muitas vezes.

Em estrutura de dados é desejável sempre reduzir a complexidade do algoritmo. Transformar um algoritmo $\Theta(n^2)$ em outro $\Theta(n \log n)$. Transformar um algoritmo $\Theta(n)$ em outro $\Theta(\log n)$.

Uma terceira notação assintótica que ainda não foi citada é a Ω . Trata-se de um limitante inferior para uma função.

Temos que $f(n) = \Omega(g(n))$ se e somente se existirem constante positivas c e n_0 tais que $0 \leq cg(n) \leq f(n)$ para todo $n > n_0$. Neste caso, $f(n)$ é limitada inferiormente por $g(n)$. Veja o exercício abaixo:

Questão 14) Mostre que $g(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.

Temos que $n^2 = \Omega(n^2)$ e também que $n^2 = \Omega(n)$, $n^2 = \Omega(\sqrt{n})$, $n^2 = \Omega(\log n)$ e assim por diante.

Vamos agora ver como se aplica a notação assintótica em desenvolvimento de algoritmos. Considere o código abaixo:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 //tamanho maximo do vetor.
5 #define MAX 1000000
6 /**
7  Gera um vetor de numeros aleatorios de tamanho n. O valor
8  de cada elemento esta entre 0 e m-1.
9  Complexidade Teta(n).
10 */
11 void gera(int v[], int n , int m){
12     if(n>MAX){
```

```
13         printf("Vetor de tamanho insuficiente\n");
14         exit(1);
15     }
16     for(int i=0;i<n;i++){
17         v[i] = rand()%m;
18     }
19 }
20
21 /**
22  Imprime o vetor na tela.
23  Complexidade Teta(n).
24  */
25 void print(int v[], int n){
26     printf("[");
27     for(int i=0;i<n;i++){
28         printf("%d",v[i]);
29         if(i<n-1){
30             printf(",");
31         }
32     }
33     printf("]\n");
34 }
35 /**
36  Encontra a posicao do maior elemento do vetor.
37  Se existirem varios elementos nesta condicao,
38  devolve a posicao do primeiro maior elemento.
39  Complexidade Teta(n).
40  */
41 int maximo(int v[], int n){
42     int max = 0;
```

```
43     for(int i=1;i<n;i++){
44         if(v[i]>v[max]){
45             max = i;
46         }
47     }
48     return max;
49 }
50 /**
51  Ordena os elementos em ordem crescente via
52  ordenacao por selecao.
53  Teta(n^2)
54  */
55 void ordena(int v[], int n){
56     while(n>1){
57         //seleciona o maior elemento
58         int max = maximo(v,n);
59         //coloca na ultima posicao
60         int aux = v[n-1];
61         v[n-1] = v[max];
62         v[max]=aux;
63         //repete o processo no vetor
64         //desconsiderando o ultimo elemento
65         n--;
66     }
67 }
68 int main(){
69     clock_t start, end;
70     double cpu_time_used;
71     int v[MAX];
72     int n;
```

```

73     printf("Digite n:");
74     scanf("%d",&n);
75     gera(v,n,n);
76     if(n<200) print(v,n);
77     // Inicio da contagem de tempo
78     start = clock();
79     ordena(v,n);
80     // Fim da contagem de tempo
81     end = clock();
82     if(n<200) print(v,n);
83     printf("Ordenou\n");
84     // Calculo do tempo gasto em milisegundos
85     cpu_time_used = (1000.0*(double) (end - start)) / CLOCKS_PER_SEC;
86     printf("Tempo de execucao: %f milisegundos\n", cpu_time_used);
87
88
89 }
```

A saída deste programa será:

```

Digite n:10
[7,9,3,8,0,2,4,8,3,9]
[0,2,3,3,4,7,8,8,9,9]
Ordenou
Tempo de execucao: 0.003000 milisegundos
```

O primeiro vetor é o vetor aleatório e o segundo vetor é o vetor ordenado. Note que a função máximo é chamada n vezes. A primeira vez é passado n , em seguida $n - 1$, depois $n - 2$ e assim sucessivamente até chegar em 2. O número de operações é $n + (n - 1) + (n - 2) + \dots + 3 + 2$. Esta soma é a soma de uma progressão aritmética (PA). A fórmula para calcular a soma de uma PA é $(a_1 + a_n)n/2$. Em nosso caso temos $\frac{(n+2)(n-1)}{2} = \frac{n^2}{2} + \frac{n}{2} - 1$.

Ou seja, o número de operações feitas para calcular o máximo é $\Theta(n^2)$. Como as demais operações são $\Theta(n)$ o resultado final é $\Theta(n^2)$.

O código foi executado para $n = 100.000$ e o tempo resultante foi 10.723 ms. Ou seja, o programa gastou 10s para ordenar um vetor com 100.000 elementos. O tempo foi medido logo antes e logo depois da ordenação. Podemos dizer que, dentro de uma certa aproximação, que $T(n) = cte * n^2 ms$. Eu pego só a parte mais importante do polinômio e ignoro elementos proporcionais a n^1 e a n^0 . Com estas informações eu posso calcular o valor da constante.

$$T(n) = cte * n^2 ms \quad (2.11)$$

$$T(100000) = cte * (100000)^2 ms = 10723 ms \quad (2.12)$$

$$cte * 10^{10} ms = 10723 ms \quad (2.13)$$

$$cte = 10723 * 10^{-10} \quad (2.14)$$

$$cte = 1,0723 * 10^4 * 10^{-10} \quad (2.15)$$

$$cte \approx 1,07 * 10^{-6} \quad (2.16)$$

$$(2.17)$$

Com isso, eu tenho a fórmula $T(n) = n^2 * 10^{-6} ms$. Agora eu posso estimar o tempo gasto para $n = 200.000$.

$$T(200.000) = 200.000^2 * 1,07 * 10^{-6} ms \quad (2.18)$$

$$= (2 * 10^5)^2 * 1,07 * 10^{-6} ms \quad (2.19)$$

$$= 4 * 10^{10} * 1,07 * 10^{-6} ms \quad (2.20)$$

$$= 4 * 1,07 * 10^4 ms \quad (2.21)$$

$$= 42800 ms \quad (2.22)$$

$$= 42.8 s \quad (2.23)$$

O tempo estimado é de 42.8s. Ao se executar o programa e medir o tempo, o resultado foi 42,734s. Note que a constante de tempo varia de computador para computador.

Questão 15) (Tempo de execução) Considere o programa anterior (função `ordena`) que possui tempo descrito por $T(n) = cn^2$.

1. Determine a constante de tempo c para a função `ordena` executando o programa em seu computador. Não use compiladores online, como *OnlineGDB*, pois eles afetam a constante de tempo.
2. Com base nessa constante, estime o tempo necessário para ordenar:
 - um vetor com 1.000.000 elementos;
 - um vetor com 10.000.000 elementos.

Note que não é preciso esperar o programa executar, pois pode demorar muito tempo. Você deve apenas estimar o tempo a partir da fórmula.

3. Escolha entre 5 e 10 valores de n , bem distribuídos, de modo que a execução do programa seja viável no seu computador. Para cada n , meça o tempo real de execução $T(n)$.
4. Organize os resultados em uma planilha com três colunas:
 - (a) n (tamanho do vetor);
 - (b) $T(n)$ (tempo medido experimentalmente);
 - (c) $T(n) = cn^2$ (tempo estimado pelo modelo teórico).
5. Construa um gráfico de linha com dados da planilha para comparar os tempos medidos e os estimados. No eixo x deve ficar o valor de n e no eixo y uma linha representando o tempo medido e outra o tempo estimado.

A função de tempo se aproxima de uma parábola. Um erro muito comum é o aluno tentar utilizar uma regra de três numa função que não é linear. Por exemplo, se para um dado n o programa anterior gastou $1s$, quanto tempo levará para $10n$?

O raciocínio errado é

$$\frac{n}{10n} = \frac{1s}{x},$$

ou seja $x = 10s$.

O raciocínio correto é:

$$T(n) = cte * n^2 \quad (2.24)$$

$$T(10n) = cte * (10n)^2 \quad (2.25)$$

$$T(10n) = 100cte * n^2 \quad (2.26)$$

$$T(10n) = 100T(n) \quad (2.27)$$

$$(2.28)$$

Ou seja, para $10n$ o tempo gasto será 100 vezes maior. Note que a constante cte muda de computador para computador. Ao se usar notação assintótica, as constantes cte são eliminadas.

Para finalizar, qual o tempo será gasto para $n=10$ milhões?

$$T(10^7) = (10^7)^2 * 1,07 * 10^{-6}ms \quad (2.29)$$

$$= 10^{14} * 1,07 * 10^{-6}ms \quad (2.30)$$

$$= 1,07 * 10^8ms \quad (2.31)$$

$$= 107.000.000ms \quad (2.32)$$

$$= 29,72h \quad (2.33)$$

O programa irá demorar $29,72h$. Note que, neste caso, fica bem mais complicado executar o programa para testar se o tempo calculado está de acordo com o valor medido.

Questão 16) Descubra as complexidades das funções abaixo:

```

1 #include <stdio.h>
2
3
4 int f1(int n){
5     int aux = 0;
6     for(int i=0;i<n;i++){
7         aux++;
8     }

```

```
9      return aux;
10 }
11
12
13 int f2(int n){
14     int x=0;
15     for(int i=0;i<n;i++){
16         for(int j=0;j<n;j++){
17             x++;
18         }
19     }
20     return x;
21 }
22
23 int f3(int n){
24     int aux = 0;
25     for(int i=1;i<n;i=i*2){
26         aux++;
27     }
28     return aux;
29 }
30
31 int f4(int n){
32     int aux = 0;
33     for(int i=0;i<n;i++){
34         aux+=f1(i);
35     }
36     return aux;
37 }
38
```

```
39 int f5(int n){
40     int aux = 0;
41     for(int i=0;i<n;i++){
42         aux+=f1(n);
43         aux+=f2(n);
44     }
45     return aux;
46 }
47
48 int f6(int n){
49     int x=0;
50     for(int i=0;i<n;i+=100){
51         x+= 1;
52     }
53     return x;
54 }
55
56 int f7(int n){
57     if(n==0){
58         return 1;
59     }else{
60         int x = f7(n-1);
61         int y = f7(n-1);
62         return x+y;
63     }
64 }
65
66 int main(int argc, const char * argv[]) {
67     int n = 10;
68 }
```

```

69     printf("f1=%d\n", f1(n));
70     printf("f2=%d\n", f2(n));
71     printf("f3=%d\n", f3(n));
72     printf("f4=%d\n", f4(n));
73     printf("f5=%d\n", f5(n));
74     printf("f6=%d\n", f6(n));
75     printf("f7=%d\n", f7(n));
76
77 }
```

Questão 17) Proponha uma função com complexidade $\Theta(2^n)$ (a) usando *for/while* (b) sem usar *for/while*.

Considere o código da função $f3(n)$ do exercício anterior. Neste código, a cada iteração, o valor de i dobra. Sendo assim, em k iterações, o valor de i vai valer 2^k . Como o programa executa k iterações, ele é $\Theta(k)$. O programa pára quando i atingir n , ou seja, quando $2^k = n$.

$$\begin{aligned}
 2^k &= n \\
 \lg 2^k &= \lg n \\
 k \lg 2 &= \lg n \\
 k &= \lg n
 \end{aligned}$$

Note que $\lg(n) = \log_2(n)$. Veja que k representa o número de iterações. Neste caso, o algoritmo é $\Theta(\lg n)$.

Veja que $\lg n$ é um número que cresce muito lentamente. Se escolhermos $n = 2^{1000}$ que tem uma ordem de grandeza de 1 seguido de 301 zeros. Temos que $\lg 2^{1000} = 1000$. Isso quer dizer que um algoritmo $\Theta(\lg(n))$ tende a ser muito rápido.

Por outro lado, funções exponenciais tem o crescimento muito acelerado. O valor 10^n cresce de maneira muito rápida. Se um algoritmo for $T(n) = 10^n \times 10^{-6}s$, para valores pequenos de n teremos um tempo muito grande.

Suponha que $T(n)$ seja 1 século. Neste caso, $T(n) = 100 \times 365 \times 24 \times 3600 = 3.153.600.000s$.

$$10^n \times 10^{-6}s = 3.153.600.000s \quad (2.34)$$

$$10^n = 3.153.600.000 \times 10^6$$

$$n = \lg_{10}(3.153.600.000 \times 10^6)$$

$$n = 15,49$$

Isso significa que para $n = 15$ o valor ainda é menor que 1 século, mas para $n = 16$ o tempo já supera um século. O programa precisa executar por mais de 1 século para resolver um problema de tamanho apenas $n = 16$.

Além dos algoritmo exponenciais e logarítmicos, existem os algoritmos polinomiais. Eles estão no meio. São melhores que os exponenciais e piores que o logarítmicos. Eles são algoritmo $\Theta(n^k)$ para um k constante. Para $k = 1$ temos os algoritmos lineares, ou seja $\Theta(n)$. Para $k = 2$ temos os algoritmos quadráticos, ou seja $\Theta(n^2)$. Para $k = 3$ temos os algoritmos cúbicos, ou seja $\Theta(n^3)$.

Quanto menor o expoente do polinômio, melhor o algoritmo. Um algoritmo $\Theta(n)$ é melhor que um algoritmo $\Theta(n^2)$ que é melhor que um algoritmo $\Theta(n^3)$ e assim por diante.

Questão 18) Suponha que $T(n) = n^3$ *cte s*. Se $T(10.000) = 1000s$, qual a constante de tempo? Qual o tempo $T(10^6)$?

Questão 19) Considere o programa de ordenação descrito a partir da página 29.

Calcule a constante de tempo c para a função `ordena`, executando o programa em seu computador. Atenção: não utilize compiladores online, como o *OnlineGDB*, pois eles podem interferir na determinação da constante de tempo.

A constante deve ser calculada a partir da fórmula:

$$c = \frac{T(n)}{n^2},$$

onde $T(n)$ representa o tempo de execução medido para ordenar um vetor de tamanho n .

1. Para cada valor de n no conjunto

$$\{1, 5, 10, 20, 100, 200, 500, 1000, 10000, 100000, 1000000\},$$

meça o tempo $T(n)$ e calcule a constante correspondente $c(n)$.

2. Analise e descreva o comportamento da constante $c(n)$ à medida que n aumenta. Discuta se os valores de $c(n)$ se estabilizam ou variam de forma significativa conforme o tamanho do vetor cresce.

Capítulo 3

Aula 3

Esta aula trata de tempo de execução de algoritmos e tipo abstrato de dados.

3.1 Tempo de execução de algoritmos

Questão 20) (tempo de execução) Este exercício foi extraído de [Cormen et al., 2002], Problema 1.1. Para cada função $f(n)$ e cada tempo t , determine o maior tamanho de n de um problema que pode ser resolvido no tempo t , supondo que o algoritmo para resolver o problema demore $f(n)$ microsegundos segundos.

$f(n)$	1 segundo	1 minuto	1 hora	1 dia	1 mês	1 ano	1 século
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
3^n							
$n!$							

Vou responder alguns exemplos e deixar outros como exercício. Suponha que $f(n) = \lg n$

e que o tempo é 1s. Para um dado n , o algoritmo gasta $\lg n * 10^{-6}s$. Temos então que $\lg n * 10^{-6}s = 1s$,

$$\lg n * 10^{-6}s = 1s \quad (3.1)$$

$$\lg n = 10^6 \quad (3.2)$$

$$n = 2^{1.000.000} \quad (3.3)$$

$$= (10^{0,3010299...})^{1.000.000} \quad (3.4)$$

$$= 10^{301029,9...} \quad (3.5)$$

O maior número calculável em 1s é n igual a $10^{301029,9...}$ ou seja, o número 1 seguido de mais de 300.000 zeros. Isso indica que um algoritmo regido pela função $\lg n$ é extremamente rápido, mesmo para valores muito grandes de n .

Questão 21) Suponha que $f(n) = \log n$, ou seja $\log$ na base 10. Qual o valor de n em 1s?

Vamos supor agora que $f(n) = n^2$ e que o tempo é 1 dia. Neste caso temos que

$$n^2 * 10^{-6}s = 24 * 3600s \quad (3.6)$$

$$n^2 * 10^{-6}s = 86400s \quad (3.7)$$

$$n^2 = 86400 * 10^6 \quad (3.8)$$

$$n = \sqrt{86400 * 10^6} \quad (3.9)$$

$$= 293938,76 \quad (3.10)$$

Ou seja, o maior n que executa em um dia é 293.938.

Questão 22) Suponha agora um computador 10x mais rápido. Qual o valor de n pode ser executado em 1 dia?

Suponha agora que $f(n) = 2^n$ e que o tempo é 1 século:

$$2^n * 10^{-6}s = 100 * 365,25 * 24 * 3600s \quad (3.11)$$

$$2^n = 100 * 365,25 * 24 * 3600 * 10^6 \quad (3.12)$$

$$\lg 2^n = \lg(100 * 365,25 * 24 * 3600 * 10^6) \quad (3.13)$$

$$n = 51.48 \quad (3.14)$$

$$(3.15)$$

Nesse caso temos o oposto da função lg. A função 2^n cresce de maneira tão grande que, em um século, seria possível executar uma instância de tamanho apenas 51.

Questão 23) Suponha agora que $f(n) = 3^n$. Qual o valor de n pode ser executado em um século?

Suponha que $f(n)$ seja n^3 e que $T(n) = 1$ mês. Qual o maior valor de n ?

A formula diz o seguinte:

$$T(n) = f(n)10^{-6}s$$

O tempo na fórmula está em convertido de microsegundos para segundos. Note que 1 mês vale $30 \times 24 \times 3600 = 2.592.000$ s. Veja que um mês tem 30 dias, um dia tem 24h e uma hora tem 3600s.

Sendo assim,

$$2.592.000s = n^3 10^{-6}s \quad (3.16)$$

$$n^3 10^{-6} = 2.592.000$$

$$n^3 = 2.592.000 \times 10^6$$

$$n = (2.592.000 \times 10^6)^{1/3}$$

$$n = 13736$$

A interpretação deste resultado é que, em um computador com constante de tempo $10^{-6}s$, em um mês de processamento, é possível resolver um problema com $n = 13.736$ usando um algoritmo regido por $T(n) = n^3 10^{-6}s$.

Questão 24) Suponha agora um computador 1000x mais rápido. Qual o valor de n pode ser executado em 1 mês?

Os demais campos da tabela são deixados como exercícios.

Questão 25) (tempo de execução) Suponha que um algoritmo demora $T(n) = 10^n 10^{-6}s$. Qual o maior valor de n que pode ser executado em 1 milhão de anos?

Para resolver este problema, basta igualar $10^n 10^{-6}s$ ao número de segundos de 1 milhão

de anos.

$$10^n * 10^{-6}s = 10^6 * 365,25 * 24 * 3600s \quad (3.17)$$

$$10^n = 10^6 * 365,25 * 24 * 3600 * 10^6 \quad (3.18)$$

$$\log 10^n = \log(365,25 * 24 * 3600 * 10^{12}) \quad (3.19)$$

$$n = 19.49 \quad (3.20)$$

No caso, o maior n inteiro é 19. Ou seja, em um milhão de anos será possível resolver um problema com n valendo apenas 19.

Questão 26) (Complexidade) Suponha que eu tenho duas matrizes $B_{n \times n}$ e $C_{n \times n}$ e desejo calcular $A_{n \times n} = B_{n \times n} + C_{n \times n}$. Qual a complexidade?

Resposta: Para percorrer uma matriz quadrada, é necessário fazer um *for* varrendo as linhas e, para cada linha, fazer um *for* percorrendo as colunas. Isso gera dois *fors* encadeados. A complexidade é $\Theta(n^2)$.

Suponha que você está analisando um código grande, composto por centenas de linhas. Suponha que existem trechos $\Theta(n)$, $\Theta(n \lg n)$, $\Theta(1)$ e apenas um pequeno trecho, de poucas linhas, $\Theta(n^2)$. Este pequeno trecho irá dominar o tempo de execução do código inteiro. Ele vai ser o gargalo do seu programa. Para localizar um gargalo, é possível analisar os códigos manualmente ou usar uma ferramenta chamada *profiler* que detecta trechos de códigos mais lentos.

Questão 27) (Complexidade) Suponha agora que eu tenho duas matrizes $B_{n \times n}$ e $C_{n \times n}$ e desejo multiplicar $A_{n \times n} = B_{n \times n} C_{n \times n}$. Qual a complexidade?

Resposta: A resposta para este problema é mais complexa. Suponha que eu desejo calcular $a_{ij} \in A_{n \times n}$. Para se obter o elemento a_{ij} é necessário fazer o produto interno da linha i de B pela coluna j de C .

$$a_{ij} = \sum_{k=1}^n b_{ik} c_{kj}.$$

Desta maneira obtemos a_{ij} em $\Theta(n)$. Como existem n^2 a_{ij} distintos, o tempo final é $\Theta(n^3)$.

Podemos propor algoritmos mais ineficientes para multiplicar duas matrizes, como $\Theta(n^4)$, mas isso não faz muito sentido. A pergunta mais importante é: existem algoritmo ainda melhores que $\Theta(n^3)$?

A resposta é sim. O algoritmo proposto em [Strassen et al., 1969] obteve uma $O(n^{\log_2 7}) \approx O(n^{2.8704})$.

O melhor algoritmo para multiplicação de matrizes foi obtido em [Alman and Williams, 2021] que obteve complexidade $(n^{2.37286})$.

Não sabemos, na data atual, se é possível encontrar algoritmos para multiplicar matrizes ainda melhores. O limite $O(n^2)$ não pode ser baixado, mas explorar valores entre n^2 e $n^{2.37286}$ ainda é um campo aberto para pesquisas.

Questão 28) (Tempo de execução). Suponha um algoritmo para multiplicar matrizes com tempo de execução $T(n) = n^3 10^{-6} s$. Qual a maior matriz que pode ser multiplicada em 1h.

Resposta:

$$T(n) = n^3 10^{-6} s = 3600s \quad (3.21)$$

$$n^3 = 3600 * 10^6 \quad (3.22)$$

$$n = \sqrt[3]{3600 * 10^6} \quad (3.23)$$

$$n = 1532,61. \quad (3.24)$$

Ou seja, o maior valor inteiro de n tal que duas matrizes $n \times n$ pode ser multiplicada em 1h pelo algoritmo proposto é 1.532. Veja que a matriz não é tão grande. Se eu tiver duas matrizes de dimensão 1532, isso irá levar uma hora.

Questão 29) (Tempo de execução) Suponha agora que o programa de multiplicar matrizes foi executado em um computador 10 vezes mais rápido. Qual o maior valor n pode ser resolvido neste computador?

Resposta: Se o computador é 10 vezes mais rápido, a constante de tempo irá mudar de 10^{-6} para 10^{-7} .

$$T(n) = n^3 10^{-7} s = 3600s \quad (3.25)$$

$$n^3 = 3600 * 10^7 \quad (3.26)$$

$$n = \sqrt[3]{3600 * 10^7} \quad (3.27)$$

$$n = 3301,92. \quad (3.28)$$

Neste caso é interessante notar que o computador é 10 vezes mais rápido, mas a instância aumentou apenas $\sqrt[3]{10} = 2.15\dots$ vezes.

Questão 30) Suponha que governo do estado de São Paulo calcula um índice sobre os salários de seus funcionários. Dado um conjunto de n salários, toda vez que existirem dois salários s_i, s_j tal que $s_i < 2000$ e $s_j > 5000$, o índice é acrescentado de $1/n^2$. Veja o código abaixo:

```

1 double indice(int salar[],int n){
2     double idx = 0;
3     for(int i=0;i<n;i++){
4         for(int j=0;j<n;j++){
5             if(salar[i]<2000.0
6                 &&salar[j]>5000.0){
7                 idx+=1.0;
8             }
9         }
10    }
11    return idx/(n*n*1.0);
12 }
```

Este código é $O(n^2)$. Altere o código para que o mesmo índice seja calculado em $O(n)$.

Questão 31) Ainda sobre o código da questão anterior. Suponha que os programas levam tempo $T_1(n) = n^2 10^{-9} s$ e $T_2(n) = n 10^{-9} s$. O algoritmo $O(n)$ é quantas vezes mais rápido que o programa $O(n^2)$ para $n = 5000$? Mostre os cálculos.

3.2 Tipo abstrato de dados

Tipo abstrato de dados (TAD) consiste na separação entre uso e implementação de uma estrutura de dados. Uma estrutura como uma pilha é implementada uma única vez e pode ser usada diversas vezes. O programador que é usuário da estrutura de dados pode utilizá-la sem precisar saber os detalhes da implementação. Mais que isso, é possível trocar a implementação e o usuário da estrutura não sente nenhuma diferença.

Uso | Implementação

Um programador iniciante pode começar a programar e criar um banco de programas prontos. Por exemplo, implementa um algoritmo de ordenação como o MergeSort. Você vai implementar este algoritmo uma única vez em toda sua vida. Todas as vezes que você precisar usar ordenar, você pode usar MergeSort que já está implementado. Isso vale para outras estruturas como *hash*, árvores, *heap*, etc.

A essência de TADs é implementar a estrutura uma única vez e usar esta estrutura sem se preocupar com detalhes da implementação. É importante saber, no caso de um algoritmo de ordenação, se ele é $O(n^2)$ ou $O(n \lg n)$. Tirando este detalhe, você pode usar a estrutura quantas vezes for necessário.

Você vai implementar uma lista ligada uma vez e vai usar quantas vezes for necessário.

Considere o código abaixo:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 //Estrutura coordenada
6 typedef struct Coordenada{
7     int x,y;
8 }Coordenada;
9
```

```
10 //Construtor da estrutura
11 Coordenada * newCoordenada(int x, int y){
12     Coordenada * aux = (Coordenada *)malloc(sizeof(Coordenada));
13     aux->x = x;
14     aux->y = y;
15     return aux;
16 }
17
18 void printCoordenada(Coordenada *c){
19     printf("(%d,%d)",c->x,c->y);
20 }
21
22 //Distancia euclidiana entre duas coordenadas
23 double euclideanDist(Coordenada * c1, Coordenada * c2){
24     double dx = (c1->x)-(c2->x);
25     double dy = (c1->y)-(c2->y);
26     return sqrt(dx*dx+dy*dy);
27 }
28
29 //Segmento de reta composto de duas
30 //coordenadas, uma de inicio e outra de fim.
31 typedef struct Segmento{
32     Coordenada *ini, *fim;
33 }Segmento;
34
35 Segmento * newSegmento(Coordenada *ini, Coordenada *fim){
36     Segmento * aux = (Segmento *)malloc(sizeof(Segmento));
37     aux->ini = ini;
38     aux->fim = fim;
39     return aux;
```

```
40 }
41
42 void printSegmento(Segmento * s){
43     printf("[");
44     printCoordenada(s->ini);
45     printCoordenada(s->fim);
46     printf("]");
47 }
48
49 double comprimentoSegmento(Segmento * s){
50     return euclideanDist(s->ini,s->fim);
51 }
52
53 int main(void){
54     Coordenada *c1 = newCoordenada(10,10);
55     Coordenada *c2 = newCoordenada(20,20);
56     printCoordenada(c1);printf("\n");
57     printCoordenada(c2);printf("\n");
58     printf("Distancia %.2lf\n",euclideanDist(c1,c2));
59     Segmento * s = newSegmento(c1,c2);
60     printSegmento(s);printf("\n");
61     printf("Comprimento:%.2lf\n",comprimentoSegmento(s));
62 }
```

Veja a saída do programa:

```
(10,10)
(20,20)
Distancia 14.14
[(10,10)(20,20)]
Comprimento:14.14
```

Os tipos abstratos de dados são *coordenada* e *segmento*. Cada uma com as respectivas operações. Vamos detalhar o código agora.

```
1 //Estrutura coordenada
2 typedef struct Coordenada{
3     int x,y;
4 }Coordenada;
```

Este trecho de código define uma estrutura chamada *Coordenada*. Tanto a estrutura quanto o tipo são *Coordenada*, por isso esta palavra aparece duas vezes. A estrutura possui dois valores, *x* e *y* do tipo *int*. Após criarmos esta estrutura, temos duas opções de instanciar estruturas *Coordenada*, veja abaixo:

```
1 int main(void){
2     //alocacao estatica da coordenada
3     Coordenada c1;
4     c1.x = 10;
5     c1.y = 20;
6
7     //alocacao dinamica da coordenada
8     Coordenada * c2 =(Coordenada *)malloc(sizeof(Coordenada));
9     c2->x = 10;
10    c2->y = 20;
11    free(c2);
12 }
```

Também podemos passar uma *Coordenada* por valor ou por referência, conforme mostrado abaixo:

```
1 //Passagem coordenada por valor
2 Coordenada func1(Coordenda c1){
3     c1.x = 5;
4     c1.y = 10;
```

```
5     return c1;
6 }
7
8 //Passagem coordenada por referencia
9 Coordenada * func2(Coordenada *c1){
10     c1->x = 5;
11     c1->y = 10;
12     return c1;
13 }
```

Nesta disciplina eu vou utilizar preferencialmente a alocação dinâmica de estruturas e a passagem por referência.

Além de usar estruturas com alocação dinâmica, neste curso iremos utilizar funções construtoras. Uma função construtora é uma técnica utilizada em orientação a objetos. C é uma linguagem estruturada, mas é uma boa prática de programação o uso de funções construtoras. Esta função possui duas etapas: a alocação dinâmica da estrutura e atribuição de conteúdo para os campos da estrutura. Veja o exemplo abaixo:

```
1 //Construtor da estrutura
2 Coordenada * newCoordenada(int x, int y){
3     Coordenada * aux = (Coordenada *)malloc(sizeof(Coordenada));
4     aux->x = x;
5     aux->y = y;
6     return aux;
7 }
```

Neste exemplo temos alocação de uma estrutura *Coordenada* e atribuição de conteúdo para os seus dois campos, que são dois inteiros passados como parâmetro. Abaixo temos outro construtor de estrutura:

```
1 Segmento * newSegmento(Coordenada *ini, Coordenada *fim){
2     Segmento * aux = (Segmento *)malloc(sizeof(Segmento));
```

```
3     aux->ini = ini;
4     aux->fim = fim;
5     return aux;
6 }
```

Neste caso, fazemos a alocação dinâmica de um segmento, composto por duas coordenadas. Em seguida atribuímos conteúdo para os dois campos da estrutura, que são duas coordenadas.

Questão 32) Faça uma função que recebe como parâmetro um segmento (passagem por referência) e inverta o segmento, trocando o início com o fim.

Temos um conjunto de operações sobre as estruturas:

```
1 void printCoordenada(Coordenada *c){
```

```
1 double euclideanDist(Coordenada * c1, Coordenada * c2){
```

```
1 void printSegmento(Segmento * s){
```

```
1 double comprimentoSegmento(Segmento * s){
```

Uma vez definidas as estruturas, feitos os construtores e definidas as operações sobre as estruturas, temos a segunda etapa, que é o uso do tipo abstrato de dados. Em nosso exemplo, o uso é todo feito dentro da função main.

```
1 int main(void){
2     Coordenada *c1 = newCoordenada(10,10);
3     Coordenada *c2 = newCoordenada(20,20);
4     printCoordenada(c1);printf("\n");
5     printCoordenada(c2);printf("\n");
6     printf("Distancia %.2lf\n",euclideanDist(c1,c2));
7     Segmento * s = newSegmento(c1,c2);
8     printSegmento(s);printf("\n");
```

```

9     printf("Comprimento: %.2lf\n", comprimentoSegmento(s));
10 }

```

Este padrão de programação composto por estrutura, construtor, operações e uso será usado por todo o curso de estrutura de dados.

Questão 33) Crie uma função que faz a translação de uma coordenada. Esta função recebe como parâmetro um *deltax* e um *deltay*. Após a translação, a posição *x* da coordenada é movida em *deltax* unidades e a posição *y* é movida *deltay* posições. Exemplo: Se coordenada for (5, 5) e a translação for (3, 3), após a translação, a nova posição será (8, 8).

Número de linhas de código esperada: 2

```

1 void translacaoCoord(Coordenada *c, int deltax, int deltay);

```

Questão 34) faça uma translação de um segmento. Esta função deve chamar `translacaoSeg()`

Número de linhas de código esperada: 2

```

1 void translacaoSeg(Segmento *seg, int deltax, int deltay);

```

Questão 35) Crie, em C, uma estrutura triângulo, composta por três coordenadas. Crie um construtor de triângulo. Crie uma função que calcula o perímetro de um triângulo. Crie uma função que imprime o triângulo no formato texto. Instancie estruturas de triângulos, passando como parâmetro as três coordenadas. Chame as funções implementadas.

A seguir temos um exercício: Calcular a interseção entre dois segmentos de reta.

Questão 36) (Operação sobre segmentos)

```

1 //Exercicio: Calcule a intersecao (cruzamento) entre dois segmentos.
2 //Se a intersecao for vazia, devolva NULL
3 Coordenada * intersecao(Segmento *a, Segmento *b){
4     Coordenada * inter = NULL;
5     //seu codigo vem aqui
6     return inter;
7 }

```

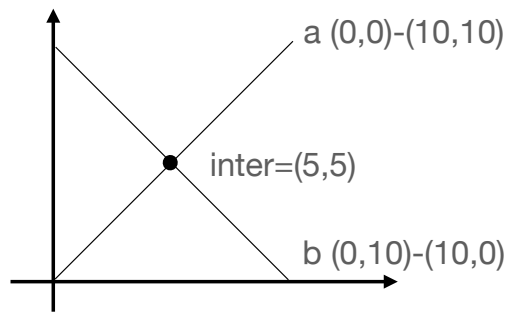


Figura 3.1: Exemplo de dois segmentos e sua interseção.

```

8 int main(void){
9     Coordenada * c1 = newCoordenada(0,0);
10    Coordenada * c2 = newCoordenada(10,10);
11    Segmento * a = newSegmento(c1,c2);
12    Coordenada * c3 = newCoordenada(0,10);
13    Coordenada * c4 = newCoordenada(10,0);
14    Segmento * b = newSegmento(c3,c4);
15    Coordenada * inter = intersecao(a,b);
16    if(inter==NULL){
17        printf("Vazio\n");
18    }else{
19        printCoordenada(inter);
20    }
21 }

```

A saída do programa acima deve ser o ponto (5,5). Veja os segmentos na Figura 3.1.

O próximo tipo abstrato de dados que veremos é um polígono. Um polígono é uma figura geométrica fechada formada por segmentos de reta. Exemplo: Quadrado, pentágono, hexágono, etc. Em nosso contexto, o polígono composto por um vetor de coordenadas.

```

1 //Poligono

```

```
2 typedef struct Poligono{
3     Coordenada * v[100];
4     int top;
5 }Poligono;
```

A estrutura de um polígono é composta por um vetor de coordenadas, com no máximo 100 posições e um contador *top* que guarda o número de coordenadas efetivamente utilizado. Por exemplo, um triângulo tem $top = 3$. Um quadrado tem $top = 4$ e assim por diante.

A seguir temos o construtor da estrutura:

```
1 //Construtor
2 Poligono * newPoligono(){
3     Poligono * pol = (Poligono *)
4         malloc(sizeof(Poligono));
5     pol->top=0;
6     return pol;
7 }
```

O construtor cria uma estrutura polígono com zero coordenadas. Nesse caso $top = 0$. Outra operação sobre polígono é a impressão, conforme visto a seguir:

```
1 //Imprime
2 void printPoligono(Poligono *pol){
3     printf("[");
4     for(int i=0;i<pol->top;i++){
5         printCoordenada(pol->v[i]);
6     }
7     printf("]");
8 }
```

O *for* inicia em 0 e vai até $pol \rightarrow top - 1$. É acessada cada coordenada no vetor $pol \rightarrow v[]$. A próxima função adiciona uma coordenada no polígono:

```

1 //Adiciona coordenada ao poligono
2 void addCoord(Poligono *pol, Coordenada *c){
3     pol->v[pol->top++]=c;
4 }

```

A variável *top* tem dois significados. Ela representa o número de elementos no vetor e também representa a primeira posição livre no vetor. Como temos *top* elementos e o primeiro está na posição zero, o último elemento está na posição $top - 1$. A primeira posição livre é *top*. A função coloca a coordenada *c* na posição $pol \rightarrow top$ e depois incrementa *top*. Essa função é semelhante à função empilha que será visto mais à frente. Veja uma representação do que acontece na Figura 3.2.

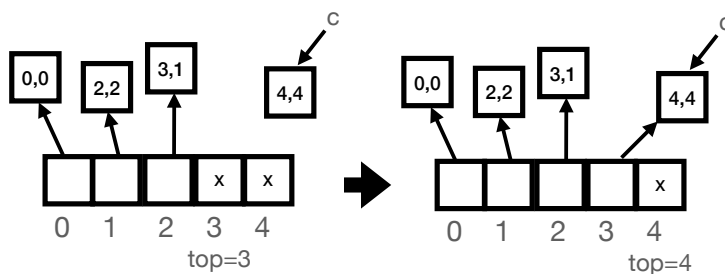


Figura 3.2: Exemplo de inserção de uma coordenada no polígono.

Uma operação comum em um polígono é o cálculo do perímetro, que é a soma dos lados. Veja a implementação a seguir:

```

1 //soma dos comprimentos dos lados de um poligono
2 double perimetro(Poligono *pol){
3     double per=0;
4     //coordenada i com i+1

```

```
5     for(int i=0;i<pol->top-1;i++){
6         per+=euclideanDist(pol->v[i],pol->v[i+1]);
7     }
8     //fechamento: ultima coordenada com a primeira
9     if(pol->top>1){
10         per+=euclideanDist(pol->v[0],pol->v[pol->top-1]);
11     }
12     return per;
13 }
```

Agora que já implementamos diversas operações sobre polígonos vamos fazer o uso do tipo abstrato de dados.

```
1 //Uso do tipo abstrato de dados Poligono.
2 int main(void){
3     Poligono * pol = newPoligono();
4     addCoord(pol,newCoordenada(0,0));
5     addCoord(pol,newCoordenada(1,1));
6     addCoord(pol,newCoordenada(2,0));
7     printPoligono(pol);printf("\n");
8     printf("Perimetro: %.2f\n",perimetro(pol));
9 }
```

A saída do programa será:

[(0,0)(1,1)(2,0)]

Perimetro: 4.83

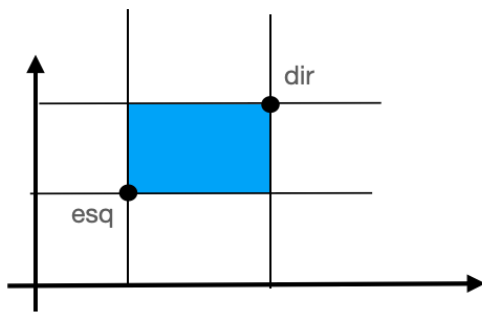
Questão 37) faça uma translação de um polígono. Esta função deve chamar `translacaoCoord()`. Cada uma das coordenadas do polígono devem sofrer a translação.

```
1 void translacaoPol(Poligono *p,int deltax, int deltay);
```

Questão 38) Crie uma estrutura `Círculo`. O círculo deve ter dois parâmetros. Uma coordenada que representa o centro e um raio. Crie uma função construtor para a estrutura círculo. Crie uma função translação para a estrutura círculo. Faça as chamadas para as funções implementadas.

Questão 39) Responda:

a) Implemente uma estrutura retângulo, composta por duas coordenadas, coordenada *esq* que representa o canto inferior esquerdo do retângulo e coordenada *dir* que representa o canto superior direito do retângulo. Este retângulo tem a mesma orientação dos eixos cartesianos. Não é um retângulo inclinado. Veja a imagem abaixo:



- b) Implemente o construtor do retângulo.
- c) Faça uma função que recebe um retângulo e imprime as suas coordenadas
- d) Implemente uma função que recebe um retângulo e devolve a sua área
- e) Implemente uma função que recebe um retângulo e devolve o seu perímetro
- f) Implemente uma função que recebe dois retângulos e verificam se existe intersecção entre eles. Note que a intersecção entre dois retângulos é vazia ou é um retângulo. Sendo assim, seu código deve devolver uma estrutura do tipo retângulo, com valor *null* se a intersecção for vazia.
- g) Faça o *main* chamando as funções implementadas.

Capítulo 4

Aula 4

4.1 Pilhas

4.1.1 Pilhas Usando Vetores

A sequência da matéria trata das estruturas de dados pilhas, filas e listas. São as três estruturas de dados mais básicas.

Uma pilha é composta de um conjunto de elementos e uma regra que diz o seguinte: “O último a entrar é o primeiro a sair”. Em inglês a regra é “*Last In First Out (LIFO)*”. Veja <https://pt.wikipedia.org/wiki/LIFO>.



Figura 4.1: Imagem de uma pilha de pratos. Note que o último prato a entrar na pilha é o primeiro prato a sair da pilha.

A Figura 4.1 mostra uma pilha de pratos. Se a regra *Last In First Out* não for seguida,

não temos uma pilha como entendida na computação. É preciso seguir regras na inserção e na remoção dos elementos. Os elementos sempre são inseridos no topo e sempre são removidos do topo da pilha.

Note que uma fila de supermercado não respeita a regra LIFO. No supermercado, o primeiro a entrar é o primeiro a sair da fila. Esta regra em inglês se chama *First In First Out (FIFO)* e a estrutura associada neste caso é uma fila e não uma pilha.

Um terceiro caso é se puder entrar e sair em qualquer posição. Neste caso não se trata nem de pilha nem de fila e sim de lista.

O ato de inserir um elemento na pilha é chamado de *push* e o ato de remover um elemento da pilha é chamado de *pop*. Em português, *push* é empilha e *pop* é desempilha. O termo pilha em inglês para pilha é *stack*.

Um uso comum da pilha é a pilha de chamadas em um programa. Suponha um programa em C que inicia no *main*. Suponha que o *main* chama uma função f_1 . O programa inicia com o *main* no topo da pilha de chamadas. Ao chamar a função f_1 , a função *main* congela e o programa começa a executar a função f_1 que vai para o topo da pilha. Se a função f_1 chamar a função f_2 , a função f_1 congela e a função f_2 vai para o topo pilha. Isso ocorre sucessivamente a cada nova chamada de função. Quando a função f_2 terminar de executar, ela é removida da pilha. A função f_1 se torna o novo topo e começa a executar. Quando a função f_1 terminar, ela sai da pilha e a função *main* passa a ser o novo topo. Note que a função *main* foi a primeira a entrar e é a última a sair. Quando a função *main* terminar a pilha de chamadas fica vazia e o programa acaba.

Suponha agora que desejamos implementar uma pilha usando um vetor. Veja a Figura 4.3.

Este vetor foi criado com uma capacidade de *MAX* elementos. Será criada uma variável inteira chamada *top* que inicialmente vale zero, ou seja, *top* está associada a primeira posição do vetor. Na programação utilizada aqui, *top* tem dois significados: (i) o número de elementos na pilha e (ii) a primeira posição livre do vetor.

Na Figura 4.3, após inserir os elementos 3, 5 e 7, a variável *top* vale 3. Isso quer dizer que temos três elementos na pilha também quer dizer que a posição 3 é a primeira posição livre do vetor. Os elementos estão nas posições 0, 1 e 2.

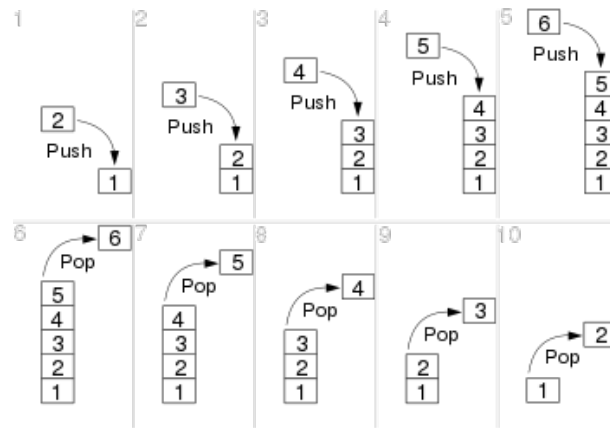


Figura 4.2: Imagem de uma pilha e as funções *push* e *pop*. Fonte: <https://pt.wikipedia.org/wiki/LIFO>.

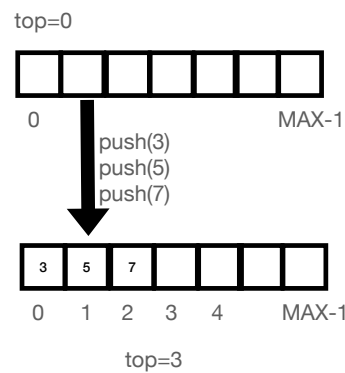


Figura 4.3: Implementação de uma pilha usando um vetor.

Veja o código abaixo:

```

1 #include <stdio.h>
2 #define MAX 100
3
4 //pilha implementada com vetor (sequencial)
5 int top=0;
6 int v[MAX];
7
8 void push(int x){

```

```
9      v[top++] = x;
10 }
11
12 int pop(){
13     return v[--top];
14 }
15
16 int main(){
17     push(3);
18     push(5);
19     push(7);
20     printf("%d\n", pop());
21     printf("%d\n", pop());
22     printf("%d\n", pop());
23 }
```

A saída deste programa será:

```
7
5
3
```

Note que a função *push* tem o seguinte implementação:

```
1      v[top++] = x;
```

O elemento x é colocado na posição top , que é a primeira posição livre do vetor. Em seguida, top é incrementado.

Já a função *pop* está mostrada a seguir:

```
1      return v[--top];
```

Veja que primeiro é executado o decremento da variável top pois o operador $--$ está à

esquerda da variável. Em seguida é devolvido o valor que está na posição *top* já decrementada. O elemento que estava na última posição da pilha foi removido logicamente, mas, na prática, ele continua lá. Isso não é problema pois a única forma de acessar tal posição é dando um *push* que irá sobrescrever o conteúdo antigo.

Note que a pilha só é acessada no topo. Ela só permite inserir e remover do topo. Se acessar posições internas da pilha, ela deixa de ser uma pilha. O ponto fraco desta implementação é que ela é limitada a *MAX* posições.

Questão 40) Uma utilidade da pilha é inverter uma determinada sequência. Para tanto, basta inserir os elementos na pilha e depois removê-los. Faça uma função *main* que faz a leitura de uma sequência de inteiros até que seja digitado um número menor que 0. Após digitar um número menor que zero, deve-se imprimir a sequência na ordem contrária a ordem que foi digitada.

Esta primeira implementação de pilha foi apenas para fins didáticos. O vetor e a variável *top* foram definidas como variáveis globais, o que permite a existência de uma única pilha. Além disso, variáveis globais não são boas práticas de programação. A seguir temos uma implementação de pilha usando vetores de maneira mais adequada.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Pilha{
5     int *v;
6     int max,top;
7 }Pilha;
8
9 Pilha * newPilha(int max){
10     Pilha * p = (Pilha *)malloc(sizeof(Pilha));
11     p->v = (int *)malloc(sizeof(int)*max);
12     p->max = max;
13     p->top = 0;
14     return p;
```

```
15 }
16
17 //empilha
18 void push(Pilha *p,int x){
19     //verifica se a pilha esta cheia
20     if(p->top==p->max){
21         printf("Erro: Pilha cheia\n");
22         exit(1);
23     }
24     p->v[p->top++]=x;
25 }
26
27 //desempilha
28 int pop(Pilha *p){
29     //verifica se a pilha esta vazia
30     if(p->top==0){
31         printf("Erro: Pilha vazia\n");
32         exit(1);
33     }
34     p->top--;
35     return p->v[p->top];
36 }
37
38 int estaVazia(Pilha *p){
39     return p->top==0;
40 }
41
42 //numero de elementos na pilha
43 int size(Pilha *p){
44     return p->top;
```

```
45 }
46
47 void printPilha(Pilha *p){
48     printf("(");
49     for(int i = p->top-1; i>=0;i--){
50         printf("%d",p->v[i]);
51         if(i>0){
52             printf("<-");
53         }
54     }
55     printf(")");
56     printf("\n");
57 }
58
59 int main(){
60     Pilha * p = newPilha(3);
61     push(p,3);
62     push(p,5);
63     push(p,7);
64     printPilha(p);
65 }
```

A saída do programa será:

(7<-5<-3)

Note que no construtor da estrutura *Pilha*, o vetor é criado por alocação dinâmica, mas seu conteúdo não é zerado. Isso não é um problema, pois para se fazer a leitura de uma determinada posição é necessário que antes seja feito um *push* e tal função irá sobrescrever algum conteúdo de lixo que porventura venha estar no vetor.

As funções *push* e *pop* são semelhantes às mostradas anteriormente. Entretanto, para se acessar o vetor e a variável *top* é necessário iniciar o acesso pela variável *p* que é uma

estrutura do tipo *Pilha*. Lembrando que a estrutura *Pilha* possui um vetor v , um inteiro top e um inteiro max que são acessado por $p \rightarrow v$, $p \rightarrow top$ e $p \rightarrow max$.

Antes de inserir um elemento na pilha, foi verificado se a pilha não está cheia e antes de remover um elemento foi verificado se a pilha não está vazia.

Também foi criada uma função que verifica se a pilha está vazia e uma função que imprime uma pilha. A pilha é impressa colocando o topo do lado esquerdo. Isto é opcional. Poderia-se imprimir a pilha colocando o topo do lado direito também.

Questão 41) Refaça a questão anterior usando a estrutura *Pilha*.

Vamos agora resolver um problema de balanceamento de parênteses usando pilha. Veja os exemplos abaixo:

1. $(2 + 2) + (2 * (2 + 2))$: Parênteses balanceados.
2. $2) + (2$: Parênteses desbalanceados. O primeiro parênteses fecha mas não tem nenhum parênteses aberto no momento. Note que neste exemplo o número de parênteses que abre é igual ao número de parênteses que fecha.
3. $(2 * (2 + 2)$: Desbalanceado. O primeiro parênteses não é fechado.

Existem muitas maneiras de se resolver este problema, mas eu optei por resolver utilizando uma pilha. Toda vez que um parênteses é aberto eu o empilho. Toda vez que um parênteses é fechado, eu desempilho. Se aparecer um fecha parênteses e a pilha estiver vazia, a resposta é “desbalanceado”. Exemplo $()$ Se, ao final, a pilha não terminar vazia, a resposta é “desbalanceado”. Exemplo $(($). Caso contrário, os parênteses estão balanceados. Veja o código a seguir:

```
1 //devolve 1 se os parenteses estao balanceados
2 //0 caso contrario
3 int balanceado(char *texto){
4     int size;
5     size = strlen(texto);
6     Pilha * p = newPilha(size);
7     for(int i=0;i<size;i++){
```

```
8         if(texto[i]=='('){
9             push(p,1);
10            //printPilha(p);
11        }
12        if(texto[i]==')'){
13            if(estaVazia(p)){
14                return 0;
15            }
16            pop(p);
17            //printPilha(p);
18        }
19
20    }
21    return estaVazia(p);
22 }
23
24 int main(){
25     char texto[100];
26     strcpy(texto,"((2+2)*2)+(2+2)");
27     printf("%s  balanc.=%d\n",texto,balanceado(texto));
28     strcpy(texto,"(())");
29     printf("%s  balanc.=%d\n",texto,balanceado(texto));
30     strcpy(texto,"()()");
31     printf("%s  balanc.=%d\n",texto,balanceado(texto));
32 }
```

A saída do programa será:

((2+2)*2)+(2+2) balanc.=1

(()) balanc.=0

()() balanc.=0

Nos programas desta aula não foram feitas as desalocações de memória. Isso significa que a memória vai ser utilizada enquanto o programa estiver rodando. Ao programa terminar, a memória é liberada. Entretanto, é mais correto fazer um *free* para cada *malloc* utilizado. Isto é deixado como exercícios.

Questão 42) (Desalocação) Faça a desalocação de memória de uma pilha sequencial.

Questão 43) Altere o código para que o *push* insira um elemento na posição livre mais à direita conforme mostrado na Figura 4.4. Refaça a função o *pop* para remover seguindo a regra LIFO (*last in first out*). Refaça também a função que imprime a pilha. Faça a função *main* chamando as funções implementadas.

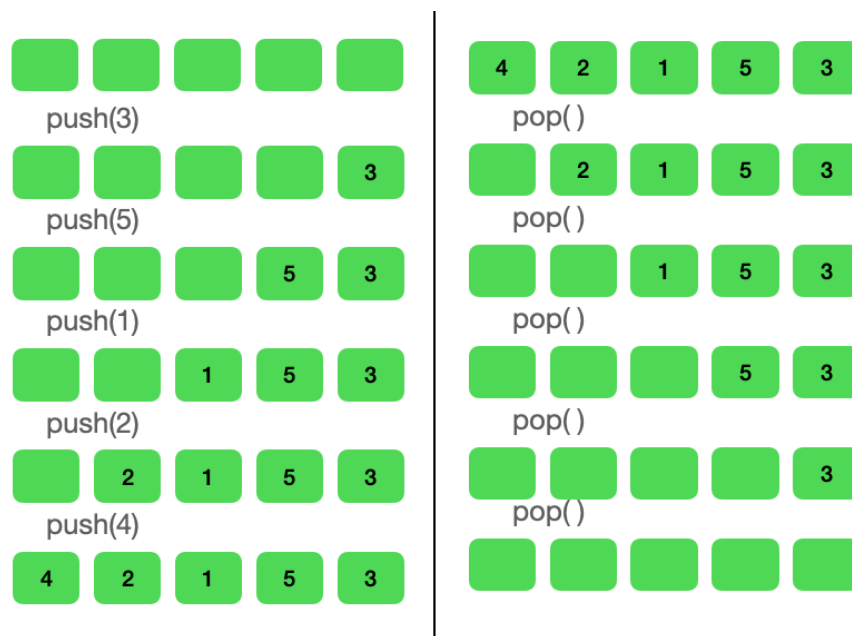


Figura 4.4: Nesta figura temos o *push* e o *pop* modificados. Note que, neste caso, o elemento é inserido na posição livre mais a direita. Os elementos são inseridos, então, da direita para a esquerda. O *pop* também precisa ser alterado para que a pilha continue funcionando como LIFO.

4.1.2 Pilhas Usando Lista Ligada

Nesta seção iremos implementar a pilha usando lista ligada também chamada de lista encadeada. A base para criação deste tipo de pilha é o uso de uma estrutura chamada *No*. Veja

a Figura 11.7.



Figura 4.5: Exemplo da estrutura *No*.

Veja como o nó é implementado na linguagem C:

```

1 //estrutura No
2 typedef struct No{
3     int valor;
4     struct No * prox;
5 }No;
  
```

Tal estrutura possui um valor que pode ser qualquer coisa. Em nosso exemplo o valor é um inteiro, mas poderia ser uma *string*, ou um aluno, uma pessoa, uma mercadoria, etc. Outro elemento na estrutura *No* é um apontador para o próximo nó. Ele pode conter um endereço de um nó ou pode conter *NULL* para indicar final da pilha.

A seguir temos o construtor de um nó:

```

1 //construtor
2 No * newNo(int valor){
3     No * no = (No *)malloc(sizeof(No));
4     no->valor=valor;
5     no->prox = NULL;
6     return no;
7 }
  
```

Ao final do construtor teremos a situação mostrada na Figura 4.6.

Abaixo a estrutura de uma pilha:

```

1 //Estrutura Pilha
2 typedef struct Pilha{
  
```



Figura 4.6: Exemplo da estrutura após a execução da função *newNo* à esquerda e após a execução do *newPilha* à direita.

```

3     No * topo;
4 }Pilha;

5

6 //Construtor
7 Pilha * newPilha(){
8     Pilha * p = (Pilha *)malloc(sizeof(Pilha));
9     p->topo=NULL;
10    return p;
11 }

```

Note que a pilha não precisa de um tamanho máximo, pois ela se ajusta aos dados. A seguir temos a função *push* utilizando pilha sobre lista ligada.

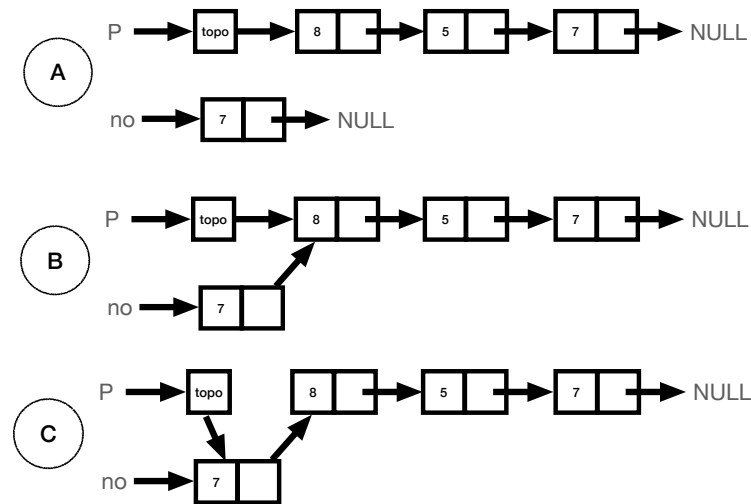
```

1 //empilha
2 void push(Pilha *p,int x){
3     No * no = newNo(x); //(a)
4     no->prox = p->topo; //(b)
5     p->topo = no; //(c)
6 }

```

Veja o passo a passo da função *push* na Figura 4.7.

Questão 44) (Função push) A função *push* a seguir está implementada de maneira errada. Suponha que a pilha possui os elementos ($5 \leftarrow 7 \leftarrow 3$) e que foi chamado a função *push(p,4)*. Mostre o estado da memória após executar esta função *push* que está errada.

Figura 4.7: Passo a passo da função *push*.

Comente o que aconteceu.

```

1 //empilha com erro.
2 void push2(Pilha *p,int x){
3     No * no = newNo(x);
4     p->topo = no;
5     no->prox = p->topo;
6 }
```

Questão 45) (Função push) Faça o mesmo para a função abaixo:

```

1 //empilha com erro.
2 void push3(Pilha *p,int x){
3     No * no = newNo(x);
4     no->prox = p->topo;
5     p->topo->prox = no;
6
7 }
```

Na sequência temos a função *pop*:

```

1 //desempilha
2 int pop(Pilha *p){
3     int valor;
4     if(p->topo==NULL){
5         printf("Pilha Vazia\n");
6         exit(1);
7     }
8     valor = p->topo->valor;
9     No * no = p->topo;
10    p->topo = p->topo->prox;
11    free(no);
12    return valor;
13 }

```

Veja o passo a passo da função *pop* na Figura 4.8.

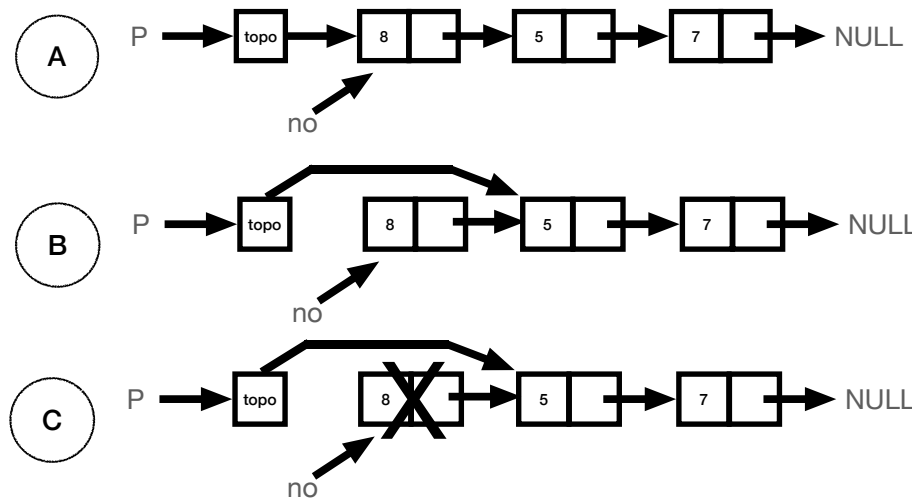


Figura 4.8: Passo a passo da função *pop*.

Questão 46) (Função pop) A função *pop* a seguir está implementada de maneira errada. Suponha que a pilha possui os elementos ($5 \leftarrow 7 \leftarrow 3$) e que foi chamado a função

pop(p). Mostre o estado da memória após executar esta função *pop* que está errada. Comente o que aconteceu.

```
1 //desempilha com erro
2 int pop(Pilha *p){
3     int valor;
4     if(p->topo==NULL){
5         printf("Pilha Vazia\n");
6         exit(1);
7     }
8     valor = p->topo->valor;
9     p->topo = p->topo->prox;
10    return valor;
11 }
```

A seguir temos a implementação da função print

```
1 void printPilha(Pilha *p){
2     printf("(");
3     No * aux = p->topo;
4     while(aux!=NULL){
5         printf("%d",aux->valor);
6         aux = aux->prox;
7         if(aux!=NULL){
8             printf("<-");
9         }
10    }
11
12    printf(")");
13    printf(" [%d]\n",size(p));
14 }
```

Veja o passo a passo da função *print* na Figura 4.9.

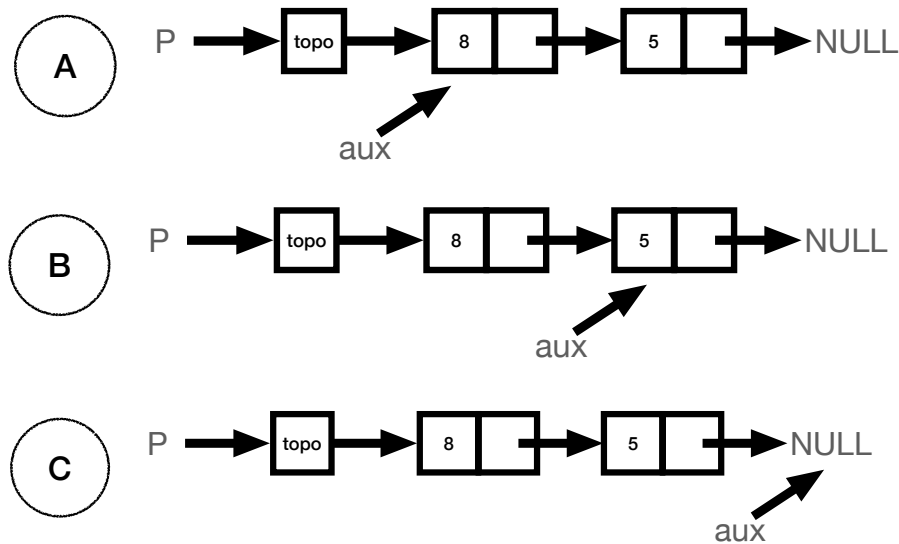


Figura 4.9: Passo a passo da função *print*.

Questão 47) (Função *print*) A função *print* a seguir está implementada de maneira errada. Suponha que a pilha possui os elementos ($5 \leftarrow 7 \leftarrow 3$) e que foi chamado a função *print(p)*. Comente o que acontece ao executar a função. Também descreva o que acontece se a pilha estiver vazia.

```

1 //funcao implementada de maneira errada
2 void printPilha(Pilha *p){
3     printf("(");
4     No * aux = p->topo;
5     while(aux->prox!=NULL){
6         printf("%d",aux->valor);
7         aux = aux->prox;
8         if(aux!=NULL){
9             printf("<-");
10        }
11    }

```

```
12
13     printf(")");
14     printf("[%d]\n", size(p));
15 }
```

Questão 48) (Função print) Proponha uma função recursiva que imprime uma pilha usando lista ligada, com o topo impresso à esquerda.

Questão 49) (Função print) Proponha uma função recursiva que imprime uma pilha usando lista ligada, com o topo impresso à direita.

Questão 50) (Função print) Proponha uma função iterativa que imprime uma pilha usando lista ligada, com o topo impresso à direita.

A função a seguir mostra o número de elementos numa pilha:

```
1 //numero de elementos na pilha
2 int size(Pilha *p){
3     int size=0;
4     No * aux = p->topo;
5     while(aux!=NULL){
6         aux = aux->prox;
7         size++;
8     }
9     return size;
10 }
```

Note que a função *size* na pilha sequencial gasta $\Theta(1)$ enquanto na pilha sobre lista ligada gasta $\Theta(n)$. A seguir temos uma implementação alternativa da função *size*:

```
1 //implementacao alternativa da funcao size
2 int size2(Pilha *p){
3     int size=0;
4     for(No * aux = p->topo; aux!=NULL; aux = aux->prox){
5         size++;
6     }
```

```
6     }
7     return size;
8 }
```

Neste implementação foi utilizado um *for* ao invés de um *while*. Este *for* sempre pode ser usado quando o objetivo for percorrer uma lista ligada.

Veja, a seguir, uma terceira forma de implementar a função *size*. Nesta função foi utilizada a estratégia de recursão:

```
1 //Size recursivo
2 int sizeRec(No *no){
3     if(no==NULL){
4         return 0;
5     }
6     return 1+sizeRec(no->prox);
7 }
8
9 //chamada do size recursivo
10 int size3(Pilha *p){
11     return sizeRec(p->topo);
12 }
```

Veja o passo a passo da função *size* recursivo na Figura 4.10.

Agora vamos ver como ficou a função para testar se um conjunto de parênteses está balanceado usando a pilha usando lista ligada.

```
1 //devolve 1 se os parenteses estao balanceados
2 //0 caso contrario
3 int balanceado(char *texto){
4     int size;
5     size = strlen(texto);
6     Pilha * p = newPilha();
```

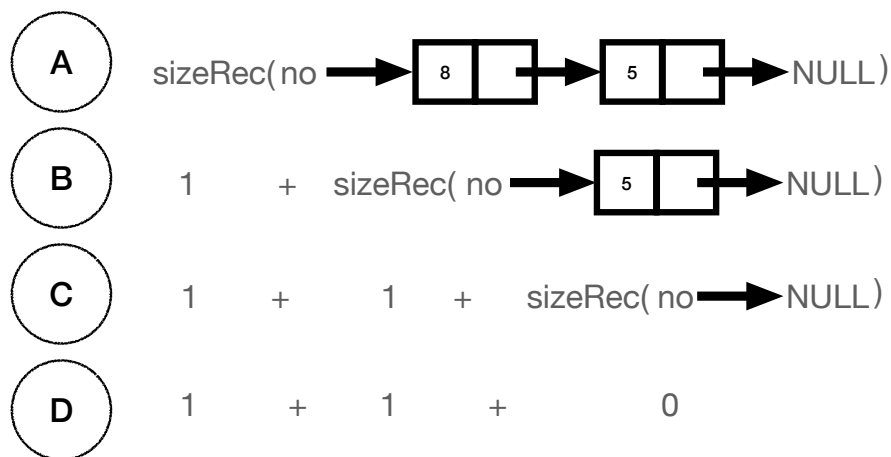


Figura 4.10: Passo a passo da função *size* recursivo.

```

7   for(int i=0;i<size;i++){
8       if(texto[i]=='('){
9           push(p,1);
10          //printPilha(p);
11      }
12      if(texto[i]==')'){
13          if(estaVazia(p)){
14              return 0;
15          }
16          pop(p);
17          //printPilha(p);
18      }
19
20  }
21  return estaVazia(p);
22 }
```

Veja que a implementação da pilha é totalmente diferente, porém o uso é praticamente idêntico. Isso quer dizer que é possível trocar a implementação mas manter o uso da estru-

tura. Este é o conceito de tipo abstrato de dados, que separa o uso de uma estrutura de sua implementação.

Questão 51) (Balanceamento) Considere três operadores para fins de balanceamento: chaves `{`, parênteses `(` e colchetes `[`. Veja alguns exemplos de *strings* balanceadas e não balanceadas:

- “`( ) [ ] { }`”. Balanceado
- “`( [ ) ]`”. Não balanceado. Neste caso houve um cruzamento entre os parênteses e os colchetes, o que não é permitido.
- `if(a[(i + 2)] > b[(i + 1)]) { printf(“%d”, (2 + 2) * 2); }` Balanceado.
- `if(a[(i + 2)] > b[(i + 1)]) { printf(“%d”, (2 + 2) * 2); }` Não Balanceado.

Sua tarefa é verificar se a *string* está ou não balanceada.

Para resolver este problema é necessário empilhar três símbolos diferentes, um para chaves, outro para colchetes e outro para parênteses. Ao chegar um fecha chaves, parênteses ou colchetes, é necessário fazer o *pop* da pilha e verificar se o símbolo que estava no topo da pilha é coerente com o símbolo referente ao fecha elemento.

Questão 52) Considere a questão anterior, onde é necessário avaliar o balanceamento de chaves, parenteses e colchetes. Além de tal balanceamento, existe dois códigos que representam início de comentário e fim de comentário. O início de comentário é marcado pelos símbolos `/*`, já o final do comentário é marcado pelos símbolos `*/`. Os caracteres dentro dos comentários devem ser ignorados. Veja os exemplos abaixo:

- “`( ) [ ] { } /* */`”. Balanceado
- “`/* ((( */`”. Balanceado
- “`/* /* */ */`”. Balanceado. Note que o abre comentário dentro do comentário é ignorado.
- “`/* /* */ */`”. Desbalanceado. O abre comentário dentro do comentário é ignorado. Assim, o segundo fecha comentário fica desbalanceado.
- “`/* a) empilha */ void push(int x) { v[top++] = x; }`”. Balanceado.

Estas regras de comentários são similares as regras da linguagem C, então você pode testar estas *strings* em um IDE para linguagem C.

Sua tarefa é verificar se uma string passada como parâmetro está ou não balanceada em relação a chaves, parênteses, colchetes e comentários.

A seguir temos uma implementação errada da função que libera a memória de uma pilha:

```

1 //funcao incorreta que libera memoria de uma pilha
2 void freePilha(Pilha *p){
3     No * aux = p->topo;
4     while(aux=NULL){
5         free(aux);
6         aux = aux->prox;
7     }
8 }
```

Questão 53) (Liberação de memória) Proponha uma função recursiva que libera a memória de uma pilha usando lista ligada.

Questão 54) (Liberação de memória) Proponha uma função iterativa que libera a memória de uma pilha implementada com lista ligada.

Questão 55) (Liberação de memória) Proponha uma função que libera a memória de uma pilha usando lista ligada por meio de sucessivas chamadas à função *pop*.

Uma forma alternativa de trabalhar com uma pilha é não utilizar a estrutura *Pilha* e trabalhar apenas com a estrutura nó. Neste caso, a pilha vazia é criada da seguinte maneira:

```

1     No * p = NULL;
```

Neste caso, para se fazer um *push* e um *pop* é necessário passar a pilha por referência, pois é necessário alterar seu conteúdo. veja como ficam as chamadas das funções:

```

1         push(&p, 1);
```

```

1         pop(&p);
```

E veja como ficam as implementações das funções. Função *push*

```

1 //empilha
2 void push(No **p, int x){
3     No * no = newNo(x);
```

```
4     no->prox = (*p);
5     (*p) = no;
6 }
```

E a função *pop*:

```
1 //desempilha
2 int pop(No **p){
3     int valor;
4     if((*p)==NULL){
5         printf("Pilha Vazia\n");
6         exit(1);
7     }
8     valor = (*p)->valor;
9     No * no = (*p);
10    (*p) = (*p)->prox;
11    free(no);
12    return valor;
13 }
```

As demais funções ficam praticamente idênticas, pois não é preciso alterar o conteúdo da pilha, sendo assim, não é necessário fazer a passagem por referência. Veja o exemplo da função *size*

```
1 //numero de elementos na pilha
2 int size(No *p){
3     int size=0;
4     No * aux = p;
5     while(aux!=NULL){
6         aux = aux->prox;
7         size++;
8     }
```

```
9     return size;  
10 }
```

Questão 56) (Implementação) Agora que você terminou a leitura desta aula sobre pilha, você deve ser capaz de implementar uma pilha do zero sem auxílio deste material. Você deve ser capaz de refazer todos as funções, como construtores, *push*, *pop*, *estaVazia*, *size*, *printPilha*, *balanceado* e a função *main*. Você deve fazer esta implementação sem auxílio tanto para a versão sequencial quanto para a versão baseada em lista ligada.

Capítulo 5

Aula 5

Você recebeu um código que implementa uma pilha usando lista ligada. Considere o código abaixo:

```
typedef struct No{
    int valor;
    struct No * prox;
}No;

typedef struct Pilha{
    No * raiz;
}Pilha;

void push(Pilha *p, int valor){
    No * no = newNo(valor);
    no->prox = p->raiz;
    p->raiz = no;
}

int pop(Pilha *p){
    if(isEmpty(p)){
        printf("Pilha Vazia\n");
```

```

        exit(6);
    }
    No * aux = p->raiz;
    p->raiz = p->raiz->prox;
    int valor = aux->valor;
    free(aux);
    return valor;
}

```

Este código insere um elemento na pilha, na posição raiz e remove da posição raiz.

Questão 57) Faça uma alteração no método *push*, chamado *push2*, que insira o novo nó na última posição da lista ligada. Veja a Figura 5.1

Questão 58) Implemente também o método *pop2* que remove o elemento da última posição da lista.

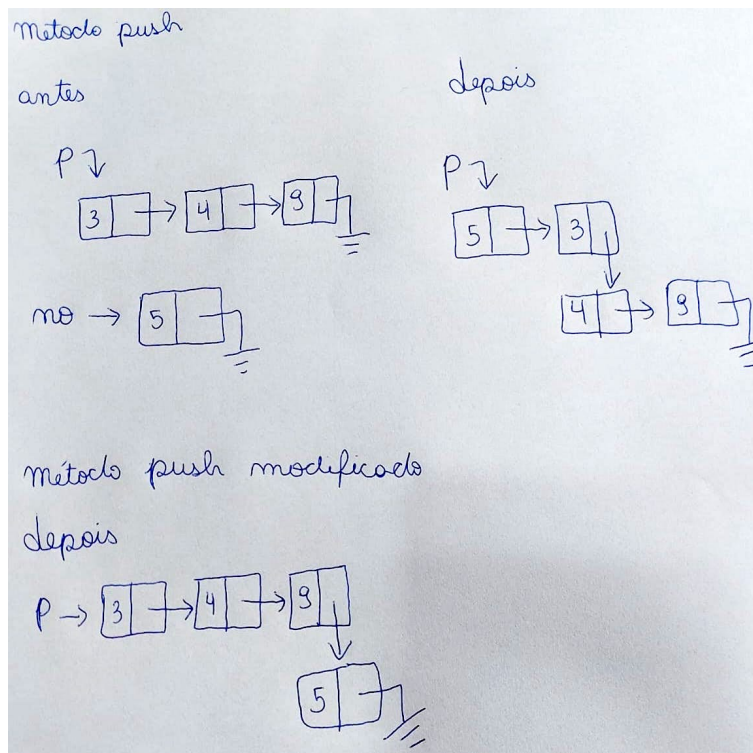


Figura 5.1: Push e Push modificado. No push padrão o novo nó é inserido na raiz. No push modificado, é inserido na última posição da lista ligada.

Questão 59) Faça uma alteração no método *push*, chamado *push3*. Se o vetor possuir 1 ou mais elementos, o elemento inserido deve estar na segunda posição da lista ligada.

Questão 60) Implemente também o método *pop3*. Se a lista possuir 2 ou mais elementos, o elemento removido deve estar na segunda posição da lista ligada.

5.1 Fila Usando Lista Ligada

Na aula passada vimos o conceito de pilha, que em inglês se chama *stack*. Pilha é uma estrutura onde se coloca elementos e se remove elementos e existe uma regra: o último a entrar é o primeiro a sair (*Last In First Out*—LIFO). Funcionamento semelhante a uma pilha de pratos, onde o prato é inserido sempre no topo e sempre removido no topo. Os elementos não precisam estar um ao lado do outro. Eles podem estar ao lado ou não estar. No caso de pilha sobre lista ligada, eles estão separados um do outro.

Na aula de hoje iremos ver uma fila. Na fila, temos que o primeiro a entrar é o primeiro a sair (*First In First Out* – FIFO). Imagine uma fila de supermercado. O primeiro a entrar na fila é o primeiro a sair da fila. O segundo a entrar na fila é o segundo a sair da fila, e assim sucessivamente. Outro exemplo de fila é a fila do açougue, onde as pessoas retiram uma senha. Neste segundo caso, os elementos não ficam um ao lado do outro. A sequência de elementos é controlada pela senha, mas a regra FIFO se mantém. Veja Figura 5.2.



Figura 5.2: Fila de supermercado à esquerda e fila baseada em senha à direita. No segundo caso, os elementos não ficam um ao lado do outro.

E se quiser acessar um elemento no meio da fila, inserindo ou removendo? Neste caso não

temos uma fila e sim uma lista. Na lista é possível inserir e remover em qualquer posição. Lista será o tema da próxima aula.

Um exemplo de fila é a fila de impressão, em que os elementos são inseridos seguindo uma ordem e esta ordem é seguida no momento da impressão. Uma variação deste problema é o uso de filas com prioridades. Suponha que professores tem prioridade na impressão comparado com alunos. Ou suponha uma fila de supermercado, onde existe uma fila prioritária para idosos, gestantes, etc.

Um problema que pode ser resolvido usando filas é o problema do produtor-consumidor. Neste problema, existe um processo gerando elementos e colocando numa fila, que é produtor. E existe outro processo que remove elementos da fila, que é o consumidor. Nesta aula iremos ver uma versão simplificada de produtor consumidor.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 //estrutura No
6 typedef struct No{
7     int valor;
8     struct No * prox;
9 }No;
10
11 //construtor
12 No * newNo(int valor){
13     No * no = (No *)malloc(sizeof(No));
14     no->valor=valor;
15     no->prox = NULL;
16     return no;
17 }
18
19 //Estrutura fila
```

```
20 typedef struct Fila{
21     No * ini;
22     No * fim;
23 }Fila;
24
25 //Construtor
26 Fila * newFila(){
27     Fila * fila = (Fila *)malloc(sizeof(Fila));
28     fila->ini=NULL;
29     fila->fim=NULL;
30     return fila;
31 }
32
33 int estaVazia(Fila *f){
34     return f->ini==NULL;
35 }
36
37 void add(Fila *f,int x){
38     No * no = newNo(x);
39     if(estaVazia(f)){
40         f->ini = no;
41         f->fim = no;
42     }else{
43         f->fim->prox = no;
44         f->fim = no;
45     }
46 }
47
48 int remover(Fila *fila){
49     int valor;
```

```
50     if(estaVazia(fila)){
51         printf("Fila Vazia\n");
52         exit(1);
53     }
54     valor = fila->ini->valor;
55     No * no = fila->ini;
56     if(fila->ini==fila->fim){
57         fila->ini=NULL;
58         fila->fim=NULL;
59     }else{
60         fila->ini = fila->ini->prox;
61     }
62     free(no);
63     return valor;
64 }

65
66 //numero de elementos na fila
67 int size(Fila *fila){
68     int size=0;
69     No * aux = fila->ini;
70     while(aux!=NULL){
71         aux = aux->prox;
72         size++;
73     }
74     return size;
75 }

76
77 void printFila(Fila *fila){
78     printf("[");
79     No * aux = fila->ini;
```

```
80     while(aux!=NULL){
81         printf("%d",aux->valor);
82         aux = aux->prox;
83         if(aux!=NULL){
84             printf(",");
85         }
86     }
87
88     printf("]");
89     printf("( %d)\n",size(fila));
90 }
91
92 int main(){
93     Fila * fila = newFila();
94     add(fila,3);
95     add(fila,7);
96     add(fila,9);
97     printFila(fila);
98
99     add(fila,10);
100    add(fila,1);
101    add(fila,5);
102    printFila(fila);
103
104    while(!estaVazia(fila)){
105        printf("Removeu %d\n",remover(fila));
106    }
107
108 }
```

A saída do programa será:

[3,7,9] (3)

[3,7,9,10,1,5] (6)

Removeu 3

Removeu 7

Removeu 9

Removeu 10

Removeu 1

Removeu 5

Agora vamos estudar alguns pontos deste código. Primeiramente, considere a estrutura *fila*.

```
1 //Estrutura fila
2 typedef struct Fila{
3     No * ini;
4     No * fim;
5 }Fila;
```

A Figura 5.3 mostra o uso da estrutura.

Na sequência temos o construtor da estrutura:

```
1 //Construtor
2 Fila * newFila(){
3     Fila * fila = (Fila *)malloc(sizeof(Fila));
4     fila->ini=NULL;
5     fila->fim=NULL;
6     return fila;
7 }
```

Este construtor instancia uma estrutura com início e fim apontando para *NULL*. A estrutura fica igual à primeira fila da Figura 5.3.

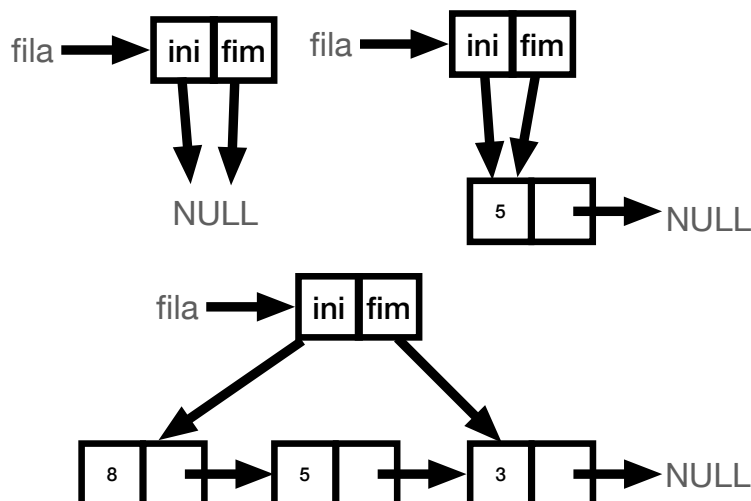


Figura 5.3: Estrutura fila com zero, um e três elementos.

A próxima função insere um elemento no final da fila. Segundo a lógica usada aqui, elementos são inseridos no final da fila e são retirados do início da fila.

```

1 void add(Fila *f, int x){
2     No * no = newNo(x);
3     if(estaVazia(f)){
4         f->ini = no;
5         f->fim = no;
6     }else{
7         f->fim->prox = no;
8         f->fim = no;
9     }
10 }
```

Primeiramente verifica se a fila está vazia (primeira fila da Figura 5.3). Em caso afirmativo, tanto o início quanto o fim apontam para o mesmo nó (segunda fila da Figura 5.3).

Caso a fila possua um ou mais elementos, o novo elemento é inserido no final da fila, conforme a Figura 5.4.

Na sequência temos a função remover:

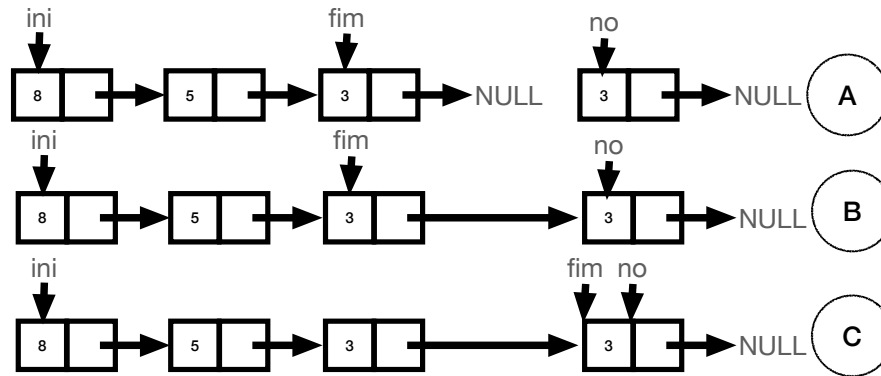


Figura 5.4: Em (a) é criado o nó. Em (b) é feito $fila \rightarrow fim \rightarrow prox = no$. Em (c) é feito $fila \rightarrow fim = no$.

```

1 int remover(Fila *fila){
2     int valor;
3     if(estaVazia(fila)){
4         printf("Fila Vazia\n");
5         exit(1);
6     }
7     valor = fila->ini->valor;
8     No * no = fila->ini;
9     if(fila->ini==fila->fim){
10         fila->ini=NULL;
11         fila->fim=NULL;
12     }else{
13         fila->ini = fila->ini->prox;
14     }
15     free(no);
16     return valor;
17 }
```

Se a fila está vazia, é dada uma mensagem de erro e o programa para de executar. Se a pilha tem um elemento, ela passa a ter zero elementos. Caso a fila tenha dois ou mais elementos são executados os passos mostrados na Figura 5.5.

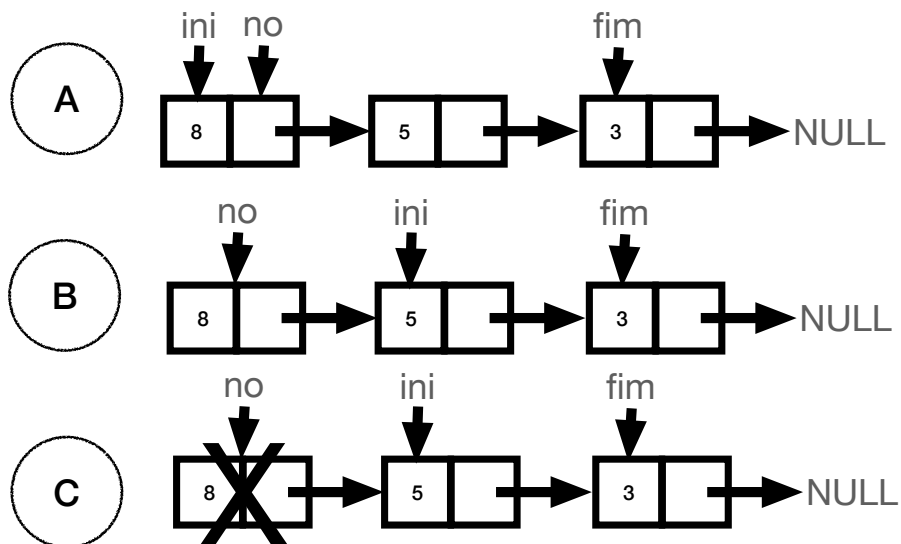


Figura 5.5: Em (a) é feito $no = fila \rightarrow ini$. Em (b) é feito $fila \rightarrow ini = fila \rightarrow ini \rightarrow prox = no$. Em (c) é feito $free(no)$.

As funções *size*, *printFila* apenas percorrem os elementos da fila.

```

1 //numero de elementos na fila
2 int size(Fila *fila){
3     int size=0;
4     No * aux = fila->ini;
5     while(aux!=NULL){
6         aux = aux->prox;
7         size++;
8     }
9     return size;
10 }
11
12 void printFila(Fila *fila){

```

```
13     printf("[");
14     No * aux = fila->ini;
15     while(aux!=NULL){
16         printf("%d",aux->valor);
17         aux = aux->prox;
18         if(aux!=NULL){
19             printf(",");
20         }
21     }
22
23     printf("]");
24     printf("(%d)\n",size(fila));
25 }
```

Questão 61) Altera a estrutura fila para que ela contenha além do início e do fim da fila, também um inteiro *size* que representa o número de elementos da fila. Altere a função construtora, para que *size* valha zero. Faça um *size++* no ao adicionar e um *size--* ao remover. Altere a função *size()* para que ela retorne o inteiro guardado dentro da estrutura. Faça chamadas às funções implementadas.

Vamos mostrar agora a implementação de um produtor consumidor

```
1 void produtor(Fila *f){
2     int tarefa = rand()%100;
3     add(f,tarefa);
4     printf("Add tarefa %d\n",tarefa);
5 }
```

Neste código é criada uma tarefa, que é um número aleatório entre 0 e 99 e este valor é adicionado na fila. Segue o código do consumidor:

```
1 void consumidor(Fila *f){
2     if(!estaVazia(f)){
```

```
3         int tarefa = remover(f);
4         printf("Processando tarefa %d\n",tarefa);
5     }
6 }
```

A função consumidor verifica se a fila não está vazia e remove uma tarefa. Tal tarefa é impressa na tela. A seguir a função executa:

```
1 void executa(){
2     Fila *filaTarefas = newFila();
3     for(int i=0;i<1000;i++){
4         int r = rand()%100;
5         if(r<40){
6             produtor(filaTarefas);
7         }else{
8             consumidor(filaTarefas);
9         }
10        printFila(filaTarefas);
11        sleep(1);
12    }
13 }
```

Nesta função repete-se 1000 vezes. Primeiro é sorteado um valor entre 0 e 99. Se o valor estiver entre 0 e 39, chama-se a função *produtor*, caso contrário, chama-se a função *consumidor*. Isto quer dizer que com 40% de probabilidade é chamada a função produtor e com 60% de probabilidade é chamada a função consumidor.

A cada passo é chamado a função *sleep*, que faz com que o programa pare de executar por 1s. Para que a função *sleep* funcione, é necessário importar a biblioteca *#include <unistd.h>*.

Suponha agora uma variação da fila, onde existem dois tipos de elementos: elementos prioritários e elementos normais. Na hora de se inserir um elemento na fila, é necessário colocá-lo na fila correta. Na hora de remover um elemento, primeiro verifica se a fila prio-

ritária não está vazia. Se não estiver vazia, remove-se o elemento da fila prioritária. Se a fila prioritária estiver vazia, retira-se o elemento da fila normal. Vamos implementar esta fila modificada. Primeiramente a estrutura:

```
1 typedef struct FilaPrioridade{
2     Fila * prioritario;
3     Fila * normal;
4 }FilaPrioridade;
```

A estrutura possui internamente duas filas, uma prioritária outra normal. Veja o construtor da estrutura:

```
1 FilaPrioridade * newFilaPrioridade(){
2     FilaPrioridade * fila = (FilaPrioridade *)
3         malloc(sizeof(FilaPrioridade));
4     fila->prioritario = newFila();
5     fila->normal = newFila();
6     return fila;
7 }
```

No construtor é feito a alocação da estrutura principal, depois são alocadas as duas filas internas da estrutura. A seguir a função de inserção na fila:

```
1 //adiciona na fila com prioridades
2 void addPri(FilaPrioridade *f,int x,int prioritario){
3     if(prioritario){
4         add(f->prioritario,x);
5     }else{
6         add(f->normal,x);
7     }
8 }
```

E a função de remoção:

```
1 //remocao na fila com prioridades
2 int removerPri(FilaPrioridade *f){
3     if(!estaVazia(f->prioritario)){
4         return remover(f->prioritario);
5     }else{
6         return remover(f->normal);
7     }
8 }
```

A seguir, a função imprime:

```
1 //imprime a fila com prioridades
2 void printFilaPri(FilaPrioridade *fila){
3     printf("Prioritarios\n");
4     printFila(fila->prioritario);
5     printf("Normal\n");
6     printFila(fila->normal);
7 }
```

A impressão da fila com prioridades imprime primeiramente a fila com prioridades e depois imprime a fila normal.

Veja a função que verifica se a fila está vazia:

```
1 int estaVaziaPri(FilaPrioridade * fila){
2     return estaVazia(fila->prioritario) &&
3     estaVazia(fila->normal);
4 }
```

Finalmente a função *main* chamando as demais funções:

```
1 int main(){
2     FilaPrioridade * fila = newFilaPrioridade();
```

```
3     int prioritario = 0;
4     int valor =3;
5     addPri(fila,valor,prioritario);
6     valor = 4;
7     addPri(fila,valor,prioritario);
8     prioritario = 1;
9     valor = 7;
10    addPri(fila,valor,prioritario);
11    prioritario = 0;
12    valor = 6;
13    addPri(fila,valor,prioritario);
14    prioritario = 1;
15    valor = 10;
16    addPri(fila,valor,prioritario);
17    printFilaPri(fila);
18    printf("----\n");
19    while(!estaVaziaPri(fila)){
20        printf("Removeu %d\n",removerPri(fila));
21    }
22 }
```

E a saída do programa:

Prioritários

[7,10] (2)

Normal

[3,4,6] (3)

Removeu 7

Removeu 10

Removeu 3

Removeu 4

Removeu 6

Questão 62) (Projeto de Implementação) Suponha agora que, ao invés de duas prioridades, existem muitas. A prioridade é um valor $prior = (0, 1, \dots, max)$, sendo que quanto menor $prior$, maior a prioridade. Crie uma estrutura fila com prioridades que internamente possui um vetor de filas. Na hora de inserir um elemento na fila, é passado o valor $prior$ e o elemento é inserido na fila correta, sendo uma fila para $prior=0$, uma fila para $prior=1$, uma fila para $prior=2$ até termos uma fila para $prior=max$.

Ao remover-se um elemento da fila, primeiro verifica se existe elemento na fila de prioridade zero. Se sim, remove-se um elemento dela. Se não, busca-se na próxima fila. Assim sucessivamente até remover-se elementos da fila com $prior = max$.

Faça a função que imprime a fila de prioridade e a função que verifica se a fila de prioridade está vazia.

Peça para o usuário digitar o valor max , que representa o número de prioridades. Após isso, faça um menu com 4 opções.

- Inserir elemento. Deve ser lido um inteiro e uma prioridade. Tal item deve ser inserido na fila com prioridades.
- Remover um elemento. Deve ser removido um elemento da fila, seguindo as prioridades e as posições na fila. Tal elemento deve ser impresso na tela.
- Imprimir a fila de prioridades.
- Sair.

5.2 Fila Usando Vetores

Vamos agora implementar uma fila usando vetores. Veja esta primeira implementação:

```
1 #include <stdio.h>
2
3 #define MAX 8
```

```
4 int v[MAX];
5 int ini=0,fim=0;
6
7 void add(int x){
8     v[fim++]=x;
9 }
10
11 int remover(){
12     int valor = v[ini];
13     v[ini]=0;
14     ini++;
15     return valor;
16 }
17
18 int estaVazia(){
19     return ini==fim;
20 }
21
22 int size(){
23     return fim-ini;
24 }
25
26 void printFila(){
27     printf("[");
28     for(int i=0;i<MAX;i++){
29         printf("%d",v[i]);
30         if(i<MAX-1){
31             printf(",");
32         }
33     }
```

```
34     printf("](%d)\n",size());
35 }
36
37 int main(){
38     for(int i=0;i<MAX;i++){
39         v[i]=0;
40     }
41     add(1);printFila();
42     add(2);printFila();
43     add(3);printFila();
44     remover();printFila();
45     remover();printFila();
46     add(4);printFila();
47     add(5);printFila();
48     add(6);printFila();
49 }
```

A saída será:

```
[1,0,0,0,0,0,0,0](1)
[1,2,0,0,0,0,0,0](2)
[1,2,3,0,0,0,0,0](3)
[0,2,3,0,0,0,0,0](2)
[0,0,3,0,0,0,0,0](1)
[0,0,3,4,0,0,0,0](2)
[0,0,3,4,5,0,0,0](3)
[0,0,3,4,5,6,0,0](4)
```

Esta implementação tem uma desvantagem de que, conforme os elementos são inseridos e removidos, tanto o *ini* quanto o *fim* são deslocados para a direita. Isso significa que, após um certo tempo de uso, eles vão sair do vetor. Uma estratégia para corrigir este problema

é, ao chegar na última posição do vetor, retornar para a primeira posição, como uma lista circular. Veja Figura 5.6.

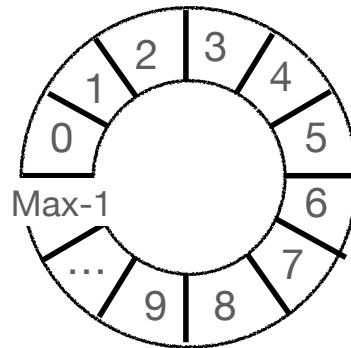


Figura 5.6: Uso de um vetor como uma lista circular. A última posição vem logo antes da primeira.

Veja como fica a implementação neste caso:

```
1 void add(int x){
2     v[fim]=x;
3     fim = (fim+1)%MAX;
4 }
5
6 int remover(){
7     int valor = v[ini];
8     v[ini]=0;
9     ini = (ini+1)%MAX;
10    return valor;
11 }
```

A função *size* também precisa ser refeita:

```
1 int size(){
2     if(fim>ini){
3         return fim-ini;
```

```
4     }else{
5         return fim-ini+MAX;
6     }
7 }
```

Veja a saída do programa, ajustando *MAX* para 5:

```
[1,0,0,0,0] (1)
[1,2,0,0,0] (2)
[1,2,3,0,0] (3)
[0,2,3,0,0] (2)
[0,0,3,0,0] (1)
[0,0,3,4,0] (2)
[0,0,3,4,5] (3)
[6,0,3,4,5] (4)
```

A impressão está sendo feita considerando o vetor inteiro, de zero até *MAX*. Note que o número 6 foi inserido na posição zero do vetor.

Para fazer uma implementação onde é possível criar várias listas ao mesmo tempo, sem usar variáveis globais, vai ser necessário colocar os elementos (*ini*, *fim*, *MAX*, *v*) dentro de uma estrutura. Veja como fica o código com esta implementação:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 //Estrutura fila
5 typedef struct Fila{
6     int *v;
7     int max;
8     int ini, fim;
9 }Fila;
10
```

```
11 //Construtor
12 Fila * newFila(int max){
13     Fila * fila = (Fila *)malloc(sizeof(Fila));
14     //alocando memoria e iniciando elementos com zero
15     //via calloc
16     fila->v = (int*)calloc(max, sizeof(int));
17     fila->ini=0;
18     fila->fim=0;
19     fila->max = max;
20     return fila;
21 }
22
23 int estaVazia(Fila *f){
24     return f->ini==f->fim;
25 }
26
27 void add(Fila *f,int x){
28     f->v[f->fim]=x;
29     f->fim=(f->fim+1)%f->max;
30     if(f->fim==f->ini){
31         printf("Fila Cheia\n");
32         exit(1);
33     }
34 }
35
36 int remover(Fila *f){
37     if(estaVazia(f)){
38         printf("Fila Vazia");
39         exit(1);
40     }
```

```
41     int valor = f->v[f->ini];
42     f->v[f->ini] = 0;
43     f->ini = (f->ini+1)%f->max;
44     return valor;
45 }
46
47 //numero de elementos na fila
48 int size(Fila *f){
49     if((f->ini)<(f->fim)){
50         return (f->fim)-(f->ini);
51     }else{
52         return (f->fim)-(f->ini)+(f->max);
53     }
54 }
55
56 void printFila(Fila *f){
57     printf("[");
58     for(int i=0;i<f->max;i++){
59         printf("%d",f->v[i]);
60         if(i<f->max-1){
61             printf(",");
62         }
63     }
64     printf("]");
65     printf("( %d)\n",size(f));
66 }
67
68 int main(){
69     Fila * f  = newFila(5);
70     add(f,1);printFila(f);
```

```
71     add(f,2);printFila(f);
72     add(f,3);printFila(f);
73     add(f,4);printFila(f);
74     remover(f);printFila(f);
75     remover(f);printFila(f);
76     remover(f);printFila(f);
77     add(f,5);printFila(f);
78     add(f,6);printFila(f);
79     add(f,7);printFila(f);
80     remover(f);printFila(f);
81     remover(f);printFila(f);
82     remover(f);printFila(f);
83 }
```

Veja a saída do programa:

```
[1,0,0,0,0] (1)
[1,2,0,0,0] (2)
[1,2,3,0,0] (3)
[1,2,3,4,0] (4)
[0,2,3,4,0] (3)
[0,0,3,4,0] (2)
[0,0,0,4,0] (1)
[0,0,0,4,5] (2)
[6,0,0,4,5] (3)
[6,7,0,4,5] (4)
[6,7,0,0,5] (3)
[6,7,0,0,0] (2)
[0,7,0,0,0] (1)
```

Os elementos são inseridos de maneira circular. Ao chegar no final da fila, volta-se para a primeira posição. O mesmo na hora de remover. Após remover o último elemento, passa-se

a remover o primeiro elemento.

Esta implementação tem um defeito. A fila fica cheia com $max - 1$ elementos. Não é possível inserir max elementos na fila. Fica como sugestão pensar em como melhorar este código para que a fila possa receber max elementos.

Fica como sugestão de estudo ao final desta aula reimplementar os códigos sem olhar para os códigos neste documento.

Capítulo 6

Aula 6

6.1 Revisão de apontadores

Considere uma estrutura Coordenada:

```
1 //Estrutura coordenada
2 typedef struct Coordenada{
3     int x,y;
4 }Coordenada;
```

Podemos criara variáveis estáticas do tipo coordenada:

```
1     Coordenada co1;
2     co1.x = 5;
3     co1.y = 6
```

Também podemos criar uma variável do tipo apontador para Coordenada:

```
1     Coordenada * co2;
2     co2 = &co1;
```

Veja que a variável é do tipo apontador para Coordenada e ela recebe o endereço de *co1*. O operador & na frente de uma variável devolve o seu endereço.

6.2 Listas Ligadas

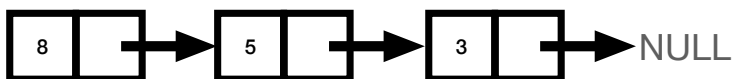
Diferente das pilhas e das filas, onde existe uma restrição para inserção e remoção de elementos, as listas permitem inserção e remoção em qualquer posição.

Na pilha temos o “último a entrar é o primeiro a sair”, (*last in first out* -LIFO). Na fila, a regra é “primeiro a entrar primeiro a sair”, (*first in first out*-FIFO). A fila tem um funcionamento análogo a uma fila de supermercado. Fila se usa em filas de impressão, fila de trabalhos, etc.

A terceira estrutura de dados que iremos ver é a lista. Na lista eu posso acessar os elementos do meio, tanto para leitura, inserção e remoção.

Vamos ver duas versões de lista: lista sequencial e lista ligada. Veja Figura 6.1

Lista Ligada



Lista Sequencial (vetor)

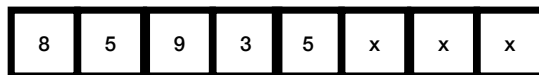


Figura 6.1: Exemplo de lista ligada e lista sequencial (vetor).

6.3 Listas Ligadas

As estruturas utilizadas para listas ligadas são *No* e *Lista* e os seus construtores.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct No {
5     int valor;
6     struct No * prox;
  
```

```
7 }No;

8

9 typedef struct Lista {
10     struct No * ini;
11 }Lista;

12

13 //construtor
14 No * newNo(int valor){
15     No * no = (No *)malloc(sizeof(No));
16     no->valor=valor;
17     no->prox = NULL;
18     return no;
19 }

20

21 //Construtor
22 Lista * newList(){
23     Lista * li = (Lista *)malloc(sizeof(Lista));
24     li->ini=NULL;
25     return li;
26 }
```

As funções *estaVazia*, *size*, *printLista* não sofreram mudanças em relação à fila:

```
1 int estaVazia(Lista *li){
2     return li->ini==NULL;
3 }

4

5 //numero de elementos na lista
6 int size(Lista *li){
7     int size=0;
8     No * aux = li->ini;
```

```
9     while(aux!=NULL){
10         aux = aux->prox;
11         size++;
12     }
13     return size;
14 }
15
16 void printLista(Lista *li){
17     printf("[");
18     No * aux = li->ini;
19     while(aux!=NULL){
20         printf("%d",aux->valor);
21         aux = aux->prox;
22         if(aux!=NULL){
23             printf(",");
24         }
25     }
26
27     printf("]");
28     printf("( %d)\n",size(li));
29 }
```

Vamos fazer agora uma função que insere um elemento no final da lista.

```
1 void addFim(Lista *li,int valor){
2     No * no = newNo(valor);
3     if(li->ini==NULL){
4         li->ini = no;
5     }else{
6         No * aux;
7         aux = li->ini;
```

```

8      while(aux->prox!=NULL){
9          aux = aux->prox;
10     }
11     aux->prox = no;
12 }
13 }

```

Como nesta estrutura não há um apontador para o final da lista, é necessário usar a variável *aux* e posicioná-la sobre o último nó. Depois disso, é só fazer $aux \rightarrow prox = no$. Veja Figura 6.2.

Questão 63) Qual a complexidade da função *addFim*?

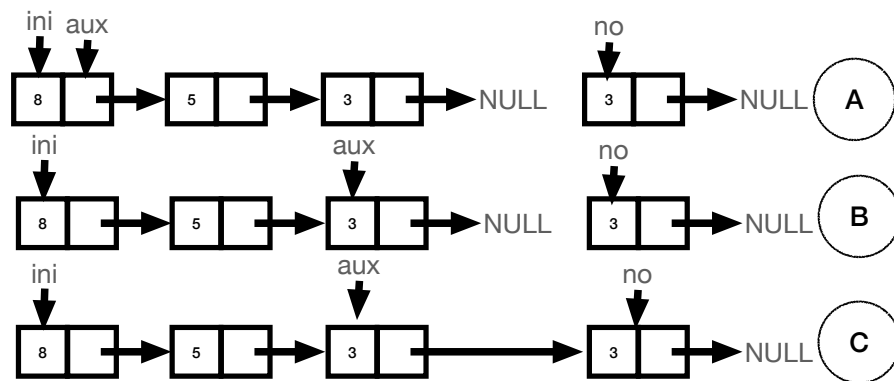


Figura 6.2: Nesta figura temos o passo a passo para inserir um elemento no final da lista ligada. Em (a) $aux = li \rightarrow ini$, e (b) *aux* vai para o final da lista. Em (c) $aux \rightarrow prox = no$.

A próxima função para se implementar é a *get* que obtém o elemento que está na posição *idx*.

```

1 //devolve um apontador para o elemento
2 //na posicao idx.
3 No * get(Lista *li, int idx){
4     No * aux = li->ini;
5     for(int i=0;i<idx;i++){
6         if(aux!=NULL)

```

```
7         aux = aux->prox;
8     }
9     if(aux==NULL){
10         printf("Fora dos limites");
11         exit(1);
12     }
13     return aux;
14 }
15
16 int main(){
17     Lista *li = newList();
18     addFim(li,3);
19     addFim(li,4);
20     addFim(li,9);
21     addFim(li,1);
22     addFim(li,4);
23     printLista(li);
24     int pos = 2;
25     No* no = get(li,pos);
26     printf("O elemento na pos. %d eh o %d\n",pos,no->valor);
27 }
```

Veja que com essa função podemos acessar um elemento que não está nas pontas da lista. Veja a saída do programa:

[3,4,9,1,4] (5)

O elemento na pos. 2 eh o 9

Questão 64) Qual a complexidade da função *get(li,idx)*?

A próxima função que iremos fazer é para remover um elemento. A Figura 6.3 mostra os passos para se remover um elemento de uma lista diferente do primeiro elemento.

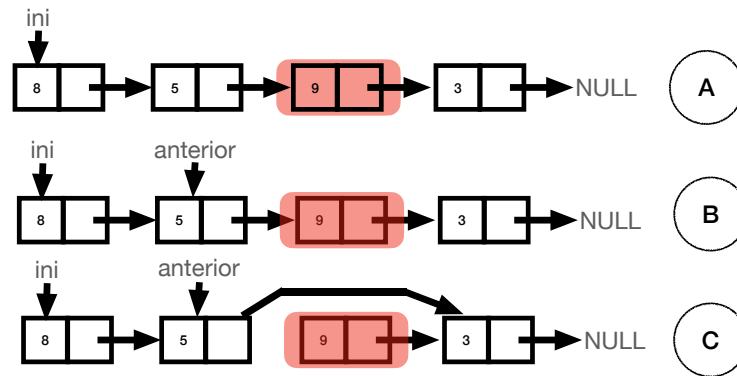


Figura 6.3: Passo a passo para se remover um elemento de uma lista que não seja o primeiro. É necessário encontrar o nó anterior ao que deseja-se remover e fazer um $anterior \rightarrow prox = anterior \rightarrow prox \rightarrow prox$.

Se for para se remover o primeiro elemento, basta fazer $li \rightarrow ini = li \rightarrow ini \rightarrow prox$.

Veja o código da remoção abaixo:

```

1 //remove o elemento da posicao idx
2 int remover(Lista *li, int idx){
3     No * no;
4     if(idx==0){
5         no = li->ini;
6         li->ini = li->ini->prox;
7     }else{
8         No * anterior = get(li,idx-1);
9         no = anterior->prox;
10        anterior->prox = anterior->prox->prox;
11    }
12    int valor = no->valor;
13    free(no);
14    return valor;
15 }
```

O próximo código substitui o elemento que está na posição *idx*.

```
1 //substitui o valor da posicao idx pelo
2 //parametro 'valor'
3 void set(Lista *li, int valor, int idx){
4     No * aux = get(li,idx);
5     aux->valor = valor;
6 }
```

A seguir é feita a inserção na posição *idx* e os elementos após *idx* são deslocados uma posição para a direita:

```
1 //insere o valor na posicao idx. Elementos apos
2 //esta posicao sao deslocados para a direita.
3 void add(Lista *li, int valor, int idx){
4     No * no = newNo(valor);
5     if(idx==0){
6         no->prox = li->ini;
7         li->ini = no;
8     }else{
9         No * anterior = get(li,idx-1);
10        no->prox = anterior->prox;
11        anterior->prox = no;
12    }
13 }
```

A Figura 6.4 mostra o passo a passo da função *add*.

Questão 65) Mostre qual o estado da memória após executar a função *add*, porém trocando-se a ordem das linhas 10 e 11. Suponha que primeiro foi executado $anterior \rightarrow prox = no$ e depois $no \rightarrow prox = anterior \rightarrow prox$.

A seguir temos a função *main*.

```
1 int main(){
```

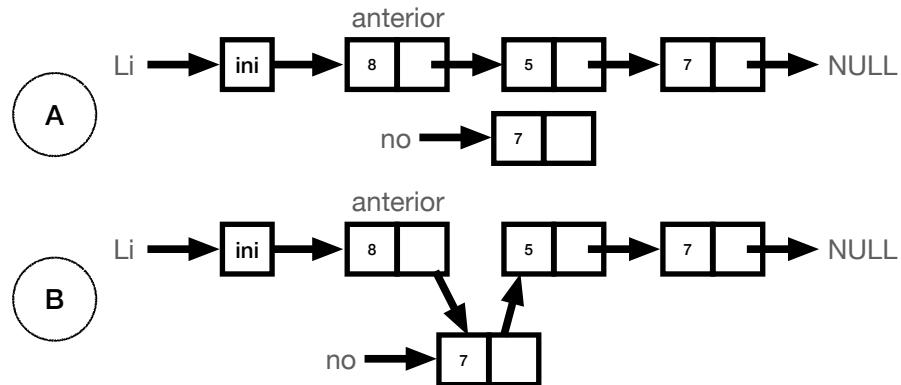


Figura 6.4: Função *add*. Primeiro $no \rightarrow prox = anterior \rightarrow prox$ e depois $anterior \rightarrow prox = no$

```

2   Lista *li = newList();
3   addFim(li,3);
4   addFim(li,4);
5   addFim(li,9);
6   addFim(li,1);
7   addFim(li,4);
8   printLista(li);
9   int pos = 2;
10  printf("Removeu elemento pos. %d = %d\n",pos,remover(li,pos));
11  printf("Lista apos remocao:\n");
12  printLista(li);
13  add(li,9,pos);
14  printf("Lista apos insercao:\n");
15  printLista(li);
16 }

```

Veja a saída do programa:

[3,4,9,1,4] (5)

Removeu elemento pos. 2 = 9

Lista apos remocao:

[3,4,1,4] (4)

Lista apos insercao:

[3,4,9,1,4] (5)

Questão 66) Faça uma função que inverte uma lista ligada. Você deve usar apenas três apontadores para fazer a inversão.

Questão 67) Faça uma função que recebe duas listas e concatena uma na outra. A segunda lista deve ser inserida no final da primeira .

Questão 68) Faça uma função que remove todos os elementos de uma lista que são iguais a uma dada chave k . A complexidade do algoritmo deve ser $\Theta(n)$.

Questão 69) Considere uma lista ligada onde existem duas possibilidades. Ou a lista termina aterrada (tipo A) ou o último elemento da lista aponta para um nó qualquer da lista (tipo B). Faça uma função que verifica se a lista é do tipo A ou B. Seu programa deve executar em $O(n)$ e usar memória adicional $O(1)$, onde n é o número de elementos da lista.

A Figura 6.5 mostra os dois tipos de lista:

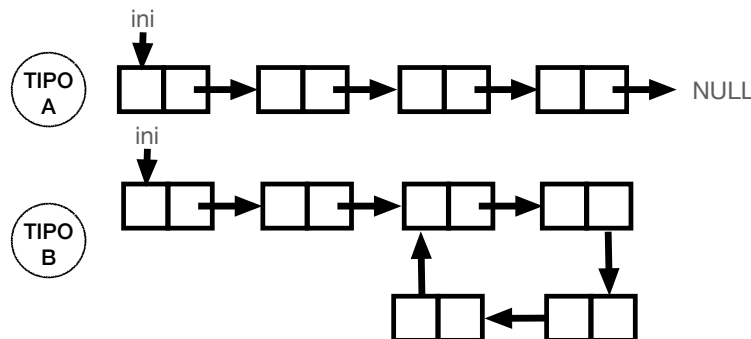


Figura 6.5: Dois tipos de lista, aterrada e não aterrada. Sua função de ser capaz de descobrir qual o tipo de uma dada lista.

O código abaixo busca um elemento dentro de uma lista:

```

1 int busca(Lista *li, int k){
2     for(int i=0;i<size(li);i++){
```

```
3         No * no = get(li,i);
4         if(no->valor==k){
5             return 1;
6         }
7     }
8     return 0;
9 }
10
11 int main(){
12     Lista *li = newList();
13     int pos=0;
14     add(li,3,pos);
15     add(li,4,pos);
16     add(li,9,pos);
17     add(li,1,pos);
18     add(li,4,pos);
19     printLista(li);
20     int k = 4;
21     printf("%d pertence a lista = %d\n",k,busca(li,k));
22
23 }
```

Veja a saída do programa:

[4,1,9,4,3] (5)

4 pertence a lista = 1

O código está intencionalmente implementado com complexidade acima esperado.

Questão 70) Qual a complexidade desta função?

Questão 71) Altere a complexidade para que o programa rode em $O(n)$.

A próxima função insere os elementos em uma lista de maneira ordenada. Suponha que temos uma lista com os elementos [3,4,8,11] e desejamos inserir um número. Se o número

for menor que o primeiro elemento, basta inseri-lo na primeira posição. Agora suponha que desejamos inserir o número 9. Temos, nesse caso, andar na lista e parar no maior elemento menor que 9. Após isso, basta fazer a inserção na posição seguinte. Veja a Figura 6.6.

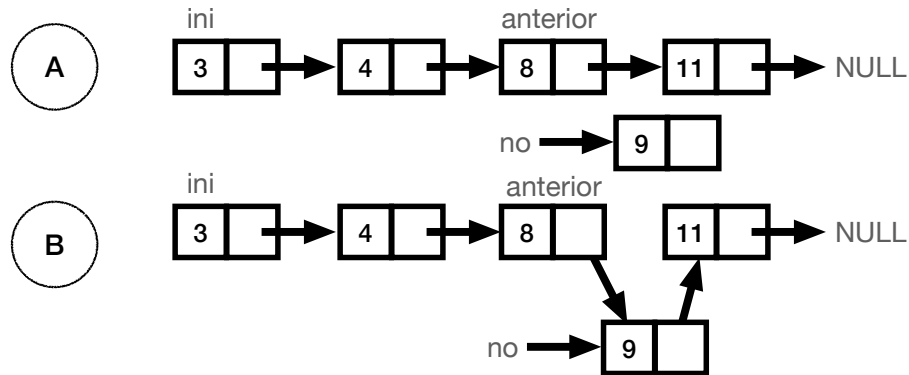


Figura 6.6: Exemplo de inserção ordenada de uma chave. Para fazer a função é necessário encontrar o maior nó menor do que a chave, caso exista.

O código abaixo insere os elementos na lista de maneira ordenada:

```

1 //insere o valor na lista de maneira ordenada
2 void addOrdenado(Lista *li, int valor){
3     No * no = newNo(valor);
4     if(li->ini==NULL || valor<=li->ini->valor){
5         no->prox = li->ini;
6         li->ini = no;
7     }else{
8         No * aux = li->ini;
9         while(aux->prox!=NULL && aux->prox->valor<valor){
10             aux = aux->prox;
11         }
12         no->prox = aux->prox;
13         aux->prox = no;
14     }
15 }
```

```
16
17
18 int main(){
19     Lista *li = newList();
20     int pos=0;
21     addOrdenado(li,3);
22     addOrdenado(li,4);
23     addOrdenado(li,9);
24     addOrdenado(li,1);
25     addOrdenado(li,4);
26     printLista(li);
27 }
```

Veja a saída do programa:

[1,3,4,4,9] (5)

Questão 72) Faça um programa que recebe uma lista ligada e a ordena. Você deve tomar o cuidado de não criar nós de maneira desnecessária. Deve usar apenas os nós da própria lista.

Este algoritmo de ordenação proposto tem uma complexidade que pode variar de acordo com os dados de entrada. Se os dados estiverem ordenados do maior para o menor, será a complexidade de melhor caso. Se eles estiverem ordenados do menor para o maior, será a complexidade de pior caso. Em situações como estas, estamos interessados em complexidade de pior caso.

Questão 73) Qual a complexidade do algoritmo de ordenação no pior caso?

Agora vamos ver uma lista circular. Nesta lista, o último elemento aponta para o primeiro. Veja A Figura 6.7.

Questão 74) Implemente as seguintes funções, utilizando uma lista circular:

```
1 int estaVazia(Lista *li){
2 }
```

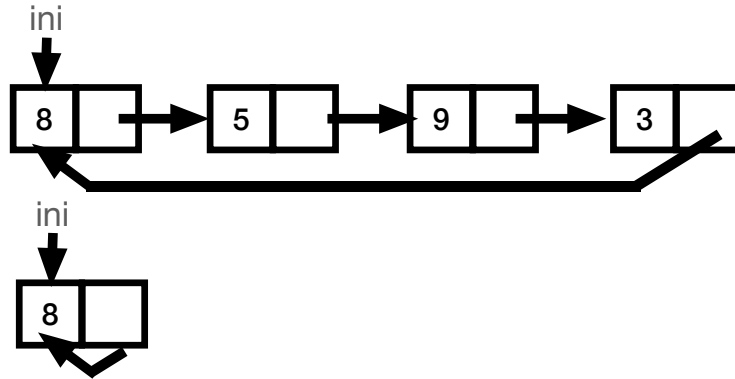


Figura 6.7: Exemplo de uma lista circular com quatro elementos e uma lista circular com um elemento.

```

3
4 //numero de elementos na lista
5 int size(Lista *li){
6 }
7
8 void printLista(Lista *li){
9 }
10
11 void addFim(Lista *li,int valor){
12 }
13
14 //obtem o elemento na posicao idx
15 No * get(Lista *li, int idx){
16 }
17
18 //remove o elemento da posicao idx
19 int remover(Lista *li, int idx){
20 }
21

```

```
22 //substitui o valor da posicao idx pelo
23 //parametro 'valor'
24 void set(Lista *li, int valor, int idx){
25 }
26
27 //insere o valor na posicao idx. Elementos apos
28 //esta posicao sao deslocados para a direita.
29 void add(Lista *li, int valor, int idx){
30 }
31
32 void addOrdenado(Lista *li, int valor){
33 }
34
35 //chamada das funcoes implementadas
36 int main(){
37 }
```

O último tema da aula é lista duplamente ligada. Em tais listas existem dois apontadores, um para frente (*prox*) e um para trás (*prev*).

```
1 //Lista duplamente ligada
2 typedef struct No {
3     int valor;
4     struct No * prox;
5     struct No * prev;
6 }No;
```

Veja Figura 6.8.

O interessante desta lista é que, de um elemento, é possível ir tanto para frente quanto para trás. Para remover um elemento, na lista simples, era preciso ter referência do elemento anterior. Nesta lista não. Basta ter uma referência do próprio elemento.

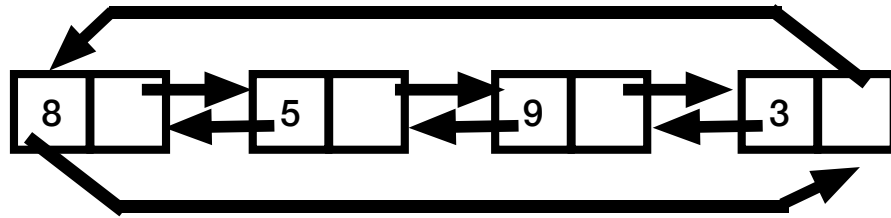


Figura 6.8: Estrutura de uma lista duplamente ligada.

A seguir, a função que insere um elemento na lista:

```

1 void add(Lista *li, int valor){
2     No * no = newNo(valor);
3     if(li->ini==NULL){
4         li->ini = no;
5         no->prox = no;
6         no->prev = no;
7     }else{
8         no->prox = li->ini->prox;
9         no->prev = li->ini;
10        li->ini->prox->prev = no;
11        li->ini->prox = no;
12    }
13 }
```

Questão 75) Faça um desenho mostrando, passo a passo, as operações executadas na função *add* em uma lista duplamente ligada.

Questão 76) Faça uma função que remove um elemento de uma lista duplamente ligada. Tal função recebe como parâmetro um apontador para o nó que será removido.

Questão 77) Suponha agora que a lista está representada por meio de um vetor. Faça a função que insere um elemento em uma dada posição de um vetor, empurrando os elementos subsequentes uma posição para frente. Faça também a função que remove um elemento de

uma lista guardada em um vetor. Qual a complexidade destas funções?

Capítulo 7

Aula 7

A Aula 7 será a prova. A seguir uma lista de exercícios para preparação para a prova.

Questão 78) Crie uma estrutura *Círculo*. O círculo deve ter dois parâmetros. Uma coordenada que representa o centro e um raio. Crie uma função construtor para a estrutura círculo. Faça uma função *main* com a chamada para a função implementada. Suponha que a função *newCoordenada* já está implementada, bem como a estrutura *Coordenada*.

Questão 79) Complete a frase: $f(n)$ é $\Theta(g(n))$ se e somente se...

Considere uma lista ligada com a estrutura mostrada abaixo. Estas estruturas dizem respeito às próximas 7 questões.

```
1 typedef struct No {
2     int valor;
3     struct No * prox;
4 }No;
5
6 typedef struct Lista {
7     No * ini;
8 }Lista;
```

Suponha que as funções *newNo*, *newLista* e *Print* já estão implementadas.

Questão 80) Crie uma função que adiciona um valor na última posição da lista.

```
1 void addFim(Lista *li, int valor){
```

Questão 81) Crie uma função que, dada uma lista, encontra a posição do maior elemento da lista. Por exemplo, se a lista for $[1, 3, 9, 5, 7]$ esta função devolve o valor 2 porque o primeiro elemento está na posição zero, o segundo está na posição 1 e o terceiro está na posição 2, sendo que este é o maior elemento. Se a lista estiver vazia, a função deve devolver o valor -1.

```
1 int posMax(Lista *li){
```

Questão 82) Crie uma função remover conforme assinatura abaixo:

```
1 No * remover(Lista *li, int pos){
```

Esta função remove da lista o nó que está na posição pos . Por exemplo, se a lista for $[1, 7, 4]$ e $pos = 0$, remove o primeiro item da lista. Se $pos = 1$, remove o segundo item da lista e se $pos = 2$ remove o terceiro item da lista. O nó removido deve ter seu apontador para $prox$ atualizado com NULL e deve receber um *return* da função.

Questão 83) Crie uma função ordena lista com as seguintes características. Primeiramente a função cria uma lista auxiliar, numa variável chamada $li2$. Enquanto a lista não estiver vazia, faça: descubra a posição do maior elemento, fazendo uma chamada da função $posMax$. Em seguida remova este elemento da lista, fazendo uma chamada à função $remove$. O elemento removido deve ser inserido na primeira posição das lista auxiliar $li2$. Não deve ser chamado uma função add . A inserção no início da lista $li2$ deve ser feito no corpo da função ordena. Também não deve ser criado nenhum nó novo. O nó que foi removido da lista li deve ser inserido na lista $li2$. Ao final a lista $li \rightarrow ini$ deve receber a lista $li2 \rightarrow ini$.

Questão 84) Qual a complexidade da função ordena? Suponha que a função ordena gastou 10s para ordenar um vetor com 100.000 elementos. Baseado nisto defina a função tempo; encontre a constante da função tempo;

Questão 85) Estime o tempo necessário para se ordenar uma lista com 1 milhão de elementos;

Questão 86) Suponha um computador 10x mais rápido que o computador da questão anterior. Quantos elementos podem ser ordenados em um tempo igual a 10s? Considere $\sqrt{10} = 3.16$. Calcule o valor final, não deixe apenas indicada a conta.

Questão 87) Faça uma função que recebe um nó e faz a desalocação da lista iniciada por este nó.

Questão 88) Proponha uma função recursiva que imprime os valores de uma lista ligada, com o início impresso à esquerda. Esta função recebe como parâmetro um apontador para nó.

Questão 89) Crie uma estrutura aluno com nome, RA e CR. O nome pode ser um vetor de caracteres com 100 posições, o RA é um inteiro e o CR é um ponto flutuante. Crie um construtor para aluno. O construtor recebe como parâmetros o nome o RA e o CR.

Questão 90) Crie uma função que imprime um aluno. Veja um exemplo de impressão do aluno:

[João,13123,0.64]

Crie uma constante *MAX* valendo 100, um vetor *Aluno * v[MAX]*, uma variável *top* que inicia com zero, todos variáveis globais. Faça uma função que adiciona um aluno no vetor e uma função que remove um aluno do vetor, ambas com funcionamento de pilha (*last in first out*). Para adicionar, é preciso analisar se a pilha não está cheia. Para remover, é preciso verifica se a pilha não está vazia.

Veja as assinaturas das funções:

```
1 void addPilha(Aluno *a){
```

```
1 Aluno * desempilha(){
```

Questão 91) Faça uma função que imprime os registros da pilha. Veja um exemplo de impressão:

[Carlos,141,0.74] [Maria,131,0.64] [João,121,0.54]

O topo da pilha deve ser impresso à esquerda.

```
1 void imprimePilha(){
```

Faça a função *main* chamando as funções que foram implementadas.

Considere as estruturas abaixo:

```
1 typedef struct No {
2     int valor;
3     struct No * prox;
4 }No;
5
6 typedef struct Lista {
7     No * ini;
8 }Lista;
9
10 No * newNo(int valor){
11     No * no = (No *)malloc(sizeof(No));
12     no->valor = valor;
13     no->prox = NULL;
14     return no;
15 }
16
17 Lista * newList(){
18     Lista * li = (Lista *)malloc(sizeof(Lista));
19     li->ini = NULL;
20     return li;
21 }
```

Questão 92) Considere uma função *divide* que recebe como parâmetro três listas li , li_a , li_b . A lista li possui n elementos, enquanto li_a e li_b estão vazias. Você deve fazer o seguinte algoritmo:

Enquanto *li* não estiver vazia:

remover o primeiro nó de *li*.

Este nó deve ser inserido no início de *li_a* e *li_b* alternadamente.

Veja a assinatura da função:

```
1 void divide(Lista *li, Lista *lia, Lista *lib){
```

Veja um exemplo de resultado do programa:

[0,1,2,3,4,5,6,7,8,9]

[8,6,4,2,0]

[9,7,5,3,1]

A primeira lista é a lista original *li*. A segunda é *li_a* e a terceira é *li_b*. Atenção, para implementar a função dividir, você **não deve** chamar qualquer outra função existente. O problema deve ser resolvido todo dentro da função *divide*. Também não deve usar *newNo* ou *malloc*. Deve trabalhar nos nós que já existem.

Questão 93) Faça uma função que calcula o tamanho de uma lista. Veja a assinatura da função:

```
1 int size(Lista *li){
```

Questão 94) Faça uma função que encontra um elemento dentro de uma lista a partir de sua posição. Veja a assinatura da função:

```
1 No * get(Lista *li, int idx){
```

Questão 95) A seguinte função verifica se uma chave *k* pertence a uma lista com complexidade $O(n^2)$. Altere o código para que a função seja $O(n)$.

```
1 int busca(Lista *li, int k){
2     for(int i=0;i<size(li);i++){
3         No * no = get(li,i);
4         if(no->valor==k){
```

```

5         return 1;
6     }
7 }
8     return 0;
9 }
```

Se as funções tem tempo $T_1(n) = n^2 10^{-6}s$ e $T_2(n) = n 10^{-6}s$, calcule o tempo das duas funções para $n = 1.000$ e para $n = 1.000.000$.

Questão 96) Algoritmos com complexidade exponencial, em geral, são muito ruins. Algoritmos logarítmicos são, em geral, muito bons. Algoritmos polinomiais são intermediários. Proponha um algoritmo $\Theta(n \lg n)$, um algoritmo $\Theta(\log n)$ e um algoritmo $\Theta(2^n)$. O algoritmo $\Theta(2^n)$ não pode usar *for* or *while*. Deve ser uma algoritmo recursivo.

Questão 97) Explique o funcionamento da fila utilizando vetores de maneira circular. Mostre como devem ser as funções para inserir e remover nesta estrutura de dados.

Questão 98) Suponha que você implementou um algoritmo que executa em tempo $f(n)$. Suponha também que $n = 1.000.000$. Descubra quanto tempo o algoritmo irá demorar para executar para as funções $f(n)$ abaixo:

- $f(n) = n 10^{-6}s$;
- $f(n) = n^2 10^{-6}s$;
- $f(n) = 10^n 10^{-6}s$
- $f(n) = (\log_{10} n) 10^{-6}$

Questão 99) Suponha que um algoritmo executa em tempo $T(n) = n^3 c$ segundos. Para $n = 1.000$ gastou 1 segundo.

- Descubra a constante de tempo c .
- Calcule o tempo que o algoritmo irá gastar para executar $n = 10.000$.
- Suponha agora que o computador foi trocado por outro 10 vezes mais rápido. Qual o tempo irá gastar para executar $n = 10.000$ neste novo computador.

Questão 100) Considere uma estrutura *Coordenada* conforme visto na apostila. Suponha que o construtor da estrutura já está implementado, com a seguinte assinatura:

```
1 //Construtor da estrutura
2 Coordenada * newCoordenada(int x,
3                               int y){
```

Considere agora uma estrutura *Circulo* abaixo:

```
1 typedef struct Circulo{
2     double raio;
3     Coordenada *centro;
4 }Circulo;
```

Implemente o construtor da estrutura círculo. Veja a assinatura abaixo:

```
1 Circulo * newCirculo(double raio,
2                       int xcentro, int ycentro){
```

Você deve necessariamente fazer uma chamada à função *newCoordenada*. Além disso, deve fazer a alocação dinâmica da estrutura e depois colocar conteúdo nos campos *raio* e *centro*. Por fim deve devolver o endereço da estrutura criada. Veja a função *main* chamando o construtor:

```
1 int main(){
2     double raio = 3.0;
3     int xcentro = 1;
4     int ycentro = 2;
5     Circulo * c = newCirculo(raio,
6                               xcentro, ycentro);
7 }
```

Questão 101) Qual a complexidade do algoritmo da função f_3 em função de n .

```
1 #include <stdio.h>
```

```
2 long f1(int n){
3     long x = 0;
4     for(int i=0;i<n;i++)
5         x++;
6     return x;
7 }
8 long f2(int n){
9     long x=0;
10    for(int i=0;i<n;i++){
11        for(int j=0;j<n;j++){
12            x++;
13        }
14    }
15    return x;
16 }
17 long f3(int n){
18     long x = 0;
19     for(int i=0;i<n;i++){
20         x+=f1(n);
21         x+=f1(n);
22         x+=f2(n);
23         x+=f2(n);
24         x+=f2(n);
25     }
26     return x;
27 }
28 int main(void) {
29     int n;
30     printf("Digite n:");
31     scanf("%d",&n);
```

```

32     printf("f3=%ld\n",f3(n));
33 }

```

Questão 102) (a) Faça um programa em C que soma duas matrizes quadradas $n \times n$.
 Você deve fazer a seguinte soma: $A_{n \times n} = B_{n \times n} + C_{n \times n}$.

```

1  #include <stdio.h>
2  #define MAX 100
3  void soma(int A[MAX][MAX],
4           int B[MAX][MAX],
5           int C[MAX][MAX],
6           int n){
7
8
9
10 }
11 int main(void) {
12     int X[MAX][MAX];
13     int Y[MAX][MAX];
14     int Z[MAX][MAX];
15     int n;
16     printf("Digite n:");
17     scanf("%d",&n);
18     for(int i=0;i<n;i++){
19         for(int j=0;j<n;j++){
20             Y[i][j]=1;
21             Z[i][j]=2;
22         }
23     }
24     soma(X,Y,Z,n);
25     for(int i=0;i<n;i++){

```

```
26         for(int j=0;j<n;j++){
27             printf("%d ",X[i][j]);
28         }
29         printf("\n");
30     }
31 }
```

(b) Qual a complexidade de seu programa (da parte que você implementou).

Questão 103) Considere a pilha implementada na apostila, usando vetor, ou seja, pilha sequencial. Suponha que as assinatura abaixo já estão implementadas.

```
1 Pilha * newPilha(int max){
```

```
1 void push(Pilha *p,int x){
```

```
1 void printPilha(Pilha *p){
```

Crie uma função *main* com as seguintes características:

1. crie três pilhas *p1*, *p2* e *p3*, cada uma com capacidade máxima de 100 elementos.
2. Leia uma tarefa do teclado. A tarefa deve ser uma variável int.
3. Enquanto a tarefa for maior que zero
 - (a) Se a tarefa for múltipla de 3, coloque na pilha *p1*.
 - (b) Se a tarefa dividido por 3 der resto 1, coloque na pilha *p2*.
 - (c) Se a tarefa dividido por 3 der resto 2, coloque na pilha *p3*.
 - (d) Leia a uma tarefa do teclado.
4. Ao final, é necessário imprimir as três pilhas.

Para as próximas duas questões, considere a estrutura abaixo:

```

1 typedef struct No {
2     int valor;
3     struct No * prox;
4 }No;

```

Considere uma função que recebe uma lista ligada e uma chave k e conta quantas vezes a chave k ocorre dentro da lista ligada. Para fins de simplificação, a estrutura recebida pela função é um apontador para nó, pode ser interpretado como uma lista ligada. Veja a assinatura da função:

```

1 int conta(No* no, int k){

```

Questão 104) Implemente a função *conta* de maneira iterativa, ou seja, percorrendo cada nó e fazendo a soma toda vez que encontrar uma chave k .

Questão 105) Implemente a função *conta* de maneira recursiva. Sua função deve testar se recebeu um nó nulo. Neste caso devolve zero. Caso contrário, faz uma chamada recursiva na lista sem o primeiro elemento e soma 1 se o primeiro elemento for igual a k .

Questão 106) Considere a estrutura abaixo para representar uma pilha:

```

1 wtypedef struct Pilha{
2     int *v;
3     int max,top;
4 }Pilha;

```

Implemente a função *push*.

Questão 107) Considere os código abaixo:

```

1 typedef struct No{
2     int valor;
3     struct No * prox;
4 }No;
5 No * newNo(int valor){

```

```
6     No * no = (No *)malloc(sizeof(No));
7     no->valor=valor;
8     no->prox = NULL;
9     return no;
10 }
11 typedef struct Pilha{
12     No * topo;
13 }Pilha;
```

Implemente o código da função *push* que insere um novo elemento na pilha. Segue a assinatura da função:

```
1 void push(Pilha *p,int x){
```

Capítulo 8

Aula 8

8.1 Ordenação $O(n^2)$

Ordenação é um dos problemas mais estudados da computação. É um tema quente em processos seletivos de empresas. O problema de ordenação tem como entrada um conjunto de elementos que deseja-se ordenar. Estes elementos podem ser inteiros, mas podem ser qualquer outra coisa, como *strings*, *pontos flutuantes* ou estruturas mais complexas.

A ordenação pode ser em ordem crescente ou decrescente. A sequência pode conter números repetidos. É mais tradicional ordenar um vetor que uma lista ligada, mas os dados podem estar no formato de lista ligada também.

O problema da ordenação consiste em receber uma sequência de elementos e fazer uma permutação de maneira que a sequência final contenha os elementos em ordem crescente¹. Veja Figura 8.1.

Após a ordenação, um elemento precisa ser menor igual ao próximo e maior igual ao anterior. Eu estou supondo a seguinte propriedade: se $a \leq b$ e $b \leq c$ então $a \leq c$. É possível pensar situações específicas onde esta propriedade não vale.

Pense em uma partida de futebol e suponha que $a < b$ significa que o time a perdeu para o time b . Nesta situação, podemos ter $a < b$ e $b < c$ e isso não implica que $a < c$.

¹O termo ordem crescente é um pouco impreciso, pois podem haver elementos repetidos. Por exemplo, ao ordenar um vetor com todos os elementos iguais, não se obtém uma ordem crescente. O mais correto é dizer ordem “não decrescente”.

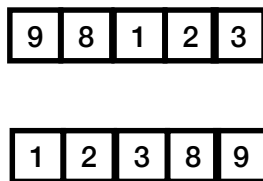


Figura 8.1: Problema da ordenação. Primeiro vetor está desordenado e o segundo está ordenado.

Apesar disso, nos problemas que nós iremos trabalhar, temos que se $a < b$ e $b < c$ então $a < c$.

Outra situação que pode gerar confusão é ordenação de *strings* versus ordenação de inteiros. No caso de *string* é utilizada a tabela ASCII, que possui um número para representar cada caracter. Pela tabela ASCII, o que é menor: “João” ou “joão”? Veja que neste caso, a diferença é apenas uma maiúscula e uma minúscula. Pela tabela ASCII, maiúsculo vem antes de minúsculo.

Suponha que estamos comparando uma string “Ana” com outra “Bicicleta”. Neste caso é comparado o primeiro caracter de “Ana” que é “A” com o primeiro caracter de “Bicicleta” que é “B”. Sendo assim, “Ana” vem antes de “Bicicleta”.

Suponha agora que estamos comparando “100” e “2” no formato *string*. O primeiro caracter de “100” é “1”, que vem antes do caracter “2”. Desta maneira, a ordenação em ordem alfabética vai colocar “100” antes de “2”.

Suponha agora que desejamos ordenar uma estrutura chamada *Aluno* que possui $(Nome, RA, CP, CR)^2$. Qual critério deve ser usado na ordenação?

A resposta é que depende. A ordenação pode ser feita por qualquer um dos critérios, segundo a necessidade no momento. Posso precisar da ordenação por nome, ou por *CR*, do maior para o menor, ou por *RA*, ou por *CP* do menor para o maior. É preciso definir qual é o critério de comparação. Também é preciso definir critérios de desempate. Caso dois alunos

²RA é registro acadêmico. É um número que identifica unicamente o aluno. CP é coeficiente de progressão e representa a porcentagem do curso que já foi cumprida pelo aluno. CR é o coeficiente de rendimento, uma média ponderada das notas do aluno.

tenham o mesmo CR eu posso usar o *Nome* para desempatar. RA não tem empate, pois é uma chave única.

Outro ponto em ordenação é que pode haver ordenação com mais de um critério. Por exemplo, ao comprar um carro, podemos analisar seu preço, quando menor melhor, e um índice de desconforto, também quanto menor, melhor. Se um carro é mais barato e menos desconfortável, ele é superior a outro. Entretanto, existem situações não comparáveis. O que é melhor? Um Fiat Uno ou uma Ferrari? A Ferrari tem mais conforto, mas é mais cara. Estes dois carros são não comparáveis. Situações como estas levam ao que chamamos de fronteiras de soluções não dominadas ou fronteira de Pareto. Veja Figura 8.2.

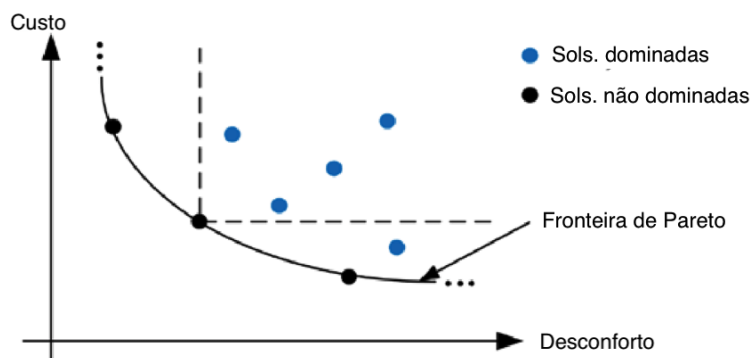


Figura 8.2: Exemplo de soluções que possuem mais de um critério de ordenação.

Na ordenação temos um ponto de partida, que é o vetor não ordenado. Temos um ponto de chegada que é o vetor ordenado. E para percorrer este trajeto entre a partida e a chegada é necessário uma sequência de passos definido por um algoritmo de ordenação.

Quais são os principais algoritmos de ordenação? Existem duas classes principais de algoritmo de ordenação que são os algoritmos $O(n^2)$ e os algoritmos $O(n \lg n)$. Os algoritmos $O(n^2)$ são mais simples e pouco eficientes. São importantes para fins de estudo, mas não são usados na prática. Existem também algoritmos $O(n \log n)$ que são bem mais eficientes e serão vistos na próxima aula.

$O(n^2)$ (Simples)	$O(n \log n)$ (Eficientes)
Algoritmo da Bolha	Merge Sort
Ordenação por Seleção	Quick Sort
Ordenação por Inserção	Heap Sort
	AVL Sort
	Árvore B Sort

Na prática são utilizados apenas os algoritmos $O(n \log n)$ que são muito mais rápidos.

A seguir, o algoritmo de ordenação por seleção, que é um dos mais simples. Em inglês *Selection Sort*.

1. Descubra a posição do maior elemento;
2. Troque o maior elemento com o último elemento;
3. Desconsidere o último elemento;
4. Repita o processo enquanto $n > 1$.

Veja Figura 8.3.

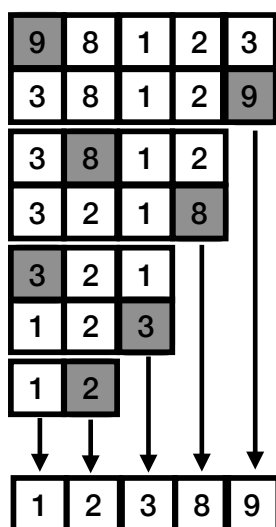


Figura 8.3: Problema da ordenação. Primeiro vetor está desordenado e o último está ordenado.

Como o custo para achar o maior elemento é $O(n)$ e esta função é chamada $O(n)$ vezes, a complexidade final é $O(n^2)$. Para ser mais preciso, a primeira busca do máximo gasta n operações. A segunda chamada gasta $(n - 1)$, a próxima $(n - 2)$ e assim sucessivamente até 2. Somando os custos da chamadas, temos uma progressão aritmética com soma $\Theta(n^2)$.

A ordenação por seleção já foi mostrada na página 29. Nela foram mostradas as funções *gera*, que cria um vetor de números aleatórios. A função *print*, que imprime o vetor. E o algoritmo da seleção, que será reproduzido abaixo:

Primeiro a função máximo:

```

1 /**
2  Encontra a posicao do maior elemento do vetor.
3  Se existirem varios elementos nesta condicao,
4  devolve a posicao do primeiro maior elemento.
5  Complexidade Teta(n).
6  */
7 int maximo(int v[], int n){
8     int max = 0;
9     for(int i=1;i<n;i++){
10         if(v[i]>v[max]){
11             max = i;
12         }
13     }
14     return max;
15 }
```

E depois a função ordena:

```

1 /**
2  Ordena os elementos em ordem crescente via
3  ordenacao por selecao.
4  Teta(n^2)
5  */
```

```
6 void ordena(int v[], int n){
7     while(n>1){
8         //seleciona o maior elemento
9         int max = maximo(v,n);
10        //coloca na ultima posicao
11        int aux = v[n-1];
12        v[n-1] = v[max];
13        v[max]=aux;
14        //repete o processo no vetor
15        //desconsiderando o ultimo elemento
16        n--;
17    }
18 }
```

A seguir uma implementação alternativa do algoritmo da seleção, usando apenas uma função. Nesta versão fica mais claro o uso de dois *for*s encadeados.

```
1 void ordenacao_selecao(int v[], int n){
2     for(int N=n;N>1;N--){
3         int max = 0;
4         for(int i=1;i<N;i++)
5             if(v[i]>v[max]) max = i;
6         int aux = v[N-1];
7         v[N-1] = v[max];
8         v[max]=aux;
9     }
10 }
```

Uma variação interessante do algoritmo da seleção é o uso de recursão. Veja a versão recursiva abaixo:

```
1 void selecao_recursivo(int v[], int n){
```

```
2     if(n>1){
3         //print para debug
4         print(v,n);
5         int max = 0;
6         for(int i=1;i<n;i++)
7             if(v[i]>v[max]) max = i;
8         int aux = v[n-1];
9         v[n-1] = v[max];
10        v[max]=aux;
11        //print para debug
12        print(v,n);
13        selecao_recursivo(v,n-1);
14    }
15 }
```

Na versão recursiva, executar o problema desconsiderando-se o último elemento é feito por meio de uma chamada recursiva. Veja os resultados dos *prints* abaixo:

Digite n:5

[4,8,9,6,3]

[4,8,3,6,9]

[4,8,3,6]

[4,6,3,8]

[4,6,3]

[4,3,6]

[4,3]

[3,4]

Vetor ordenado

[3,4,6,8,9]

O vetor a ser ordenado tinha 5 posições. Primeiramente colocou o 9 na última posição. Depois repetiu-se o processo no vetor com 4 posições. Colocou o 8 na última posição e

repetiu-se o processo no vetor com 3 posições. Assim sucessivamente até obter o vetor ordenado.

Questão 108) Implemente uma variação do algoritmo da seleção. Primeiramente selecione o menor elemento e ponha na primeira posição do vetor. Repita o processo no vetor sem considerar a primeira posição. Faça este algoritmo de maneira iterativa.

O próximo algoritmo de ordenação que veremos é o algoritmo da bolha. O nome deste algoritmo é baseado no fato de que as bolhas em um líquido sobem, mas quanto maior a bolha, mais rápida é a subida. As bolhas maiores sobem mais rapidamente.

O algoritmo da bolha consiste em comparar sempre um elemento com o elemento seguinte. Se eles estiverem fora de ordem, faz uma troca.

Veja Figura 8.4.

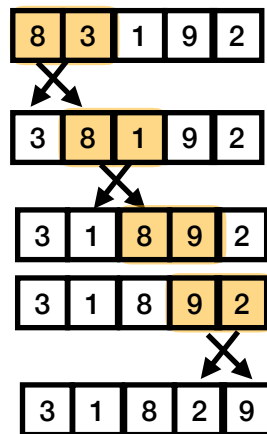


Figura 8.4: Exemplo de uma iteração do algoritmo da bolha. Ao final da iteração, o maior elemento está na última posição.

Veja que o algoritmo inicia subindo o valor 8. No terceiro passo, o valor 8 encontra um valor superior, o 9. A partir deste momento, quem sobe é o 9. Ao final da primeira iteração, o maior elemento estará na última posição. Para finalizar o algoritmo, basta repetir o processo com um elemento a menos, até que reste um elemento.

¹ /**

² *Ordena os elementos em ordem crescente via*

```
3  ordenacao metodo da bolha. Teta(n^2)
4  */
5  void bolha_rekursiva(int v[], int n){
6      if(n>1){
7          for(int i=0;i<n-1;i++){
8              if(v[i]>v[i+1]){
9                  int aux = v[i];
10                 v[i] = v[i+1];
11                 v[i+1]=aux;
12             }
13         }
14         bolha_rekursiva(v,n-1);
15     }
16 }
```

O código compara $v[i]$ com $v[i + 1]$. Se estiverem em ordem contrária, troca os dois elementos de posição. Ao final do *for*, o maior elemento estará na última posição. A chamada recursiva repete o processo com um elemento a menos no vetor.

Abaixo uma implementação iterativa do algoritmo da bolha.

```
1  void bolha_iterativa(int v[], int n){
2      while(n>1){
3          for(int i=0;i<n-1;i++){
4              if(v[i]>v[i+1]){
5                  int aux = v[i];
6                  v[i] = v[i+1];
7                  v[i+1]=aux;
8              }
9          }
10         n--;
11     }
```

12 }

O terceiro algoritmo de ordenação $O(n^2)$ é o algoritmo de ordenação por inserção. Este algoritmo se assemelha ao método de ordenar cartas de baralho nas mãos. Veja figura 8.5.

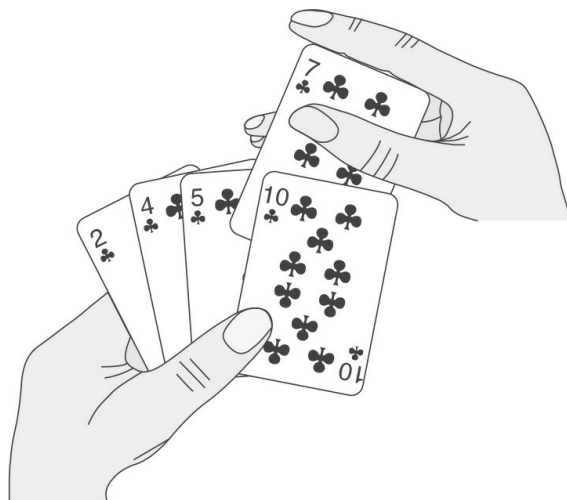


Figura 8.5: Ordenação de cartas utilizando o algoritmo da inserção. Fonte: [Cormen et al., 2002]

Suponha um vetor com 7 posições, sendo que as 6 primeiras já estão ordenadas. Exemplo: $v = (0, 1, 2, 4, 4, 9, 3)$. O algoritmo de inserção tem como invariante que o vetor está ordenado, menos a última posição.

E se o vetor não tiver ordenado? O insertion sort vai começar com um vetor com 2 elementos. Resolvendo para 2, parte para 3 elementos. Resolvendo para 3, parte para 4 elementos e assim por diante. Ou seja, trabalha com o vetor ordenado e insere um elemento de cada vez.

Vamos partir do princípio que está ordenado, a não ser o último elemento. Veja o código abaixo:

```
1 /**
2  Ordena o vetor, desde que esteja ordenado,
3  a não ser o ultimo elemento.
```

```
4  */
5  void insercaoA(int v[], int n){
6      int lixo = -1;
7      int chave = v[n-1];
8      v[n-1]=lixo;
9      int i = n-2;
10     print(v,n);
11     while(i>=0 && v[i]>chave){
12         v[i+1]=v[i];
13         v[i]=lixo;
14         i--;
15         print(v,n);
16     }
17     v[i+1] = chave;
18 }
19 int main(){
20     int n=7;
21     int v[] = {0,1,2,4,4,9,3};
22     insercaoA(v,n);
23     print(v,n);
24 }
```

Veja a saída do programa:

[0,1,2,4,4,9,-1]

[0,1,2,4,4,-1,9]

[0,1,2,4,-1,4,9]

[0,1,2,-1,4,4,9]

[0,1,2,3,4,4,9]

O primeiro passo do algoritmo foi salvar o valor 3 na variável *chave* e colocar *lixo* em seu conteúdo. Note que lixo vale -1 . Após esta operação, o vetor está assim:

[0,1,2,4,4,9,-1]

Agora, todos os números maiores que *chave* serão deslocados uma posição para a direita:

[0,1,2,4,4,-1,9]

[0,1,2,4,-1,4,9]

[0,1,2,-1,4,4,9]

Ao final desta etapa, basta colocar o valor guardado em *chave* na posição que esta o lixo, para o vetor ficar ordenado.

[0,1,2,3,4,4,9]

Não é preciso colocar *lixo* na posição desocupada. Eu fiz isso apenas para fins didáticos, para imprimir o vetor nos passos intermediários.

E para ordenar o vetor inteiro? Podemos resolver isso recursivamente ou iterativamente. Veja a solução recursiva:

```
1 /**
2  Algoritmo da insercao recursivo.
3  */
4 void insercaoRec(int v[], int n){
5     if(n>1){
6         insercaoRec(v,n-1);
7         int chave = v[n-1];
8         int i = n-2;
9         while(i>=0 && v[i]>chave){
10             v[i+1]=v[i];
11             i--;
12         }
13         v[i+1] = chave;
14     }
15 }
```

Primeiro verifica se o vetor tem mais de uma posição. O vetor com uma posição já está ordenado. Se $n > 1$ faz uma chamada recursiva para resolver o problema com um elemento a menos. Isso vai se repetir, chamada após chamada, até chegar no vetor com um único elemento.

Uma vez ordenado o vetor com 1 elemento, será executado o processo definido anteriormente, que insere o segundo elemento no vetor. Depois repete o processo para 3 elementos e assim sucessivamente até se ordenar o vetor inteiro.

Também temos a versão iterativa do algoritmo da inserção:

```
1  /**
2   Algoritmo da insercao iterativo.
3   */
4  void insercaoIte(int v[], int n){
5      int N = 2;
6      while(N<=n){
7          int chave = v[N-1];
8          int i = N-2;
9          while(i>=0 && v[i]>chave){
10             v[i+1]=v[i];
11             i--;
12         }
13         v[i+1] = chave;
14         N++;
15     }
16 }
```

Na versão iterativa, começamos com $N = 2$ elementos e aplicamos a estratégia da inserção repetidas vezes. A cada iteração N é incrementado até que ele atinja o valor n original.

Questão 109) Implemente:

a) Crie uma estrutura Aluno contendo nome, RA e CR.

- b) Faça um construtor para esta estrutura.
- c) Faça uma função que imprime um Aluno em uma única linha
- d) Crie um vetor de apontadores para Aluno com 5 alunos e mande ordenar usando algum algoritmo de ordenação visto nesta aula. Além do vetor e do n deve ser passada uma opção. Se opção for 1, ordena por nome. Se a opção for 2, ordena por RA. Se a opção for 3, ordena por CR.
- e) Faça um interface com o usuário em linha de comando que pede para o usuário digitar a opção. Após ler a opção, o programa deve ordenar e fazer a impressão do vetor ordenado.

Questão 110) Neste exercício, você pode usar os códigos disponíveis nas aulas 4, 5 e 6, fazendo as adaptações necessárias.

- a) Crie uma estrutura *Aluno* contendo nome, RA e CR e um apontador para próximo.
- b) Crie uma lista ligada, onde cada elemento é um aluno.
- c) Faça uma função que imprime um aluno e uma função que imprime uma lista ligada de alunos. A impressão da lista ligada deve, necessariamente, chamar a função que imprime um aluno.
- d) Faça uma função que faz a ordenação da lista ligada em relação ao nome. O parâmetro da função é apenas uma lista *li*. Nada deve ser impresso na função, a não ser impressões para *debug*. A função ordena não deve criar ou destruir Alunos, deve apenas trabalhar com os estruturas existentes em *li*. Sua função ordenação deve executar os seguintes passos:
 - d.1) Criar uma lista vazia chamada *aux*.
 - d.2) Percorrer a lista *li* e descobrir a posição *max_pos* do maior elemento.
 - d.3) Remover o elemento da posição *max_pos*. Diferente do que foi mostrado na Aula 6, neste caso a função deve devolver a estrutura aluno que foi removido e não apenas o conteúdo.
 - d.4) Inserir o aluno removido no início de *aux*.
 - d.5) Depois que a lista *li* estiver vazia, fazer a lista receber *aux*.
- e) Na função *main*, faça:
 - e.1) Insira 15 alunos na lista ligada. Pode ser direto no código, sem fazer a leitura dos dados. Os dados precisam estar bastante desordenados.
 - e.2) Imprima a lista.
 - e.3) Chame a função que ordena a lista.
 - e.4) imprima a lista novamente.

Inserir mais exercícios aqui.

Capítulo 9

Aula 9

9.1 Ordenação em $O(n \lg n)$

O tema da ordenação é bastante importante. É comum, em processo seletivos, se fazer alguma pergunta sobre ordenação.

Nas aulas passadas vimos o algoritmo da seleção, bolha e inserção, todos $O(n^2)$. Também vimos um algoritmo que insere em uma lista ligada de maneira ordenada.

Algoritmos de ordenação $O(n^2)$ servem apenas para fins didáticos, pois são muito lentos. Na prática, sempre utilizamos algoritmos $O(n \lg n)$.

Suponha que o algoritmo $O(n^2)$ tem tempo $T_1(n) = n^2 10^{-9}s$ e que o algoritmo $O(n \lg n)$ tem tempo $T_2(n) = n \log n 10^{-9}s$. Suponha $n = 10^6$. Neste caso, o primeiro algoritmo irá demorar:

$$T_1(10^6) = (10^6)^2 10^{-9}s \tag{9.1}$$

$$= 10^{12} 10^{-9}s \tag{9.2}$$

$$= 10^3s \tag{9.3}$$

$$= 1000s \tag{9.4}$$

$$\tag{9.5}$$

Já o segundo algoritmo irá demorar:

$$T_2(10^6) = 10^6 \log 10^6 10^{-9}s \quad (9.6)$$

$$= 10^6 \times 6 \times 10^{-9}s \quad (9.7)$$

$$= 6 \times 10^{-3}s \quad (9.8)$$

$$= 0.006s \quad (9.9)$$

Por estas contas, temos que o primeiro algoritmo é 166 mil vezes mais lento que o segundo. Note que a entrada e a saída dos dois algoritmos é a mesma, porém um demora uma centena de milhares de vezes mais tempo que o outro.

Esse comportamento é comum no desenvolvimento de algoritmos. Quando se faz uma melhora na complexidade, não é para ter um ganho de 30% no tempo e sim um ganho de 10x mais rápido, 100x mais rápido, 1000x mais rápido.

9.2 MergeSort

O primeiro algoritmo que vamos estudar nesta aula é o MergeSort. ele não possui uma boa tradução, mas seria algo como ordenação por intercalação. Mas usa-se o nome em inglês mesmo, MergeSort.

O algoritmo MergeSort é naturalmente recursivo. Talvez seja possível fazer uma versão não recursiva, mas isso é complicado e não será visto nesta apostila.

O primeiro passo do MergeSort consiste em dividir o vetor ao meio. Veja figura 9.1.

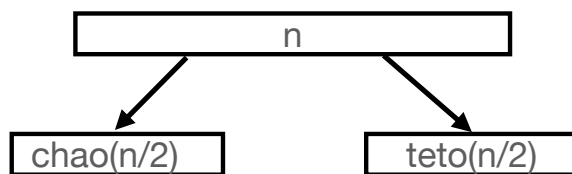


Figura 9.1: Primeiro passo do MergeSort consiste em dividir o vetor ao meio.

Caso o vetor tenha tamanho par, as duas partes terão $n/2$. Caso contrário, a primeira parte terá $(n - 1)/2$ elementos e a segunda parte terá $(n + 1)/2$ elementos.

```
1      int  n1,n2;
2      n1  =  n/2;
3      n2  =  n-n1;
```

Vamos primeiro copiar a metade de v para $v1$ e a segunda metade de v para $v2$.

```
1      v1  =  (int *)malloc(sizeof(int)*n1);
2      v2  =  (int *)malloc(sizeof(int)*n2);
3
4      for(int i=0;i<n1;i++){
5          v1[i]=v[i];
6      }
7      for(int i=0;i<n2;i++){
8          v2[i]=v[n1+i];
9      }
```

Quando eu divido meu problema ao meio, eu tenho um problema semelhante para uma instância menor. Quando isso ocorre, eu posso resolver o problema recursivamente.

Se v foi dividido em $v1$ e $v2$, eu posso fazer $MergeSort(v1)$ e $MergeSort(v2)$. Veja figura 9.2.

```
1      mergeSort(v1,n1);
2      mergeSort(v2,n2);
```

Após aplicar o MergeSort, eu posso assumir que tanto $v1$ quanto $v2$ estão ordenados. Note que eu ainda não terminei de implementar o MergeSort mas quando eu terminar, ele vai ordenar corretamente as duas metades. Após o fim da chamada recursiva, tanto $v1$ quanto $v2$ estão ordenados.

Após isso, é preciso fazer o algoritmo da intercalação, ou *merge*, juntando $v1$ e $v2$ em um vetor final ordenado v . O menor elemento de $v1$ é o primeiro elemento do vetor. O mesmo para $v2$. O primeiro passo consiste em colocar um contador no início de cada vetor, conforme

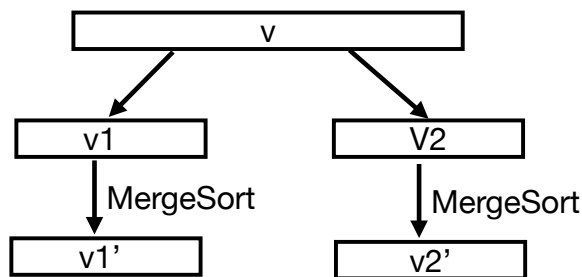


Figura 9.2: Segundo passo do MergeSort consiste em fazer uma chamada recursiva para $v1$ e outra para $v2$. Considere na figura $v1'$ e $v2'$ como as versões ordenadas de $v1$ e $v2$.

Figura 9.3. Os vetores v' , $v1'$ e $v2'$ são as versões ordenadas de v , $v1$ e $v2$, respectivamente.

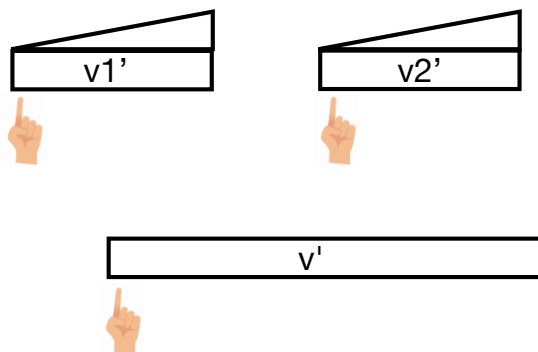


Figura 9.3: Algoritmo da intercalação (*merge*) inicia com 3 contadores. Um no início de $v1$, um no início de $v2$ e um no início de v

O contador no início de $v1$ será $top1$. O contador no início de $v2$ será $top2$ e o contador no início de v será top . Os três contadores começam com zero. O menor entre $v1[top1]$ e $v2[top2]$ é escolhido para entrar em $v[top]$. Veja o código abaixo:

```

1      if (v1[top1] < v2[top2]) {
2          v[top++] = v1[top1++];
3      } else {
4          v[top++] = v2[top2++];
5      }
```

Após colocar o menor elemento em $v[top]$ tanto top quanto o contador do respectivo vetor são incrementados. Isso faz com que, logicamente, um elemento seja consumido em $v1$ ou $v2$ e um elemento seja inserido em v . Este passo se repete enquanto houver elemento nos dois vetores:

```

1   while (top1 < n1 && top2 < n2) {
2       if (v1[top1] < v2[top2]) {
3           v[top++] = v1[top1++];
4       } else {
5           v[top++] = v2[top2++];
6       }
7   }
```

Desta maneira, os elementos vão sendo consumidos de $v1$ e $v2$ e inseridos em v , conforme figuras 9.4 e 9.7.

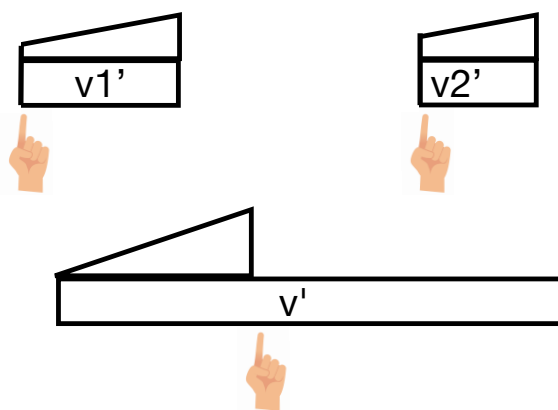


Figura 9.4: Algoritmo da intercalação (*merge*). Os elementos são consumidos de $v1$ e de $v2$ e são colocados no vetor v

Ao final do *while*, um entre $v1$ e $v2$ se encerrou. Neste caso, é necessário copiar os elementos faltantes diretamente para v .

```

1   //copia elementos faltantes de v1 para v
2   //caso haja algum
```

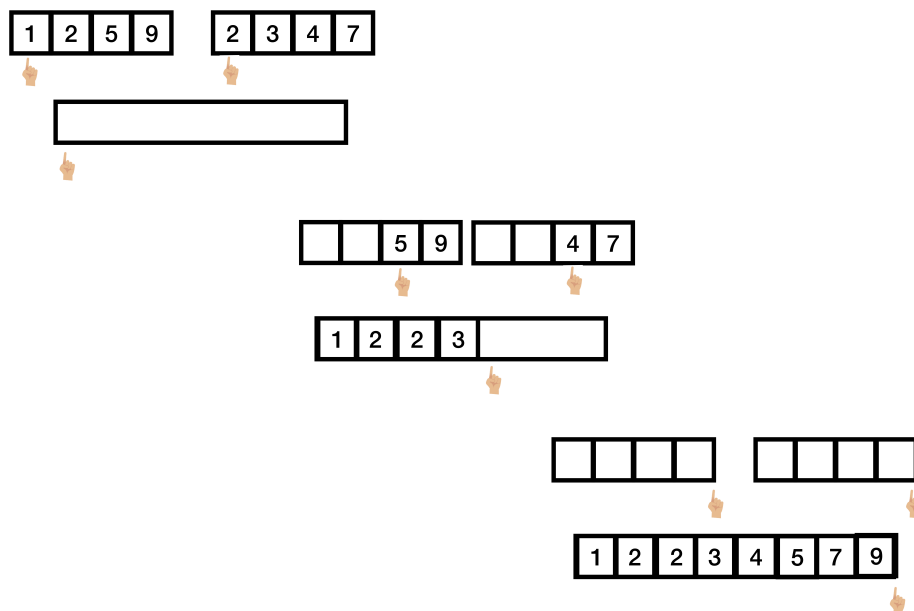


Figura 9.5: Algoritmo da intercalação (*merge*). Os elementos são consumidos de $v1$ e de $v2$ e são colocados no vetor v

```

3   while(top1<n1){
4       v[top++]=v1[top1++];
5   }
6   //copia elementos faltantes de v2 para v
7   //caso haja algum
8   while(top2<n2){
9       v[top++]=v2[top2++];
10  }

```

Este é o fim do algoritmo *merge*. Veja como ficou todo o *merge*.

```

1 void merge(int *v1,int n1,int *v2,int n2,int *v){
2     int top1=0,top2=0,top=0;
3     while(top1<n1&&top2<n2){
4         if(v1[top1]<v2[top2]){
5             v[top++]=v1[top1++];

```

```
6         }else{
7             v[top++]=v2[top2++];
8         }
9     }
10    //copia elementos faltantes de v1 para v
11    //caso haja algum
12    while(top1<n1){
13        v[top++]=v1[top1++];
14    }
15    //copia elementos faltantes de v2 para v
16    //caso haja algum
17    while(top2<n2){
18        v[top++]=v2[top2++];
19    }
20 }
```

Veja agora como fica o algoritmo do MergeSort:

```
1 void mergeSort(int v[],int n){
2     if(n>1){
3         int n1,n2;
4         n1 = n/2;
5         n2 = n-n1;
6         int *v1,*v2;
7         v1 = (int *)malloc(sizeof(int)*n1);
8         v2 = (int *)malloc(sizeof(int)*n2);
9
10        for(int i=0;i<n1;i++){
11            v1[i]=v[i];
12        }
13        for(int i=0;i<n2;i++){
```

```
14         v2[i]=v[n1+i];
15     }
16
17     mergeSort(v1,n1);
18     mergeSort(v2,n2);
19     merge(v1,n1,v2,n2,v);
20     free(v1);
21     free(v2);
22 }
23 }
```

Um passo que pode ainda estar meio nebuloso é o que ocorre nas duas chamadas recursivas. Como é possível fazer a chamada recursiva na metade do vetor e ela ordenar corretamente?

Imagine um vetor com 64 elementos. Será feita uma chamada recursiva nos primeiros 32 elementos e uma chamada recursiva nos 32 elementos seguintes. Como a chamada recursiva irá executar o MergeSort, ela vai dividir o vetor com 32 posições em dois vetores com 16 posições e fazer uma chamada recursiva para cada um. Novamente será executado o MergeSort, que dividirá o vetor em dois vetores com 8 elementos. O vetor com 8 elementos será dividido em dois vetores com 4 elementos. O vetor com quatro elementos será dividido em dois vetores com 2 elementos. Finalmente o vetor com 2 elementos será dividido em dois vetores com 1 elemento.

Um vetor com um único elemento já está ordenado. Veja, no código, que nada é feito neste caso. Sendo assim, as duas chamadas recursivas para 1 elementos não farão nada e, ao fim, teremos dois vetores com um elemento ordenados. Os dois vetores de um elemento passarão pela intercalação (*merge*) e ao fim teremos um vetor com duas posições ordenado. Os vetores com duas posições serão intercalados, e, ao fim teremos um vetor com 4 posições ordenado. Os vetores com 4 posições serão intercalados, e, ao fim, teremos um vetor com 8 posições ordenado. No próximo passo, teremos dois vetores com 16 posições ordenados que, quando intercalados, gerarão o vetor com 32 posições ordenado. Finalmente, os dois vetores

com 32 posições serão intercalados em um vetor final ordenado com 64 posições.

As chamadas recursivas vão, de maneira organizada, dividindo o vetor até que ele fique com um elemento e quem ordena, de fato, é a intercalação (*merge*) iniciando a ordenação em vetores com 1 elemento, depois 2, 4, 8, 16 etc. Veja Figura 9.6.

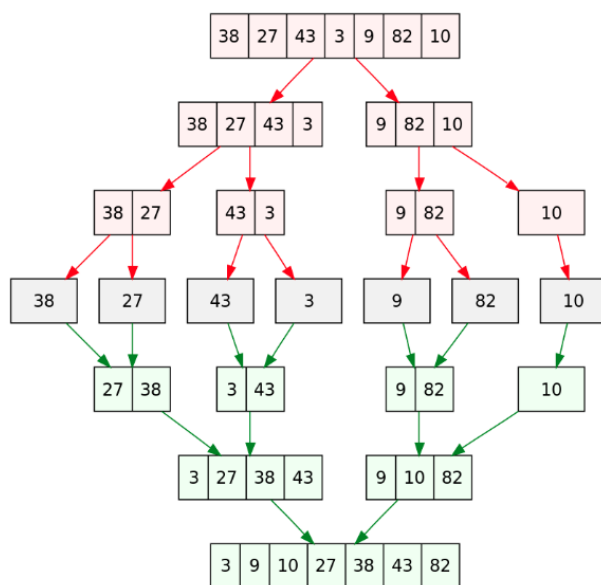


Figura 9.6: Imagem mostrando as chamadas recursivas do MergeSort. Fonte: Wikipedia, Merge sort algorithm diagram.svg.

Quanto à complexidade do MergeSort, ele tem o tempo regido por $T(n) = 2T(n/2) + cn$. Ou seja, o tempo do MergeSort com n elementos é igual a duas vezes o tempo do MergeSort com $n/2$ elementos mais o código da intercalação, que é $O(n)$. Esta relação de recorrência, quando resolvida, resulta em $\Theta(n \log n)$. Abaixo será dada uma idéia de como a complexidade pode ser obtida.

A cada nível das chamadas recursivas, dobra-se o número de problemas, mas o tamanho do vetor cai pela metade. A soma das intercalações em cada nível se mantém cn . Como n é dividido ao meio em cada nível até atingir o valor 1, a altura desta árvore é $\lg n$. Multiplicando-se o número de níveis, que é $\lg n$ pelo custo de cada nível, que é cn , temos um custo final $cn \lg n$. Veja Figura 9.7.

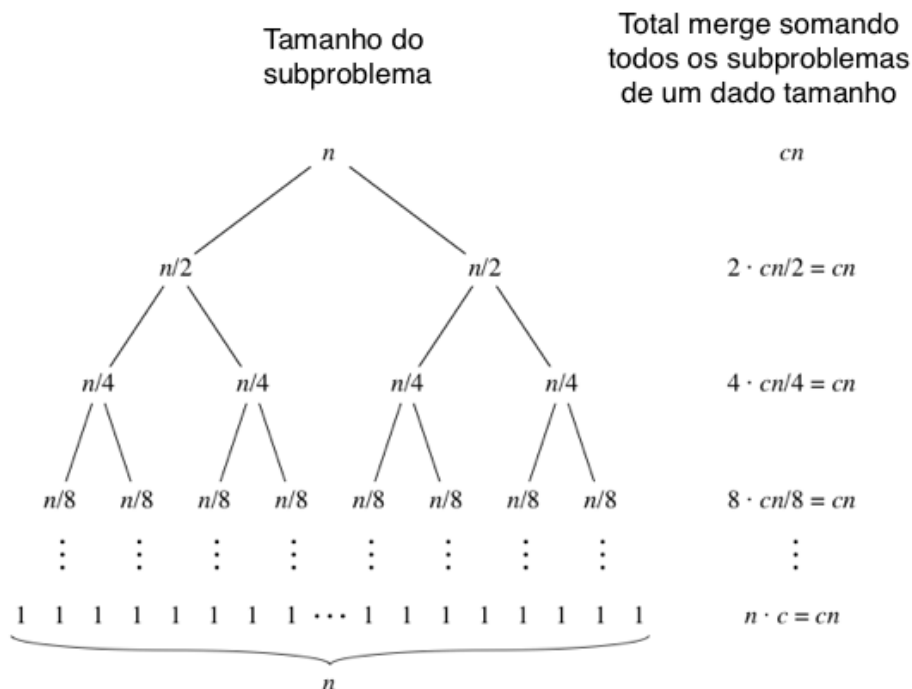


Figura 9.7: Complexidade do MergeSort via árvore de chamadas recursivas.

9.3 QuickSort

O próximo algoritmo de ordenação que estudaremos é o QuickSort. O termo *Quick* significa rápido. O QuickSort consiste em escolher um elemento para ser o pivô. Após a escolha do pivô, o vetor é dividido em duas partes. Elementos menores que o pivô ficam à esquerda e elementos maiores que o pivô ficam à direita. A melhor maneira de se escolher o pivô é escolher aleatoriamente.

A divisão não ocorre exatamente na metade. Uma vez que os elementos menores estiverem à esquerda e os elementos maiores estiverem à direita, aí basta fazer a chamada recursiva.

Veja o código do QuickSort abaixo:

```

1 void quickSort(int v[], int p, int r){
2     if(p < r){
3         int q = particao(v, p, r);
4         quickSort(v, p, q-1);
5         quickSort(v, q+1, r);

```

```
6     }  
7 }
```

A essência do QuickSort está na partição. Veja o código da partição abaixo:

```
1 int particao(int v[], int p, int r){  
2     int pivo = v[p];  
3     int topo = p;  
4     for(int i=p+1;i<=r;i++){  
5         if(v[i]<pivo){  
6             v[topo] = v[i];  
7             v[i] = v[topo+1];  
8             topo++;  
9         }  
10    }  
11    v[topo]=pivo;  
12    return topo;  
13 }
```

Considere a situação da Figura 9.8.

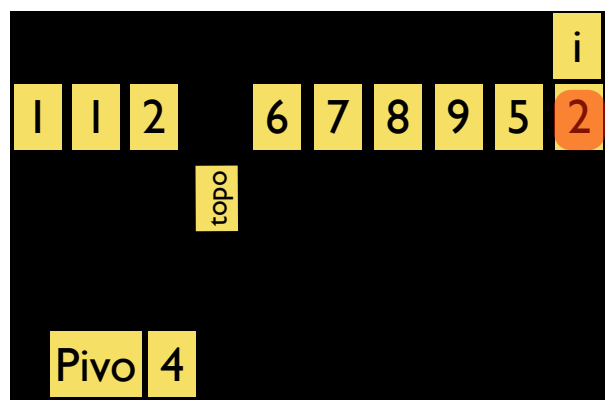


Figura 9.8: Exemplo de execução do QuickSort.

Na Figura 9.8 são executados os comandos:

```

1      if (v[i] < pivo) {
2          v[topo] = v[i];
3          v[i] = v[topo+1];
4          topo++;
5      }

```

O elemento $v[i] = 2$ é colocado na posição *topo*. O elemento da posição *topo* + 1 é colocado na posição *i* e é feito um *topo* ++. A situação após a execução deste código pode ser encontrado na Figura 9.9.

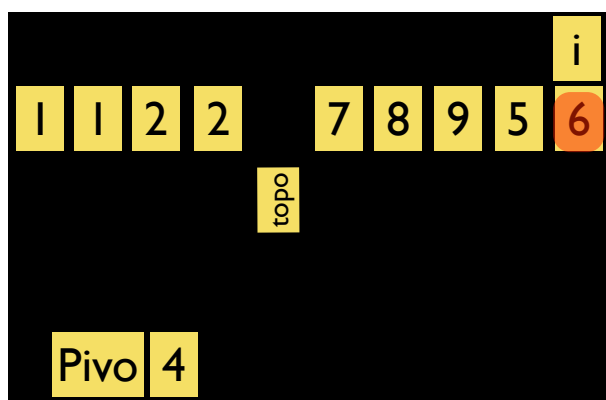


Figura 9.9: Exemplo de execução do QuickSort.

Ao final da execução, elementos menores que o *pivo* = 4 estão na primeira parte do vetor e elementos maiores que *pivo* = 4 estão na segunda parte do vetor.

Ao final da execução da partição, o pivô está na posição correta.

O melhor caso do QuickSort é quando o vetor é dividido ao meio. O pior caso é quando um lado da chamada recursiva fica com $n - 1$ elemento e o outro lado com 0 elementos. Se sempre ocorrer este desbalanceamento, o QuickSort se torna $O(n^2)$. Para evitar este comportamento, é necessário escolher o pivô aleatoriamente entre todos os elementos do vetor e tratar o caso de elementos repetidos.

Escolhendo um elemento aleatório e tratando o caso de elementos repetidos, o QuickSort passa a ter um comportamento esperado $O(n \lg n)$.

Questão 111) Considere a lista ligada vista nesta apostila. Implemente o MergeSort utilizando listas ligadas.

Questão 112) Considere o algoritmo MergeSort e o algoritmo da seleção.

1. Usando o algoritmo de seleção, execute com um valor razoável de n e descubra a constante. Lembre que $T(n) = cte * n^2 s$
2. Uma vez descoberta a constante, descubra matematicamente o maior valor de n que pode ser executado em 5 minutos pelo algoritmo da seleção. Chame este valor de n'_5 .
3. Execute o algoritmo da seleção com o valor descoberto na questão anterior, n'_5 , e verifique se o tempo é igual ou próximo a 5 min.
4. Execute o algoritmo do MergeSort com n'_5 elementos. Descubra o tempo de execução. Compare com o tempo do algoritmo de seleção.
5. Usando o algoritmo de MergeSort, execute com um valor razoável de n e descubra a constante. Lembre que $T(n) = cte * n \lg n s$
6. Uma vez descoberta a constante, descubra matematicamente o maior valor de n que pode ser executado em 5 minutos pelo MergeSort. Chame este valor de n''_5 .
7. Execute o algoritmo do MergeSort com o valor descoberto na questão anterior, n''_5 , e verifique se o tempo é igual ou próximo a 5 min.
8. Descubra matematicamente o tempo gasto pelo algoritmo da seleção para executar uma instância com n''_5 elementos.

Inserir mais exercícios aqui!

Capítulo 10

Aula 10

Vamos considerar um algoritmo que gasta n operações e outro que gasta $\lg n$ operações. Veja a tabela a seguir:

n	$\lg n$
1.000	≈ 10
1.000.000	≈ 20
1.000.000.000	≈ 30

Ao se trocar n por $\lg n$, o número cai de 1 bilhão para apenas 30.

10.1 Busca Sequencial

Suponha que temos um vetor de CPFs e gostaríamos de saber se um dado CPF pertence à lista. O elemento buscado se chama *chave*. O vetor não está ordenado. A maneira de se buscar uma *chave* dentro do vetor consiste em passar por todas as posições do vetor e verificar se o elemento corrente é igual a chave.

```
1 #include <stdio.h>
2
3 int busca(int v[], int n, int chave){
4     for(int i=0;i<n;i++){
5         if(v[i]==chave){
6             return 1;
```

```
7         }
8     }
9     return 0;
10 }
11
12 int main(){
13     int v[] = {2,4,6,8,9,2,1,3,5,0};
14     int n=10;
15     int chave =5;
16     printf("Busca =%d\n",busca(v,n,chave));
17 }
```

Saída:

Busca =1

Esta busca se chama busca sequencial. Ela percorre todas as posições do vetor. É possível fazer algo análogo em uma lista ligada.

```
1 int busca(Lista *li,int chave){
2     for(No * aux = li->ini; aux!=NULL; aux=aux->prox){
3         if(aux->valor==chave){
4             return 1;
5         }
6     }
7     return 0;
8 }
9
10 int main(){
11     Lista *li = newList();
12     addFim(li,3);
13     addFim(li,4);
```

```
14     addFim(li,9);
15     addFim(li,1);
16     addFim(li,5);
17     int chave =5;
18     printf("Busca =%d\n",busca(li,chave));
19 }
```

A busca sequencial é claramente $O(n)$. No pior caso é $\Theta(n)$. No melhor caso, encontra a chave na primeira posição, o que é $\Theta(1)$. Na média, para elementos que pertencem ao vetor, vai encontrar o elemento percorrendo a metade do vetor. Note que $\Theta(n/2) = \Theta(n)$.

Vamos supor agora que eu possuo um conjunto de RGs do estado de São Paulo salvos em um vetor e um conjunto de RGs do estado do Paraná salvo em outro vetor.

Hoje, quando você cria um RG, ele vem com um número único. Mas no passado não foi sempre assim. Era gerado um número único para cada estado. Por esta razão, era preciso especificar o número de RG e o estado de origem. Por exemplo, poderia existir um RG 5.123.456 PR e outro RG com número 5.123.456 SP. O que diferenciava o primeiro RG do segundo é o estado. Não existia um sistema centralizado e cada estado era responsável por gerar seus próprios RGs.

Atualmente, o RG é um número único para todo o Brasil.

A população de São Paulo é aproximadamente 44 milhões de habitantes. Já a população do estado do Paraná é aproximadamente 11 milhões de habitantes.

Eu gostaria de fazer um programa para descobrir quantos elementos estão repetidos nos dois estados. Veja Figura 10.1.

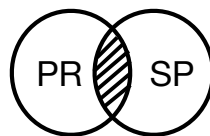


Figura 10.1: O objetivo do algoritmo é calcular a intersecção entre os conjuntos *PR* e *SP*.

É preciso criar um vetor *int SP*[44.000.000] e um vetor *int PR*[11.000.000]. Para fazer

a interseção dos dois vetores, é necessário fazer buscas. Veja o código abaixo:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 //faz um embaralhamento do vetor
5 void embaralha(int v[], int n){
6     for(int i=0;i<n;i++){
7         int pos_rand = rand()%n;
8         int aux = v[i];
9         v[i] = v[pos_rand];
10        v[pos_rand] = aux;
11    }
12 }
13
14 int main(){
15     int* SP;
16     int n1 = 44000000;
17     SP = (int *)malloc(sizeof(int)*n1);
18
19     int* PR;
20     int n2 = 11000000;
21     PR = (int *)malloc(sizeof(int)*n2);
22
23     for(int i=0;i<n1;i++){
24         SP[i] = 13*i;
25     }
26     embaralha(SP,n1);
27
28     for(int i=0;i<n2;i++){
29         PR[i] = 19*i;
```

```
30     }
31     embaralha(PR,n2);
32
33     int size = 0;
34     for(int i=0;i<n1;i++){
35         for(int j=0; j<n2; j++){
36             if(SP[i]==PR[j]){
37                 size++;
38             }
39         }
40     }
41     printf("Tamanho Interseccao=%d\n",size);
42
43
44 }
```

O primeiro código cria os vetores de São Paulo e Paraná. Preenche com alguns valores arbitrários para testes. Veja que existem muitas repetições no método usado para preencher o vetor. Por exemplo, existem valores como 13×19 em São Paulo e 19×13 no Paraná. Note que dentro de um vetor não existe repetição.

O código faz i varrer todos os elementos de São Paulo. Para cada valor de i , percorre-se todos os elementos do Paraná. Caso haja alguma repetição, incrementa a variável *size*.

A complexidade desta solução é $\Theta(n_1 n_2)$ sendo que $n_1 = |SP|$ e $n_2 = |PR|$. Para $n = \max\{n_1, n_2\}$, a complexidade é $O(n^2)$. Dependendo dos vetores, pode ser que não haja nenhum elemento na intersecção, mas é necessário rodar o programa para descobrir isto.

Ao colocar o programa para executar, ele começou a execução e não terminou. O tempo gasto pelo programa é de horas, chegando a vários dias a depender do computador.

O tempo de execução é aproximadamente $T(n_1, n_2) = cn_1 n_2 s$. Eu executei em meu computador com $n_1 = n_2 = 40.000$ e o tempo da intersecção foi 3,887s.

$$T(n_1, n_2) = cn_1n_2 \quad (10.1)$$

$$T(40.000, 40.000) = c40.000^2 \quad (10.2)$$

$$3,887 = c40.000^2 \quad (10.3)$$

$$c = \frac{3,887}{40.000^2} \quad (10.4)$$

$$c = 2,42 \times 10^{-9} \quad (10.5)$$

Usando esta constante temos que $T(n_1, n_2) = n_1n_2 2,42 \times 10^{-9}s$. Agora vamos usar os vetores com os tamanhos corretos:

$$T(11000000, 44000000) = 11000000 \times 44000000 \times 2,42 \times 10^{-9}s \quad (10.6)$$

$$= 1171280 \text{ s} \quad (10.7)$$

$$= 325,35 \text{ h} \quad (10.8)$$

$$= 13,55 \text{ dias} \quad (10.9)$$

Ou seja, o tempo de execução deste algoritmo é de mais de 13 dias em meu computador. O tempo pode variar de computador para computador, mas é um tempo em muitas situações impraticável.

Questão 113) Descubra o tempo para executar o código anterior em seu computador.

Vamos organizar um pouco o código. Vamos criar uma função busca:

```

1 //busca a chave dentro do vetor com n elementos.
2 //se encontrou, devolve a primeira posicao i tal que v[i]==chave.
3 //se nao encontrou, devolve -1.
4 // Complexidade caso medio: Theta(n).
5 int busca(int v[], int n, int chave){
6     for(int i=0;i<n;i++){
7         if(v[i]==chave){
8             return i;
9     }

```

```
10     }
11     return -1;
12 }
```

E abaixo a função que calcula o número de elementos na intersecção:

```
1 //Calcula quantos elementos de v1 estao tambem em v2.
2 //v1 eh um vetor com n1 posicoes.
3 //v2 eh um vetor com n2 posicoes.
4 int interseccao(int v1[], int n1, int v2[], int n2){
5     int size = 0;
6     for(int i=0;i<n1;i++){
7         int chave = v1[i];
8         if(busca(v2,n2,chave)>=0){
9             size++;
10        }
11    }
12    return size;
13 }
```

O objetivo agora é trocar a busca em $O(n)$ por uma busca em $O(\lg n)$.

10.2 Busca Binária

Vamos agora ver a busca mais eficiente que é a busca binária. Em termos de complexidade, a busca binária é $O(\lg n)$. O número de operações vai cair de proporcional a n para proporcional a $\lg n$. Lembrando que $\lg 10^9 \approx 30$. Ou seja, se n vale um bilhão, o algoritmo $\lg n$ irá executar algo proporcional a 30 operações ou menos. Isso vai deixar o algoritmo muito mais rápido.

A busca binária só funciona se o vetor já está ordenado. Se o vetor não estiver ordenado, é necessário um passo adicional de ordenação.

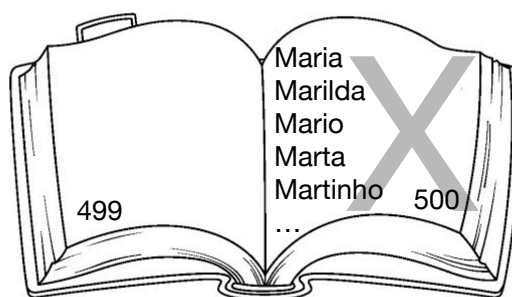
Vamos imaginar que eu tenho um livro de 1000 páginas com nomes e os nomes estão

ordenados em ordem alfabética. O primeiro nome poderia ser Aarom, depois Abba e no final do livro nomes como Zuleica.

Você recebe um nome e precisa encontrar neste livro. Para cada nome também está escrito o telefone da pessoa. Suponha agora que você deseja encontrar o telefone de uma pessoa específica, chamada Florinda.

A primeira busca vai ser feita exatamente na metade do livro. Se o livro tem 1000 páginas eu pego o primeiro nome que está na página 500. Vamos supor que tal nome que é Maria. Eu sei que Florinda vem antes de Maria. Fiz uma única pergunta. Eu posso concluir que Florinda está na primeira metade.

Com uma simples pergunta eu consigo eliminar 50% dos dados. A cada pergunta, o número de elementos cai pela metade. Veja Figura 10.2.



Chave Busca = Florinda

Figura 10.2: Ao buscar-se pelo nome Florinda, abre-se o livro na página 500 que inicia com o nome Maria. Como Florinda vem antes de Maria, elimina-se as páginas maiores ou igual a 500.

Eu poderia tentar dividir não no meio. Dividir na página 300, já que Florinda está no começo do alfabeto. Neste caso, se eu der sorte, Florinda estará nas primeiras 300 páginas, mas, se eu der azar, Florinda estará na 700 páginas restantes. Neste caso eu não garanto que eliminei 50% dos dados.

Para garantir a eliminação de 50% dos dados é preciso comparar o termo de busca com o nome que está no meio. Uma vez eliminado metade da lista, a busca continua de maneira idêntica na metade que sobrou.

Eu conheci um professor que levava uma lista telefônica para sala de aula e, após comparar o nome buscado com o elemento central da lista, ele rasgava a lista ao meio e jogava no lixo uma das metades. Após isso ele repetia o processo até chegar no nome desejado.

Veja a tabela abaixo:

Número de Páginas	Número de consultas	Número de Páginas	Número de consultas
$2^{20} \approx 1.000.000$	0	$2^{10} = 1024$	10
$2^{19} \approx 500.000$	1	$2^9 = 512$	11
$2^{18} \approx 250.000$	2	$2^8 = 256$	12
$2^{17} \approx 125.000$	3	$2^7 = 128$	13
$2^{16} \approx 65.000$	4	$2^6 = 64$	14
$2^{15} \approx 32.000$	5	$2^5 = 32$	15
$2^{14} \approx 16.000$	6	$2^4 = 16$	16
$2^{13} \approx 8.000$	7	$2^3 = 8$	17
$2^{12} \approx 4.000$	8	$2^2 = 4$	18
$2^{11} \approx 2.000$	9	$2^1 = 2$	19
$2^{10} \approx 1.000$	10	$2^0 = 1$	20

Ou seja, se partir de um conjunto de 1.000.000 de páginas, em 20 perguntas ou menos chega na página que está o registro. Se forem 1.000.000 de registros ordenados, em 20 perguntas irá chegar ao registro desejado, ou se concluirá que ele não pertence ao vetor.

O número de vezes que um número pode ser dividido por 2 até que atinja um valor menor ou igual a 1 é $\lg n$. Esta é a definição de $\lg n$. Sendo assim, a complexidade deste algoritmo é $O(\lg n)$.

Veja Figura 10.3.

Veja abaixo o código da busca binária:

```

1 //busca a chave dentro do vetor ordenado
2 //com n elementos. se encontrou, devolve
3 //a uma posicao i tal que v[i]==chave.
4 //se nao encontrou, devolve -1.
5 // Complexidade pior caso: Theta(lg n).
6 int buscaBinaria(int v[], int n, int chave){
```

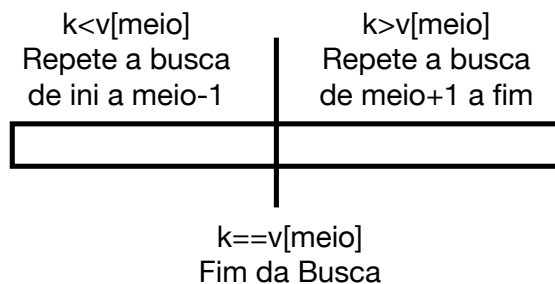


Figura 10.3: Ao realizar a busca binária há três possibilidades: $k < v[\text{meio}]$, $k == v[\text{meio}]$ e $k > v[\text{meio}]$. No primeiro caso, repete-se a busca entre *ini* e *meio* - 1. No segundo caso, encontrou o elemento e é o fim da busca. No terceiro caso, repete-se a busca entre *meio* + 1 e *fim*.

```

7   int ini = 0;
8   int fim = n-1;
9   while(ini<=fim){
10      int meio = (ini+fim)/2;
11      if(v[meio]==chave){
12          return meio;
13      }
14      if(chave<v[meio]){
15          fim = meio-1;
16      }else{
17          ini = meio+1;
18      }
19  }
20  return -1;
21 }
```

Um detalhe é que a cada passo, pelo menos um elemento é removido do vetor, que é o $v[\text{meio}]$. Isso garante que o algoritmo não caia em um loop infinito. Outro ponto, é que não

é eliminado exatamente a metade do vetor. Se o número de elementos é ímpar, a próxima etapa terá $(n - 1)/2$, um pouco menos que a metade. Se n é par, a próxima iteração terá $n/2 - 1$ ou $n/2$ elementos se for a primeira ou a segunda metade, respectivamente.

Apesar de ser mais ineficiente, é possível fazer naturalmente a busca binária de maneira recursiva. Veja o código abaixo:

```
1 int buscaBinariaRec(int v[], int ini, int fim, int chave){
2     if(ini<=fim){
3         int meio = (ini+fim)/2;
4         if(chave==v[meio])return meio;
5         if(chave<v[meio]){
6             return buscaBinariaRec(v,ini,meio-1,chave);
7         }else{
8             return buscaBinariaRec(v,meio+1,fim,chave);
9         }
10    }else{
11        return -1;
12    }
13 }
```

O código verifica se o vetor tem pelo menos 1 elemento. Se não for este o caso, devolve -1. Se possuir mais de um elemento, calcula o elemento central *meio*. Se $k == v[meio]$, finaliza devolvendo *meio*. Se $k < v[meio]$, faz uma chamada recursiva passando como limites *ini* e *meio* - 1. Se $k > v[meio]$, faz a chamada recursiva passando como limites *meio* + 1 até o *fim*.

Vamos supor que o vetor ordenado é [1, 2, 4, 6, 8, 9, 12, 15, 20] e que a chave de busca é 3. Vamos executar o algoritmo da busca binária passo a passo.

Primeiro, *ini* = 0 e *fim* = 8. Agora calcula-se o meio, que é $meio = (8 + 0)/2 = 4$. Compara-se a *chave* = 3 com $v[4] = 8$. Como a chave é menor, $fim = meio - 1 = 3$

Enquanto $ini \leq fim$. Entra neste enquanto, pois *ini* = 0 e *fim* = 3. Calcula-se $meio = (0 + 3)/2 = 1$. Compara-se a *chave* = 3 com $v[1] = 2$. Como a chave é maior,

$ini = meio + 1 = 2$

Enquanto $ini \leq fim$. Entra neste enquanto, pois $ini = 2$ e $fim = 3$. Calcula-se $meio = (2 + 3)/2 = 2$. Compara-se a $chave = 3$ com $v[2] = 4$. Como a chave é menor, $fim = meio - 1 = 1$

Enquanto $ini \leq fim$. Sai do laço, pois $ini = 2$ e $fim = 1$. Devolve -1 .

Questão 114) Mostre o passo a passo do algoritmo ao executar no vetor anterior, mas com a chave de busca igual a 15.

Questão 115) Faça a seguinte modificação no algoritmo da busca binária, mudando a busca para ternária. Divida o vetor em três partes, cada uma contendo $1/3$ dos elementos. Os pontos de divisão são $terco_1$ e $terco_2$. Verifique se a chave é igual a $v[terco_1]$ ou $v[terco_2]$ ou se está no primeiro terço, segundo terço ou terceiro terço do vetor. Se necessário, continue a busca ternária no terço correto. O vetor já está ordenado. Se a chave de busca pertencer ao vetor, devolver a posição do elemento, caso contrário, deve-se devolver -1 .

Questão 116) O algoritmo da questão anterior é melhor ou pior que a busca binária?

Questão 117) Suponha que governo do estado de São Paulo calcula um índice sobre os salários de seus funcionários. Dado um conjunto de n salários, toda vez que existirem dois salários s_i, s_j tal que $s_i < 2000$ e $s_j > 5000$, o índice é acrescentado de $1/n^2$. Veja o código abaixo:

```

1 double indice(int salar[], int n){
2     double idx = 0;
3     for(int i=0; i<n; i++){
4         for(int j=0; j<n; j++){
5             if(salar[i]<2000.0
6                 && salar[j]>5000.0){
7                 idx+=1.0;
8             }
9         }
10    }
11    return idx/(n*n*1.0);
12 }
```

Suponha que o vetor de salários está ordenado. Mostre como calcular o índice em $O(\lg n)$. Implemente a solução e teste para alguns valores.

Questão 118) Ainda sobre o código da questão anterior. Suponha que os programas levam tempo $T_1(n) = n^2 10^{-9}s$ e $T_2(n) = \lg n 10^{-9}s$. O algoritmo $O(\lg n)$ é quantas vezes mais rápido que o programa $O(n^2)$ para $n = 5000$? Mostre os cálculos.

10.3 Intersecção de Conjunto em $O(n \lg n)$

Para realizar intersecção de conjuntos em $O(n \lg n)$ são necessários dois passos.

- Ordenar o vetor v_2 .
- Buscar cada elemento de v_1 em v_2 usando busca binária.

Veja como fica o algoritmo da intersecção de conjuntos:

```
1 //Calcula quantos elementos de v1 estao tambem em v2.
2 //v1 eh um vetor com n1 posicoes.
3 //v2 eh um vetor com n2 posicoes.
4 int interseccao(int v1[], int n1, int v2[], int n2){
5     mergeSort(v2,n2);
6     int size = 0;
7     for(int i=0;i<n1;i++){
8         int chave = v1[i];
9         if(buscaBinaria(v2,n2,chave)>=0){
10             size++;
11         }
12     }
13     return size;
14 }
```

O algoritmo $O(n^2)$ teve um tempo estimado de execução de mais de 13 dias. Este algoritmo de interseção, que é $O(n \lg n)$ foi executado e gastou apenas 25s para obter a resposta. Para ser mais preciso, o algoritmo tem complexidade $O(n_2 \lg n_2 + n_1 \lg n_2)$ que é $O((n_1 + n_2) \lg n_2)$. Isso porque é feita a ordenação do vetor v_2 em $O(n_2 \lg n_2)$, depois, para cada elemento de v_1 é feita uma busca binária em v_2 .

Veja como é simples ordenar um vetor uma vez que o MergeSort está implementado. Basta fazer a chamada ao algoritmo de ordenação.

Questão 119) Suponha dois vetores de inteiros, v_1 e v_2 , com tamanhos n_1 e n_2 . O vetor v_1 está ordenado e v_2 também está ordenado. Suponha que não existem elementos repetidos dentro de cada vetor. Calcule o número de elementos na intersecção dos vetores, gastando $O(n_1 + n_2)$.

Questão 120) Supondo que os vetores não estão ordenados, seria possível calcular a intersecção em $O(n_1 + n_2)$.

Capítulo 11

Aula 11

11.1 Definição de Árvores

Em teoria dos grafos, uma árvore é **um grafo conexo e acíclico**. Ainda não estudamos grafos, mas, simplificadamente, um grafo é um conjunto de elementos, chamados de nós ou vértices e um conjunto de relações ou conexões entre estes elementos, chamadas arestas. Como exemplo, você pode imaginar o grafo do Facebook, onde cada vértice é um usuário e existe uma conexão entre dois vértices se existir uma relação de amizade entre eles.

A Figura 11.1 tem exemplos de grafos. Dos grafos mostrados, apenas um é acíclico (sem ciclos). Um ciclo em um grafo é uma sequência fechada de vértices conectados por arestas. Por exemplo, no último grafo, a sequência $abcd$ forma um ciclo. A sequência $abda$ também forma ciclo.

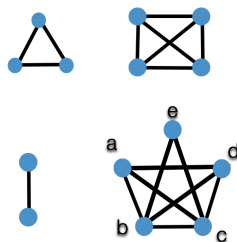


Figura 11.1: Exemplos de grafos.

Veja o grafo da Figura 11.2 esquerda. Este grafo não é uma árvore, pois existem dois

ciclos.

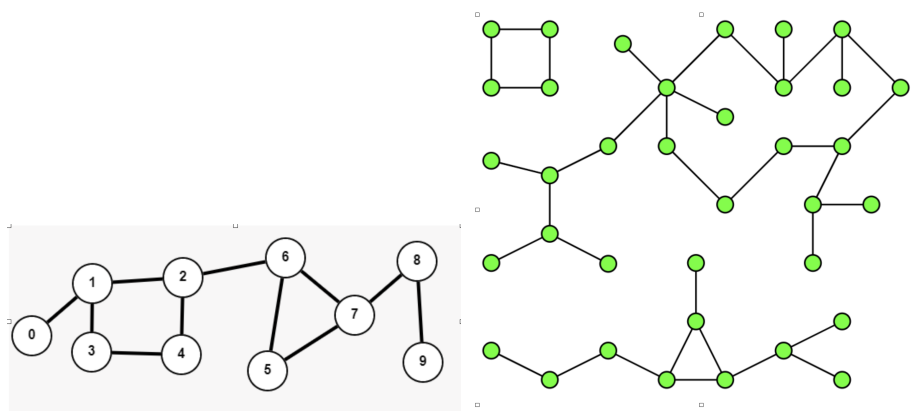


Figura 11.2: Exemplo de grafo com ciclos. O grafo da esquerda é conexo e o da direita é desconexo

A primeira condição para um grafo ser uma árvore, é que ele precisa ser acíclico (sem ciclos). A segunda característica é que ele precisa ser conexo. Um grafo é conexo se existe um caminho entre quaisquer par de vértices. Por exemplo, o grafo do Facebook é conexo? Provavelmente sim. Provavelmente existe um caminho entre qualquer par de usuários do Facebook. Podemos partir de qualquer ponto e chegar a qualquer outro usuário seguindo conexão de amizade. Mas, se existirem, por exemplo, três usuário na Sibéria que são amigos entre si e de mais ninguém, o grafo do Facebook deixa de ser conexo. Veja o grafo da Figura 11.2 direita. Este grafo é desconexo pois possui três componentes conexas, ou seja, possui três conjuntos maximais de vértices que são conexos entre si, mas isolados dos demais.

Finalmente chegamos a exemplos de árvores, que são grafos conexos e acíclicos. Veja alguns exemplos de árvores na Figura 11.3. Note que os três grafos são conexos e acíclicos, ou seja, árvores.

Em estrutura de dados, estamos interessados em um tipo especial de árvore, que é a árvore enraizada. Curiosamente, a raiz da árvore, quando desenhada, fica na parte de cima. Cada par de nó conectado possui uma relação de pai e filho entre si. Pai é o nó acima e filho é o nó abaixo. Nós que não possuem filhos são chamados de folha e ficam, geralmente na parte de baixo. O nó que fica acima de todos é o nó raiz. Veja, na Figura 11.4, um exemplo de árvore enraizada. A raiz é o nó 1. Para dar alguns exemplos, o nó 1 é pai do nó 7 e do

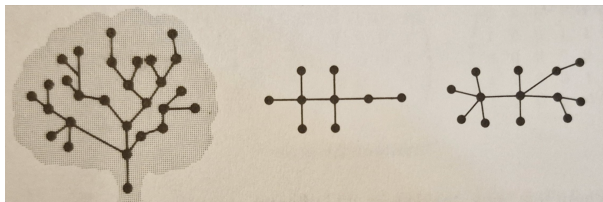


Figura 11.3: Exemplo de árvores. Fonte [Wilson and Watkins, 1990].

9. Os nós 2 e 6 são filhos do nó 7. Os nós 2, 5, 11 e 5 são folhas.

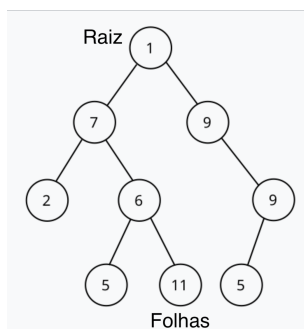


Figura 11.4: Exemplo de árvores enraizada. Fonte [Wikimedia Commons contributors, 2024a].

Um exemplo de árvores são as árvores de diretórios. Existe um nó raiz, onde a árvore se inicia. Cada diretório é filho de algum outro diretório até chegar nas folhas, que são os arquivos. Veja Figura 11.5.

Árvore é uma estrutura de dados naturalmente recursiva, pois cada filho de um nó também é uma árvore. Veja Figura 11.6.

Existe um tipo especial de árvore que é a árvore binária. O nó da árvore binária possui, no máximo, dois filhos. A árvore da Figura 11.4 é binária. Já a árvore da Figura 11.5 não é binária, pois a Pasta 2 possui três filhos.

Questão 121) Considere uma árvore com n vértices. Esta árvore possui quantas arestas?

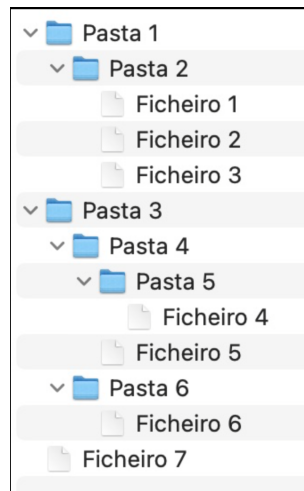


Figura 11.5: Exemplo de árvore de diretórios. Fonte [Wikimedia Commons contributors, 2024b]

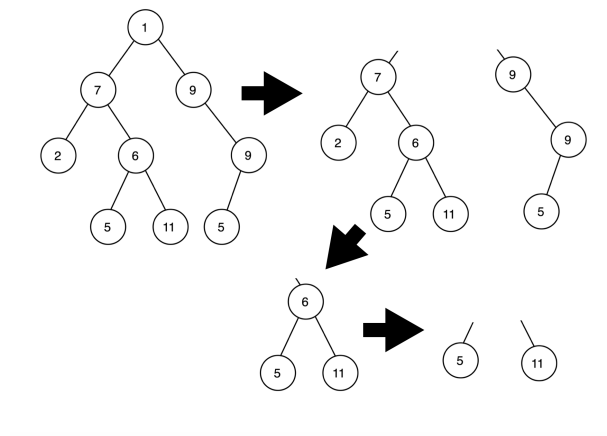


Figura 11.6: Note que as subárvores de uma árvore também são árvores.

11.2 Implementação Árvore Binária

A estrutura básica de uma árvore binária é o nó. Nó possui um valor e dois apontadores, um para a esquerda e um para a direita.

```

1 //Estrutura No
2 typedef struct No {
3     int valor;
  
```

```
4     struct No *esq, *dir;  
5 }No;
```

Veja Figura 11.7.

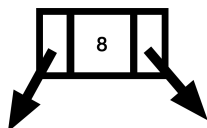


Figura 11.7: Exemplo de uma estrutura nó.

Nesta apostila eu tenho utilizado um padrão de construtor de estruturas. Veja como fica o construtor da estrutura nó.

```
1 //construtor da estrutura noh.  
2 No * newNo(int valor){  
3     No * no = (No *)malloc(sizeof(No));  
4     no->valor = valor;  
5     no->esq = NULL;  
6     no->dir = NULL;  
7     return no;  
8 }
```

Existem dois padrões para se fazer uma árvore. A primeira é definir a árvore como um apontador para nó, *No *tree*. O segundo padrão é utilizar uma estrutura chamada de árvore que guarda um apontador para a raiz da árvore, como mostrado abaixo:

```
1 //estrutura arvore  
2 typedef struct Tree {  
3     No * raiz;  
4 }Tree;
```

E o seu construtor:

```
1 //construtor da estrutura arvore
2 Tree * newTree(){
3     Tree * t = (Tree *)malloc(sizeof(Tree));
4     t->raiz = NULL;
5     return t;
6 }
```

Eu preferi utilizar esta estrutura árvore. A desvantagem é a necessidade de uma estrutura adicional. A vantagem é uma melhor padronização do código para métodos como *add*. No caso de um *add* em uma árvore vazia utilizando o primeiro padrão, temos a necessidade de receber apontador para apontador para nó, enquanto utilizando o padrão desta apostila, basta um apontador para árvore. É uma questão de gosto, mas eu optei por utilizar uma estrutura adicional guardando o apontador para a raiz da árvore.

Questão 122) Ao final desta aula, refaça os códigos sem utilizar a estrutura árvore. Utilize somente a estrutura nó. Veja os benefícios e malefícios de se utilizar este segundo padrão de codificação.

Veja como fica a função *main* criando uma árvore simples.

```
1 int main(int argc, const char * argv[]) {
2     Tree * t = newTree();
3     t->raiz = newNo(5);
4     t->raiz->esq = newNo(6);
5     t->raiz->dir = newNo(7);
6     t->raiz->dir->esq = newNo(2);
7     return 0;
8 }
```

A árvore criada no código acima está mostrada na Figura 11.8.

Existe um tipo especial de árvore que é a árvore binária **de busca**. Neste tipo de árvore, ao se inserir um elemento, se ele for menor ou igual a raiz, ele é inserido no ramo esquerdo.

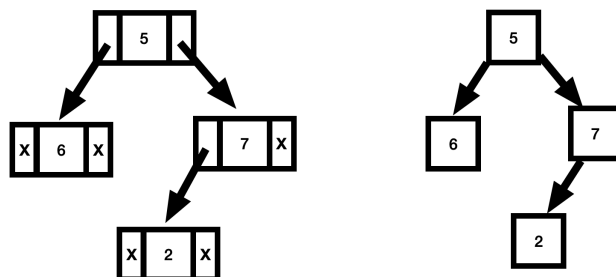


Figura 11.8: Árvore binária criada na função *main*.

Se ele for maior que a raiz, ele é inserido no ramo à direita da árvore. Esta propriedade vale para as subárvores. Veja Figura 11.9, onde foram inseridos os elementos (5, 3, 7, 5) numa árvore binária de busca.

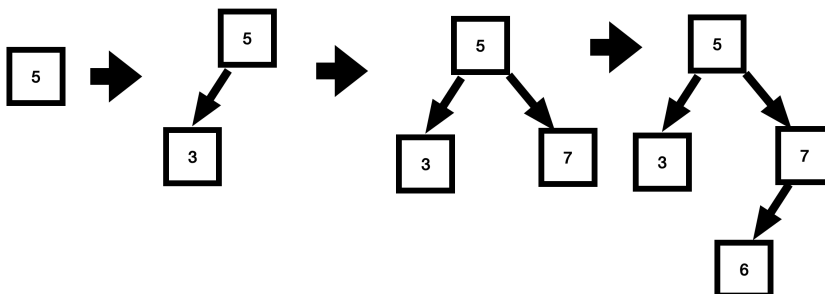


Figura 11.9: Inserção dos elementos (5, 3, 7, 6) numa árvore binária de busca.

Veja que o primeiro nó, que é o 5, foi inserido na raiz. O próximo elemento é o 3. Como ele é menor que o 5, será inserido no ramo à esquerda. Em seguida o 7, que é maior que a raiz, logo será inserido no ramo à direita. Neste momento, falta inserir apenas o nó 6. Veja que 6 é maior que a raiz, logo irá para o ramo à direita. Considerando o 7 como a nova raiz, o nó 6 será inserido no ramo à esquerda, pois é menor que o 7. O passo a passo está mostrado na Figura 11.10.

Suponha agora que os mesmos nós foram inseridos em uma outra ordem diferente. Considere a seguinte ordem (3, 6, 7, 5). Veja a Figura 11.10.

Um caso considerado ruim é quando os elementos estão ordenados e são inseridos em

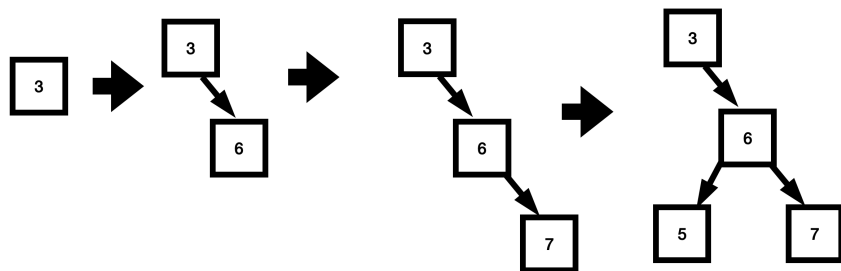


Figura 11.10: Inserção dos elementos (3, 6, 7, 5) numa árvore binária de busca.

uma árvore binária de busca. Veja o que acontece se inserirmos os elementos em ordem, ou seja, inserir (3, 5, 6, 7). Veja Figura 11.11. A árvore ficou degenerada. Ficou semelhante a uma lista ligada.

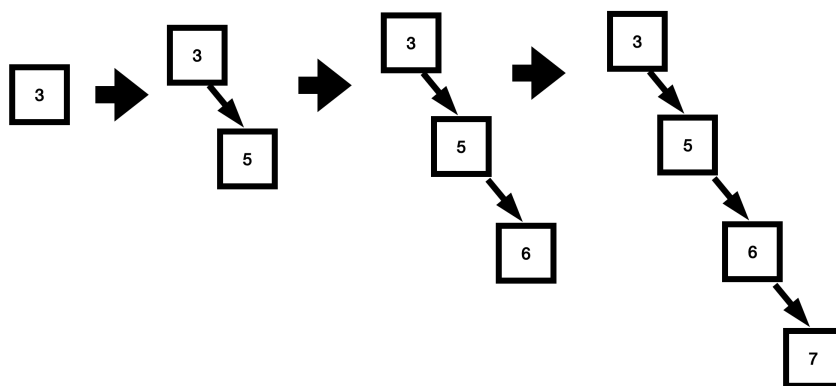


Figura 11.11: Inserção dos elementos (3, 5, 6, 7) numa árvore binária de busca. Note que, por os elementos estarem ordenados, a árvore ficou degenerada, parecendo uma lista ligada.

Questão 123) Descubra a árvore que será gerada ao se inserirem os elementos (7, 3, 9, 3, 10, 1, 8) em uma árvore binária de busca.

Questão 124) Insira os elementos abaixo em árvore binárias de busca. São três árvores: A, B e C.

A: (7, 2, 0, 6, 5, 9, 3, 1, 4, 8)

B: (0, 9, 6, 4, 8, 2, 7, 5, 3, 1)

C: (0, 1, 2, 3, 4, 9, 6, 8, 7, 5)

Seja h_a , h_b e h_c a altura das árvores A, B e C, respectivamente. Quanto vale $x = h_a + h_b + h_c$?
 Você deve mostrar as árvores A, B e C as respectivas alturas.

Questão 125) Supondo que os elementos (3, 5, 6, 7) podem ser inseridos em qualquer ordem, monte todas as árvores binárias de busca distintas com estes quatro elementos.

Vamos agora fazer o código que insere um elemento em uma árvore binária de busca. Veja o código abaixo:

```

1 //Insere o valor na arvore
2 void addTree(Tree *t, int valor){
3     if(t->raiz==NULL){
4         t->raiz = newNo(valor);
5     }else{
6         addNo(t->raiz, valor);
7     }
8 }
```

Esta parte do código é um tratamento especial no caso da raiz valer nulo. Neste caso, é necessário criar um nó e atribuir à raiz da árvore. Caso a raiz não valha nulo, é chamado a função *addNo* que insere um elemento em uma árvore representada por um nó. Veja que a função *addNo* sempre será chamada passando como parâmetro um nó que não vale nulo. Isso é importante na hora de implementar o *addNo*. Veja, abaixo, a função *addNo*. Esta implementação é uma implementação recursiva.

```

1 /**
2  Insere o valor na arvore.
3  O noh nao pode ser nulo por definicao.
4  */
5 void addNo(No* no, int valor){
6     if(valor<=no->valor){
7         //insere na subarvore aa esquerda
```

```
8         if (no->esq != NULL){
9             addNo(no->esq, valor);
10        }else{
11            no->esq = newNo(valor);
12        }
13    }else{
14        //insere na subarvore aa direita
15        if (no->dir != NULL){
16            addNo(no->dir, valor);
17        }else{
18            no->dir = newNo(valor);
19        }
20    }
21 }
```

Veja que o código para inserir um elemento tenta do lado esquerdo ou do lado direito, caso valor seja menor ou maior, respectivamente. Suponha que o valor vai para o lado esquerdo. Se ali existe um apontador nulo, achamos o lugar onde o valor entra na árvore. Este apontador que valia nulo, passa a apontar para o novo nó que acabou de ser criado. Se o valor deve ser inserido do lado esquerdo e ali existe uma subarvore, basta fazer uma chamada recursiva, inserindo o valor nesta árvore. O apontador $no \rightarrow esq$ é passado como parâmetro da função recursiva e será considerada a nova raiz da árvore. O processo se repete até que haja um apontador nulo onde o nó pode ser inserido. Expliquei para o caso que o nó vai para o lado esquerdo, mas o caso do lado direito é idêntico. Veja Figura 11.12.

A complexidade do algoritmo *addNo* é $O(h)$ onde h é a altura da árvore. A altura de uma árvore é definida como o maior caminho da raiz até uma folha. Por definição, uma árvore nula tem altura igual a -1. Veja algumas alturas de árvores na Figura 11.13.

Em breve nesta apostila iremos fazer um algoritmo para calcular a altura de uma árvore. Se a árvore tiver um bom fator de balanceamento, ela terá altura $h = O(\lg n)$. Entretanto, se a árvore estiver desbalanceada, a altura poderá atingir até $h = O(n)$. Existem maneiras

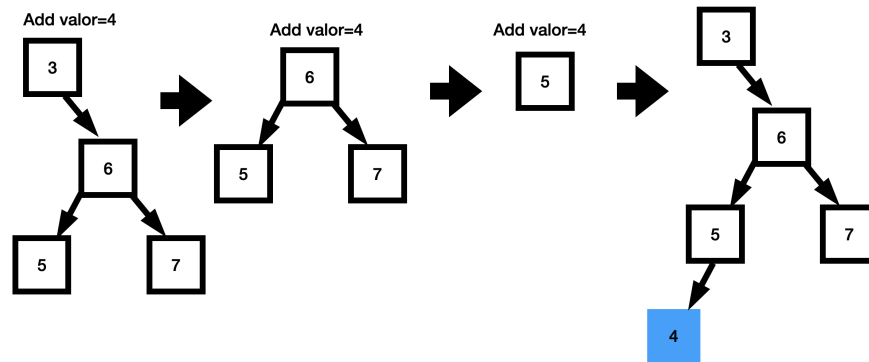


Figura 11.12: Inserindo o valor 4 em uma árvore binária de busca. Note que o problema é resolvido recursivamente, passando como parâmetro uma subárvore.

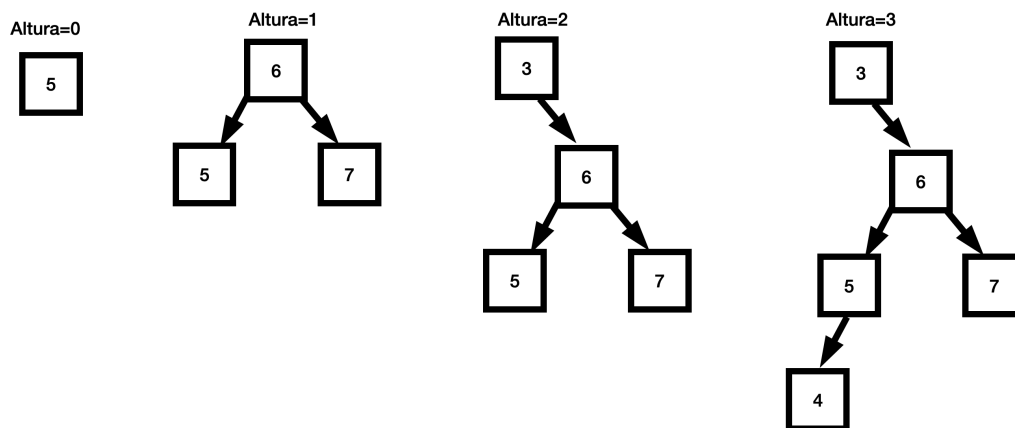


Figura 11.13: Exemplo de alturas de árvores.

de inserir elementos em uma árvore e garantir que ela possui altura $\lg n$. É o caso de um tipo especial de árvore chamada AVL. Entretanto, nesta aula, não estamos garantindo que a árvore terá altura $\lg n$. A seguir o código de inserção na árvore sem usar recursão, ou seja, inserção iterativa.

```

1 #define TRUE 1
2 #define FALSE 0
3 //Insercao em uma arvore de
4 //maneira iterativa

```

```
5 void addTreeIt(Tree *t, int valor){
6     if(t->raiz==NULL){
7         t->raiz = newNo(valor);
8         return;
9     }
10    No * aux = t->raiz;
11    int inseriu = FALSE;
12    while(!inseriu){
13        if(valor<aux->valor){
14            if(aux->esq!=NULL){
15                //descer para o ramo esquerdo
16                aux=aux->esq;
17            }else{
18                aux->esq = newNo(valor);
19                inseriu=TRUE;
20            }
21        }else{
22            if(aux->dir!=NULL){
23                //descer para o ramo direito
24                aux=aux->dir;
25            }else{
26                aux->dir = newNo(valor);
27                inseriu=TRUE;
28            }
29        }
30    }
31 }
```

Este código é baseado em uma variável *aux* do tipo apontador para nó que é colocada na raiz da árvore. Se valor for menor igual a *aux* $\rightarrow$ *valor* ele desce para o ramo esquerdo da

árvore caso exista tal ramo. Caso não exista tal ramo, foi encontrado o ponto correto para se inserir o valor. Neste caso, cria-se um novo nó e atribui a $aux \rightarrow esq$. Caso o valor deva ser inserido no ramo à direita, o algoritmo é idêntico. Veja a Figura 11.14

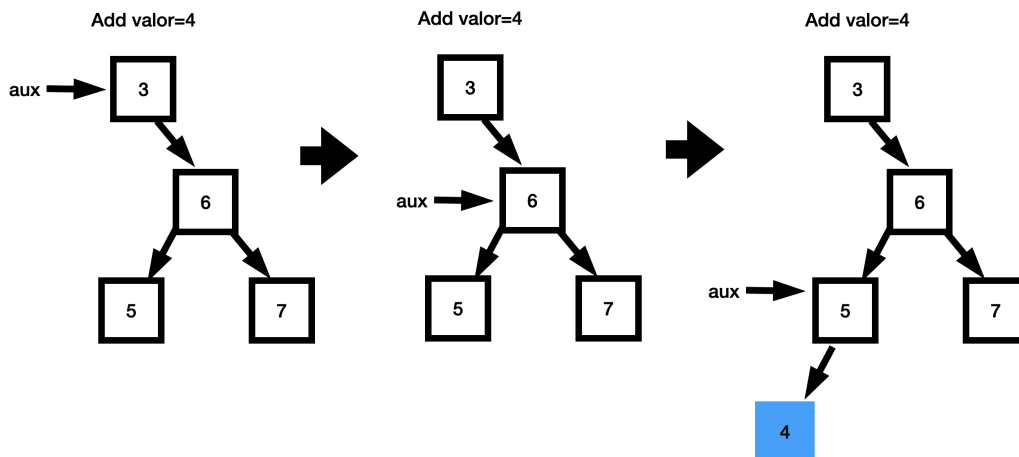


Figura 11.14: Inserindo o valor 4 em uma árvore binária de busca por um método iterativo.

Questão 126) Implemente uma função que encontra o valor máximo em uma árvore binária de busca. Implemente também a função máximo caso a árvore não tenha a propriedade de ser *de busca*. De maneira análoga, implemente as funções mínimo em uma árvore de busca e em um árvore não de busca.

11.3 Imprimindo uma Árvore Binária

Um algoritmo importante consiste na impressão de uma árvore. Veja o código abaixo que faz uma impressão simplificada de uma árvore:

```

1 //impressao de uma arvore
2 void printNo(No * no){
3     if(no==NULL){
4         return;
5     }
6     //ramo da esquerda
```

```

7     printNo(no->esq);
8     //raiz
9     printf("%d,",no->valor);
10    //ramo da direita
11    printNo(no->dir);
12 }
13 void printTree(Tree *t){
14     printf("[");
15     if(t->raiz!=NULL) printNo(t->raiz);
16     printf("]\n");
17 }

```

Este código é recursivo. Começa imprimindo o ramo da esquerda, recursivamente, depois imprime a raiz, que é a impressão de fato, em seguida o ramo da direita. Essa impressão se chama **em-ordem**. Se começasse pela raiz, seria pré-ordem. Se terminasse pela raiz, seria pós-ordem. Veja que para a impressão funcionar, basta que seja uma árvore binária, não necessariamente de busca. Caso a árvore seja uma árvore binária de busca, os elementos serão impressos em ordem crescente. Sendo assim, temos um novo algoritmo de ordenação que consiste em: *(i)* inserir todos os elementos em uma árvore binária de busca e *(ii)* fazer um percurso recursivo visitando os nós à esquerda, depois a raiz e depois os nós à direita. Veja alguns exemplos de impressão na Figura 11.15

Questão 127) Mostre a impressão em-ordem das árvores da Figura 11.16

A impressão de uma árvore binária em toda a sua complexidade não é uma tarefa muito simples. Poderíamos, ao fazer o percurso em-ordem utilizar colchetes para indicar quais são os ramos da árvore. Neste caso, para as árvores da Figura 11.15 teríamos a seguinte impressão:

```

[5]
[[5]3[7]]
[9[[5]3[7]]]
[9[[[10]5]3[7]]]

```

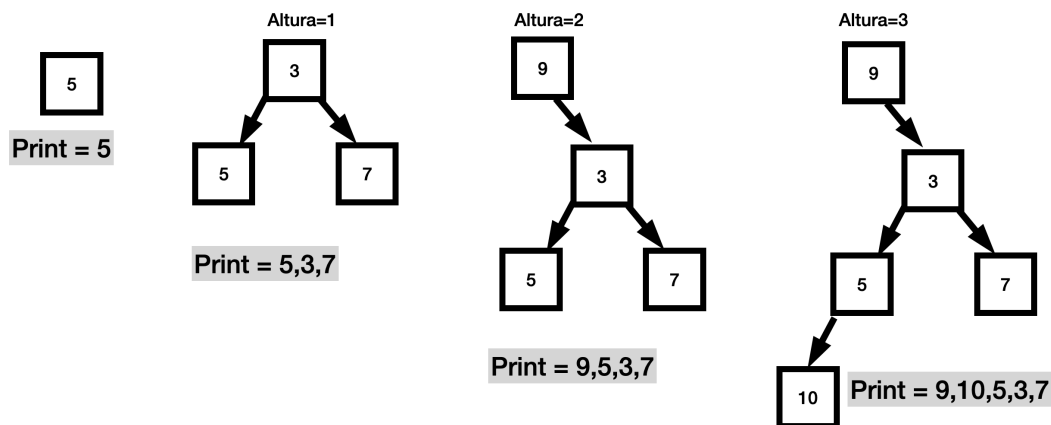


Figura 11.15: Exemplo de impressão das árvores binárias em-ordem.

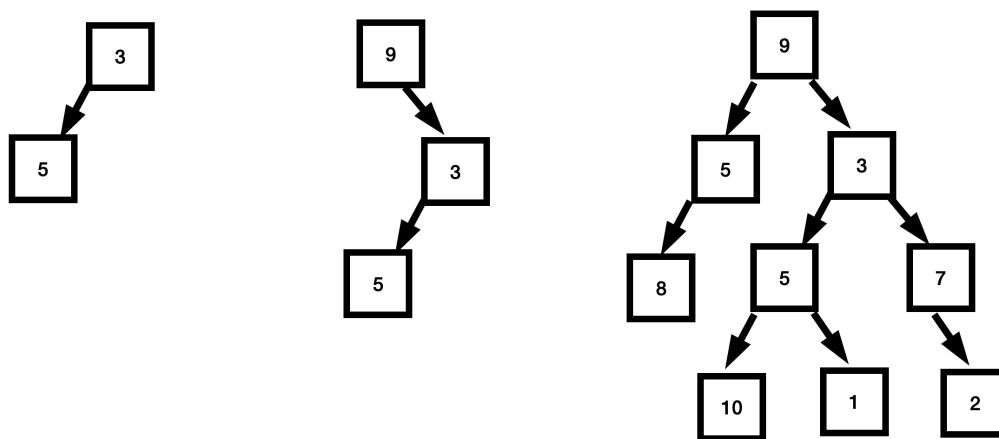


Figura 11.16: Mostre a impressão em-ordem das árvores acima.

Questão 128) Faça a impressão da árvore em-ordem, delimitando os ramos da árvore por meio de colchetes.

Não é a melhor forma, mas ajuda a depurar um código. Suponha agora uma árvore com milhares de nós. Neste caso, como poderíamos visualizar a árvore para analisar se ela está correta ou errada? A melhor forma que eu pensei foi a utilização de uma estrutura XML. O ramo à esquerda fica entre os marcadores `<esq>` e `</esq>` e o ramo à direita fica entre os marcadores `<dir>` e `</dir>`. Veja um exemplo na Figura 11.17

Abrindo este arquivo XML no Chrome, ele interpreta o XML e permite a navegação

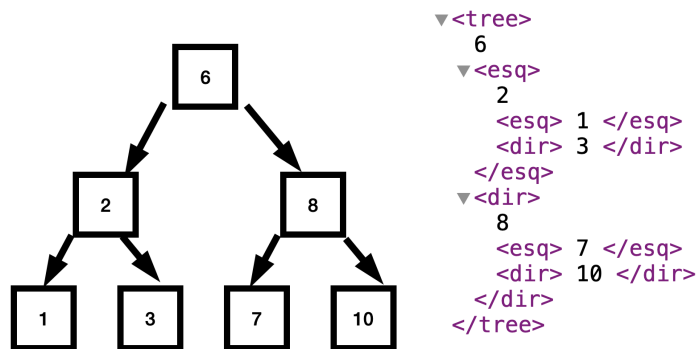


Figura 11.17: Árvore à esquerda e sua representação em XML à direita.

na árvore, mesmo que ela possua milhares de nós, permitindo visualização e depuração do código. Fiz uma árvore inserindo 1000 números aleatórios nela, onde o valor está uniformemente distribuído entre 0 e 999. Veja a navegação na árvore usando o Google Chrome na Figura 11.18.

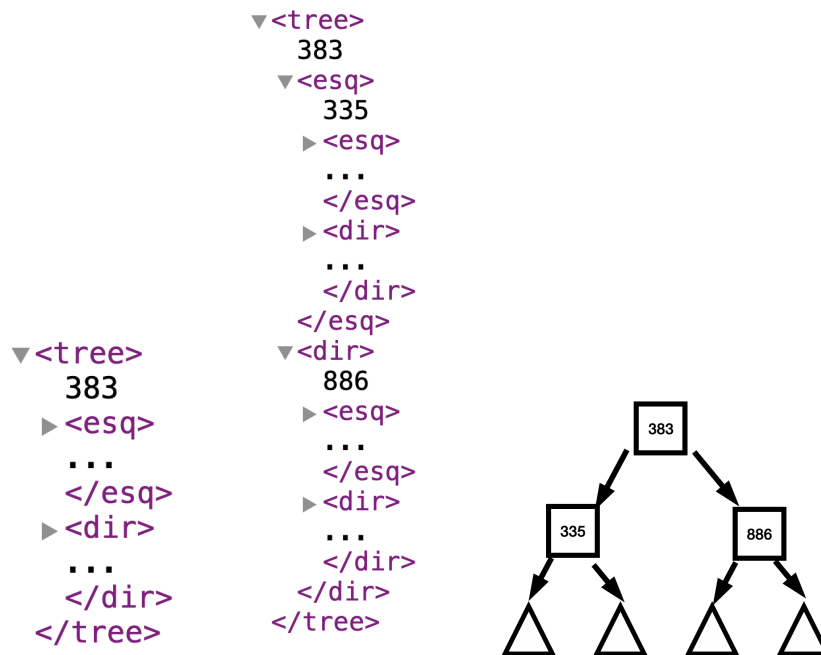


Figura 11.18: Navegação em um XML de uma árvore binária de busca impressa em XML e aberta no Google Chrome. À direita, um desenho da árvore.

Segue o código que faz a impressão da árvore em XML. Primeiro o código que recebe a estrutura árvore.

```
1 //imprime a arvore em XML
2 void printTreeXML(Tree *t){
3     FILE * fp = fopen("Tree.xml","w+");
4     fprintf(fp,"<tree>\n");
5     if(t->raiz!=NULL)
6         printNoXML(t->raiz,fp);
7     fprintf(fp,"</tree>\n");
8     fclose(fp);
9 }
```

E agora o código que imprime a árvore propriamente dita:

```
1 //imprime a arvore em XML
2 void printNoXML(No *no,FILE * fp){
3     fprintf(fp,"%d\n",no->valor);
4     if(no->esq!=NULL){
5         fprintf(fp,"<esq>\n");
6         printNoXML(no->esq,fp);
7         fprintf(fp,"</esq>\n");
8     }
9     if(no->dir!=NULL){
10        fprintf(fp,"<dir>\n");
11        printNoXML(no->dir,fp);
12        fprintf(fp,"</dir>\n");
13    }
14 }
```

O código primeiro imprime o valor da raiz. Depois abre uma marcação esquerda e faz a impressão recursiva de todo o ramo à esquerda, caso exista. Em seguida, abre uma marcação

direita e imprime todo o ramo à direita, caso exista. O primeiro código, *printTreeXML* trata o caso de impressão da raiz, criando um marcador *tree* ao redor da árvore.

Veja a função *main* chamando os códigos implementados:

```
1 int main(int argc, const char * argv[]) {
2     Tree * t = newTree();
3     addTree(t, 6);
4     addTree(t, 8);
5     addTree(t, 2);
6     addTree(t, 10);
7     addTree(t, 7);
8     addTree(t, 1);
9     addTree(t, 3);
10    printTree(t);
11    printTreeXML(t);
12    return 0;
13 }
```

A função *printTree* terá a seguinte saída, que são os dados impressos de maneira ordenada.

[1,2,3,6,7,8,10,]

Questão 129) Crie o XML das árvores da Figura 11.16 e navegue nas árvores usando o Google Chrome.

Questão 130) Faça um programa que insere 1000 números pseudoaleatórios na árvore binária de busca. Os valores pseudoaleatórios devem ser uniformemente distribuídos entre 0 e 999. Crie o XML e navegue na árvore. Desenhe os primeiros 4 níveis da árvore. Seu desenho deve ser semelhante ao desenho à direita na Figura 11.18, só que com os valores dos primeiros quatro níveis.

Questão 131) Responda às seguintes questões referente a árvore a seguir. Considere uma árvore ternária, com três filhos. Veja a nova estrutura nó.

```
1 typedef struct No {  
2     int valor;  
3     struct No * esq;  
4     struct No * meio;  
5     struct No * dir;  
6 }No;
```

A árvore tem as seguintes propriedades no momento da inserção. Elementos menores que a raiz devem ser inseridos à esquerda. Elementos maiores que a raiz devem ser inseridos à direita e elementos iguais à raiz devem ser inseridos no meio.

Para os itens abaixo use um gerador de números pseudoaleatório e coloque o seu RA como semente, para gerar sempre a mesma sequência.

1. Refaça o método para adicionar um nó nesta nova árvore.
2. Refaça o método que imprime esta árvore no formato XLM.
3. Insira 30 nós nesta árvore. Os nós devem ser gerados como números pseudoaleatórios entre 0 e 19.
4. Faça um desenho desta árvore.
5. Crie a árvore novamente e insira 1000 nós nesta árvore. Os nós devem ser gerados como números pseudoaleatórios entre 0 e 299.
6. Faça desenho, mostrando o conteúdo dos 5 primeiros níveis da árvore. A partir do sexto nível, deve ser desenhando um triângulo representando toda a subárvore daquele ponto em diante, como na Figura 11.18.

11.4 Método de Ordenação Baseado em Árvores de Binárias de Busca

Vamos agora implementar um algoritmo de ordenação baseado na árvore binária de busca. Suponha que desejamos ordenar uma lista salva em um vetor com a estrutura abaixo:

```
1 //Lista salva em vetor
2 typedef struct Lista{
3     int *v;
4     int top,max;
5 }Lista;
```

As funções construtoras e para adicionar um elemento na lista já estão prontos conforme visto em aulas anteriores desta apostila. O objetivo agora é ordenar esta lista usando a árvore binária de busca. O primeiro passo consiste em colocar todos os elementos da lista em uma árvore binária de busca, conforme mostrado abaixo:

```
1 void TreeSort(Lista *l){
2     if(l->top<=1){
3         return ;
4     }
5     //insere os elementos da lista
6     //em uma arvore binaria de busca
7     Tree *t = newTree();
8     for(int i=0;i<l->top;i++){
9         addTree(t,l->v[i]);
10    }
11    //deixar o vetor vazio.
12    l->top=0;
13    percurso(t->raiz,l);
14 }
```

Agora, os dados já estão inseridos na árvore. Basta fazer um percurso em-ordem só que, ao invés de imprimir na tela, insere na lista. Veja a função:

```

1 void percurso(No * no, Lista *l){
2     if(no==NULL){
3         return;
4     }
5     percurso(no->esq,l);
6     addLista(l,no->valor);
7     percurso(no->dir,l);
8 }

```

Questão 132) O que acontece se os dados da lista estiverem originalmente ordenados? Qual a complexidade neste caso? Proponha uma melhoria para o *TreeSort* de maneira a ter complexidade de tempo esperada em $O(n \lg n)$.

Questão 133) O método *TreeSort* está com vazamento de memória. Ou seja, são alocados alguns nós e depois perde-se os apontadores para estes nós, ficando uma parte da memória gasta, porém inacessível. Suponha que você foi contratado para remover este vazamento de memória. Resolva este problema.

Questão 134) Implemente *TreeSort* para ele ordenar uma lista ligada ao invés de um lista em vetor.

Capítulo 12

Aula 12

12.1 Busca em uma árvore binária de busca

O processo de busca é parecido com o processo de inserção. Ao buscar um elemento, verifica-se se ele é igual a raiz. Se for igual, a busca termina. Se for menor, a busca continua do lado esquerdo e se for maior, a busca continua do lado direito. Este código pode ser feito tanto de maneira recursiva quanto iterativa.

Veja a versão recursiva.

```
1 No * buscaRec(No *no,int valor){
2     if(no==NULL)
3         return NULL;
4     else if(no->valor==valor)
5         return no;
6     else if(valor<no->valor)
7         return buscaRec(no->esq,valor);
8     else
9         return buscaRec(no->dir,valor);
10 }
```

Veja um exemplo de busca executando este código na Figura 12.1

A complexidade da busca é $O(h)$ onde h é a altura da árvore. Veja a versão iterativa do

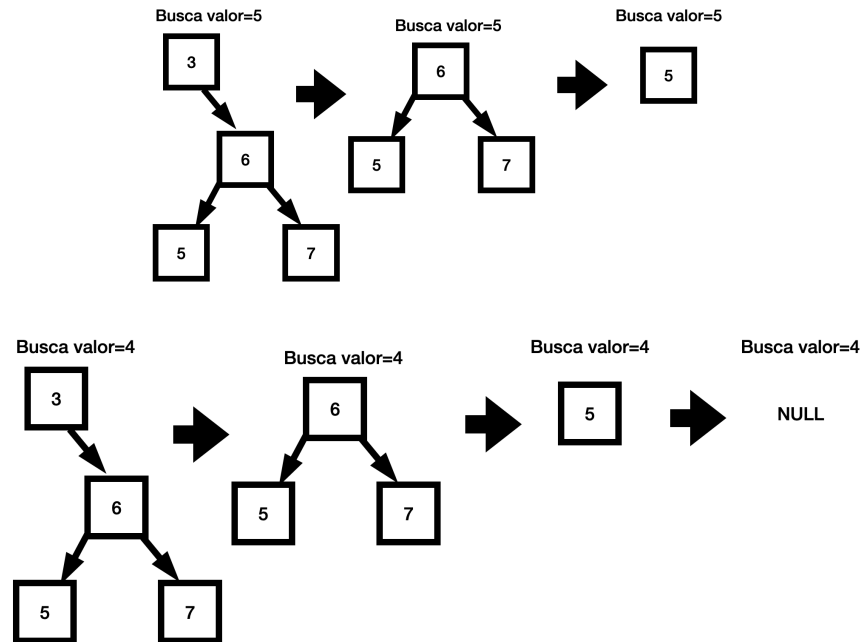


Figura 12.1: Buscando o valor 5 e 4 recursivamente na árvore.

algoritmo de busca:

```

1 No * buscaIterativa(No *no,int valor){
2     No * aux = no;
3     while(aux!=NULL){
4         if(aux->valor==valor)
5             return aux;
6         else if(valor<aux->valor)
7             aux = aux->esq;
8         else
9             aux = aux->dir;
10    }
11    return NULL;
12 }
```

Veja um exemplo de busca executando este código na Figura 12.2

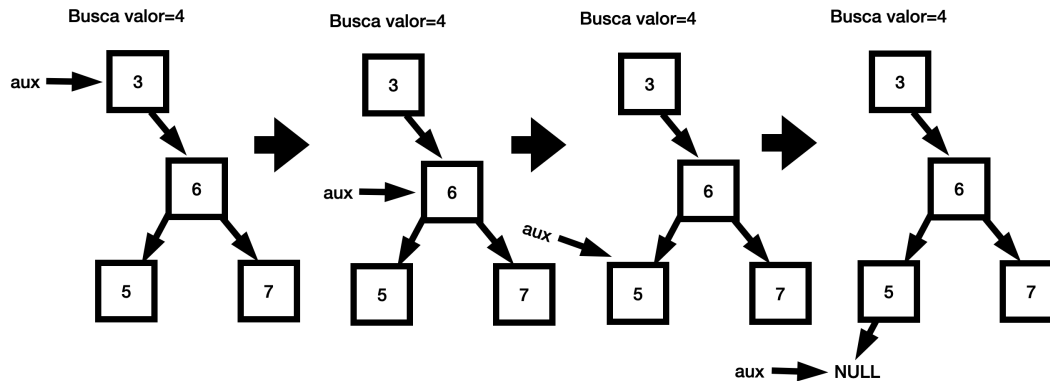


Figura 12.2: Buscando o valor 4 iterativamente na árvore.

Questão 135) Suponha agora que você tem em mãos uma árvore que não é de busca, ou seja, não possui a propriedade de menores à esquerda e maiores à direita. Implemente uma função de busca nesta árvore e analise a sua complexidade. Seu algoritmo deve ser naturalmente recursivo.

12.2 Percursos em uma árvore

Agora vamos ver como percorrer os nós de uma árvore. Existem três maneiras bem estabelecidas de realizar um percurso em um árvore. São elas: em-ordem, pré-ordem e pós-ordem. Veja a definição abaixo:

- **em-ordem** Visita-se primeiro os nós do ramo à esquerda da raiz, recursivamente. Depois visita-se a raiz e, em seguida, visita-se os nós do ramo à direita da árvore, recursivamente.
- **pré-ordem** Primeiro a raiz, depois ramo à esquerda e, em seguida, ramo à direita.
- **pós-ordem** Primeiro o ramo à esquerda, depois o ramo à direita e, finalmente visita-se a raiz.

O código dos percursos são naturalmente recursivos. Veja os códigos:

```
1 //percurso em-ordem
```

```
2 void printEmOrdem(No *no){
3     if(no!=NULL){
4         printEmOrdem(no->esq);
5         printf(" %d",no->valor);
6         printEmOrdem(no->dir);
7     }
8 }
9 //percurso em pre-ordem
10 void printPreOrdem(No *no){
11     if(no!=NULL){
12         printf(" %d",no->valor);
13         printPreOrdem(no->esq);
14         printPreOrdem(no->dir);
15     }
16 }
17 //percurso em pos-ordem
18 void printPosOrdem(No *no){
19     if(no!=NULL){
20         printPosOrdem(no->esq);
21         printPosOrdem(no->dir);
22         printf(" %d",no->valor);
23     }
24 }
```

Note que as árvores pode ser de busca ou não para realizar percursos. Veja exemplos de percurso em-ordem na Figura 12.3. Note que é sempre impresso o ramo da esquerda, recursivamente, depois a raiz e, em seguida, os nós do ramo à direita.

O próximo exemplo trata da impressão dos nós utilizando o percurso em pré-ordem. Veja Figura 12.4. Veja agora a impressão da árvore em pós-ordem. Veja Figura 12.5.

Questão 136) Faça a impressão das árvores da Figura 12.6, em-ordem, em pré-ordem

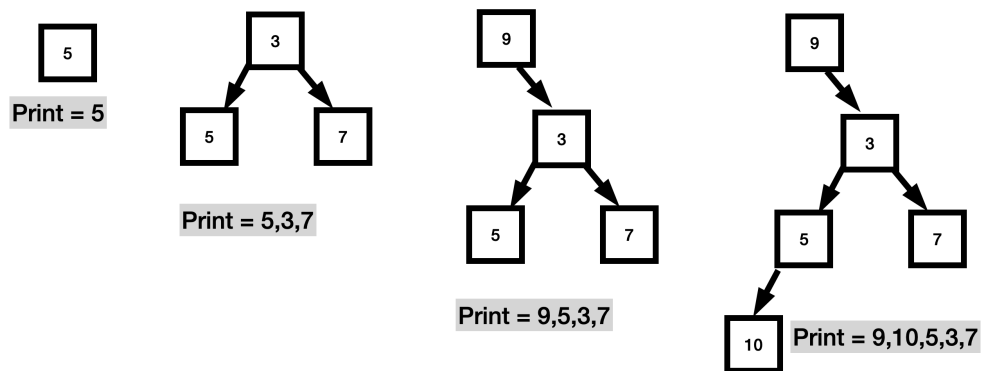


Figura 12.3: Impressão das árvores em-ordem

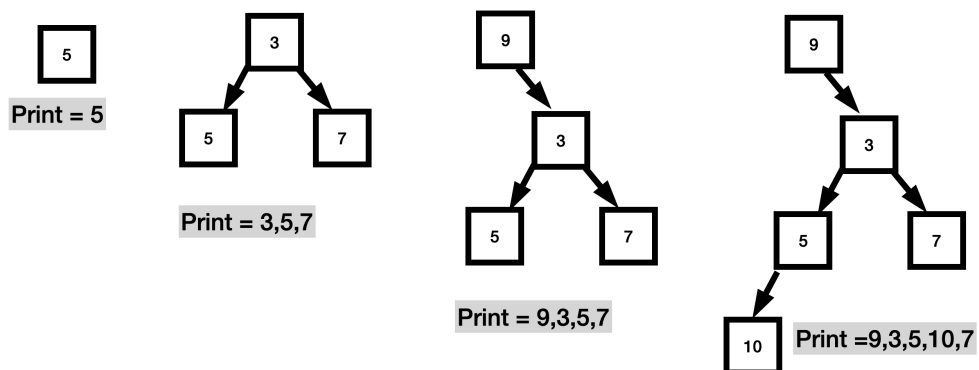


Figura 12.4: Impressão das árvores em pré-ordem

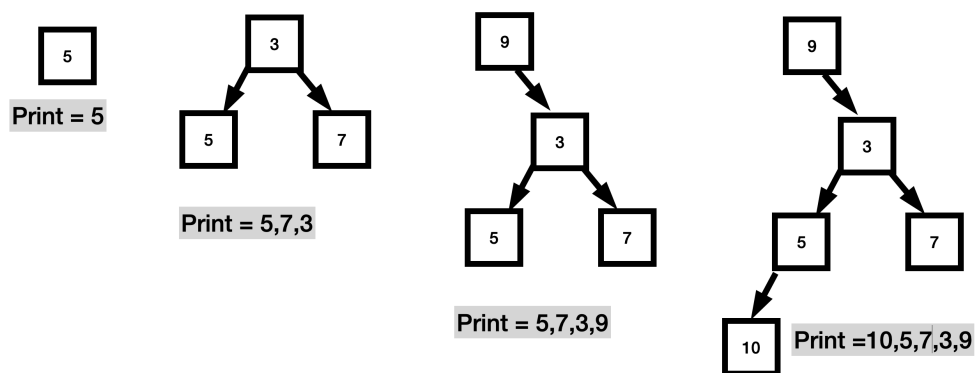


Figura 12.5: Impressão das árvores em pós-ordem

e em pós-ordem.

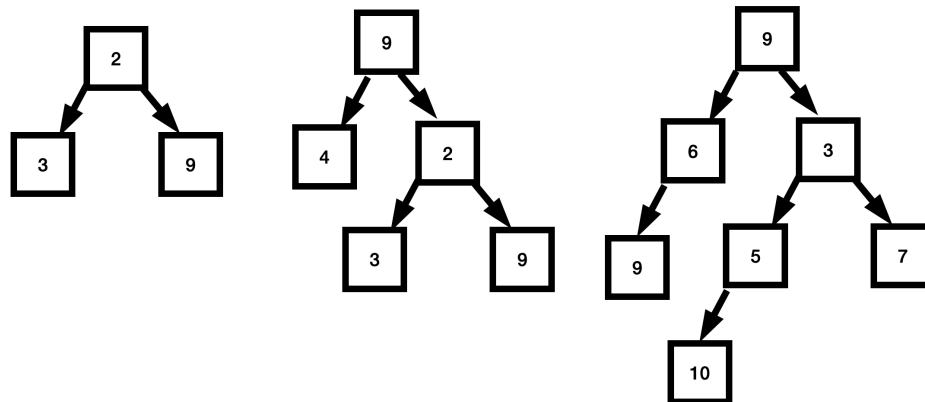


Figura 12.6: Faça a impressão das árvores em-ordem, em pré-ordem e em pós-ordem.

Uma característica interessante destes percursos em árvores é que, se estiver disponível o percurso em-ordem e em pré-ordem, é possível reconstruir a árvore original. Suponha os seguintes percursos:

- pré-ordem : (7, 6, 5, 8, 10, 9, 1)
- em-ordem: (8, 5, 6, 7, 9, 10, 1)

Vamos reconstruir a árvore. Primeiro vamos descobrir quem é a raiz. No percurso em pré-ordem, inicia-se a impressão pela raiz, logo, a raiz da árvore é valor 7. Sabendo que a raiz é o valor 7, voltamos ao percurso em ordem. Veja que, em ordem, a impressão é (8, 5, 6, **7**, 9, 10, 1). Sabendo que a impressão começa com o ramo da esquerda, podemos concluir que (8, 5, 6) é o ramo à esquerda e que (9, 10, 1) é o ramo à direita. Vamo analisar o ramo da esquerda, que, em pré-ordem, vale (6, 5, 8) e o ramo da direita, que em pré-ordem é (10, 9, 1). Veja a Figura 12.7.

Veja o algoritmo para reconstrução da árvore a partir dos percursos:

1. Localize, no percurso em pré-ordem, a raiz da árvore.
2. Com a raiz, descubra quais nós à esquerda e à direita, usando o percurso em-ordem.
3. Divida a árvore em duas.

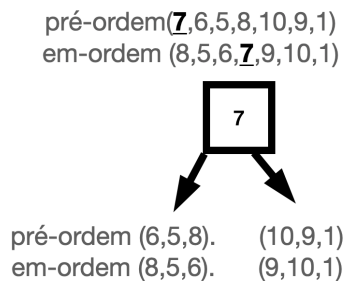


Figura 12.7: Reconstrução da árvore a partir dos percursos.

4. Repita o processo recursivamente nas duas árvores, até que reste um único vertice.

O algoritmo na subárvore à esquerda e à direita. Veja Figura 12.8 Veja a árvore final

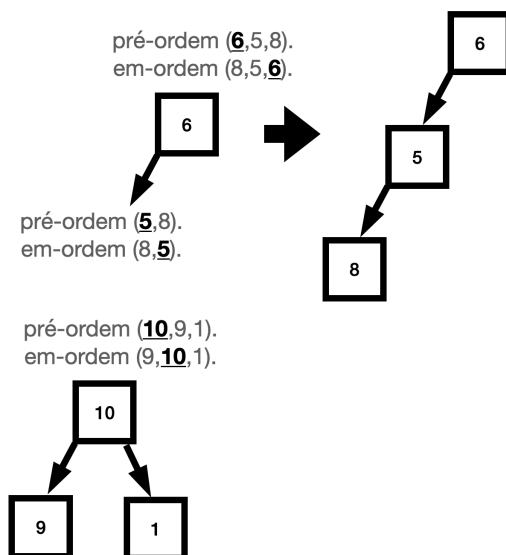


Figura 12.8: Reconstrução da árvore a partir dos percursos. Aplicando o algoritmo na árvore à esquerda e à direita da Figura 12.7.

após a reconstrução na Figura 12.9.

Questão 137) Reconstrua as árvores a partir dos percursos:

- (a)

pré-ordem: (1, 3, 2)

em-ordem: (3, 1, 2)

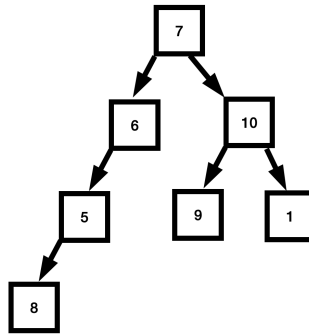


Figura 12.9: Árvore final após reconstrução.

- (b)

pré-ordem: (1, 2, 3)

em-ordem: (1, 2, 3)

- (c)

pré-ordem: (1, 2, 3, 4, 5)

em-ordem: (2, 1, 4, 3, 5)

- (d)

pré-ordem: (7, 10, 5, 2, 8, 9, 4, 3, 1, 6)

em-ordem: (2, 5, 8, 10, 9, 7, 3, 4, 6, 1)

Questão 138) Após criar as árvores, faça o percurso em-ordem e em pré-ordem para tirar a prova de que as árvores estão corretas.

Uma característica interessante é que, se a árvore for uma árvore de busca, o percurso em-ordem visitará seus nós em ordem crescente, conforme foi mostrado no algoritmo de ordenação mostrado na aula passada.

Questão 139) Considere 3 árvores binárias de busca. O percurso em pré-ordem das três árvores estão abaixo:

A: (3, 0, 1, 2, 9, 7, 4, 5, 6, 8)

B: (8, 4, 0, 2, 1, 3, 5, 6, 7, 9)

C: (9, 8, 7, 6, 4, 0, 2, 1, 3, 5)

Seja h_a , h_b e h_c a altura das árvores A, B e C, respectivamente. Quanto vale $x = h_a + h_b + h_c$?
Você deve mostrar as árvores A, B e C as respectivas alturas.

Suponha agora que você deseja dar um *free*, ou seja, liberar a memória de uma árvore. Para realizar esta tarefa, é preciso percorrer todos os nós e dar um *free* em cada nó. O percurso mais indicado é o pós-ordem. Primeiro libera recursivamente a árvore à esquerda, depois libera-se a memória da árvore à direita e, por fim, libera-se a memória do nó raiz.

Veja o código abaixo:

```
1 void liberaMemoria(No * no){
2     if(no==NULL) return ;
3     liberaMemoria(no->esq);
4     liberaMemoria(no->dir);
5     printf("Free. Valor=%d\n",no->valor);
6     free(no);
7 }
```

Veja o resultado da impressão após a execução da função *liberaMemoria* na árvore da Figura 12.10. Note que os nós são liberados considerando-se um percurso em pós-ordem.

Free. Valor=0

Free. Valor=3

Free. Valor=8

Free. Valor=9

Free. Valor=7

Após liberar a memória, a árvore não poderá ser mais utilizada, pois agora ela guarda endereços de memórias inválidos. A sugestão é fazer com que a raiz da árvore valha nulo após a execução da função.

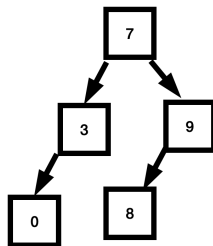


Figura 12.10: Árvore utilizada pela função libera memória.

Questão 140) Altere a função libera memória para que os nós sejam liberados em pré-ordem. Note que, após executar $free(no)$ não é mais possível acessar $no \rightarrow esq$ e $no \rightarrow dir$, logo estes valores precisam ser salvos em variáveis auxiliares antes de executar o $free$.

Note que, para realizar os percursos, todos os nós da árvore são visitados, logo, complexidade será $O(n)$ onde n é o número de nós da árvore.

12.2.1 Calculando o número de elementos da árvore

Para calcular o número de elementos de uma árvore, que chamaremos de $size$, é preciso percorrer todos os nós da árvore e, a cada nó encontrado, somar o valor 1. Veja que a definição de tamanho é algo recursivo. O $size(no)$ é igual ao número de elementos dos ramo à esquerda, $size(no \rightarrow esq)$, somado ao número de nós do ramo à direita, $size(no \rightarrow dir)$, mais um, que é o nó da própria raiz. Veja Figura 12.11.

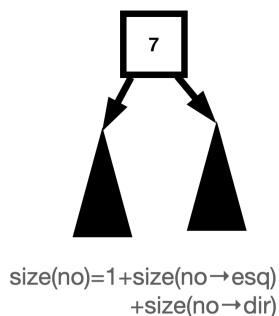


Figura 12.11: Número de nós de uma árvore possui uma definição recursiva.

Veja o código para calcular o size a seguir:

```
1 int size(No *no){
2     if(no==NULL){
3         return 0;
4     }else{
5         int sizeEsq = size(no->esq);
6         int sizeDir = size(no->dir);
7         return 1+sizeEsq+sizeDir;
8     }
9 }
```

Veja que a função *size* faz um percurso em pós-ordem. Primeiro executa *size*(*no* → *esq*) depois executa *size*(*no* → *dir*) e finalmente considera a raiz, ao somar 1. A árvore nula tem *size* igual a zero.

Até o momento, para se percorrer os nós de uma árvore, foi utilizado algoritmos recursivos, mas é possível percorrer os elementos de uma árvore sem usar recursão. Veja a questão à seguir:

Questão 141) Faça a função *size* sem utilizar recursão.

A seguir um algoritmo em pseudo-código para resolver a Questão 141.

1. Crie uma pilha onde cada elemento é um apontador para nó.
2. insira o nó raiz da árvore na pilha.
3. Enquanto pilha não estiver vazia:
 - (a) Remova um nó da pilha e salve em *aux*.
 - (b) imprima o valor do nó para fins de depuração.
 - (c) faça *size++*.
 - (d) Insira os filhos de *aux* na pilha, caso sejam diferentes de nulo. Insira primeiro o filho da direita e depois o filho da esquerda.

Você deve implementar este algoritmo na linguagem C.

Questão 142) Este algoritmo é capaz percorrer os nós de uma árvore trocando-se a recursão por uma pilha. Em que ordem os nós da árvore serão impressos?

Suponha agora uma pequena variação deste algoritmo conforme descrito abaixo:

1. Crie uma **fila** onde cada elemento é um apontador para nó.
2. insira o nó raiz da árvore na **fila**.
3. Enquanto **fila** não estiver vazia:
 - (a) Remova um nó da **fila** e salve em *aux*.
 - (b) imprima o valor do nó *aux* para fins de depuração.
 - (c) faça *size++*.
 - (d) Insira os filhos de *aux* na **fila**, caso sejam diferentes de nulo. Insira primeiro o filho da esquerda e depois o filho da direita.

Questão 143) Descreva como serão impressos os nós da árvore neste algoritmo.

A complexidade do algoritmo *size* e suas variantes é sempre $O(n)$ onde n é o número de nós da árvore.

12.2.2 Calculando a altura de uma árvore

A altura de uma árvore é a maior distância entre a raiz e uma das folhas. Por definição, a árvore nula tem altura igual a -1. Veja alguns exemplos de altura. A altura será identificada pela letra h do inglês *height*. Veja Figura 12.12. Note que a altura de uma árvore é uma propriedade recursiva. Ela vale para a raiz e para todas as subárvores. Veja Figura 12.13

Suponha uma árvore cujo filho da direita tem altura 13 e o filho da esquerda tem altura 10. Neste caso, para se chegar até a raiz dá-se mais um passo. Com estes passos adicionais, temos altura igual a 14 e 11. Como a altura é o maior caminho entre a raiz e as folhas, ficamos com o maior valor. Veja a definição recursiva de altura abaixo:

$$h(no) = \max\{h(no \rightarrow esq), h(no \rightarrow dir)\} + 1,$$

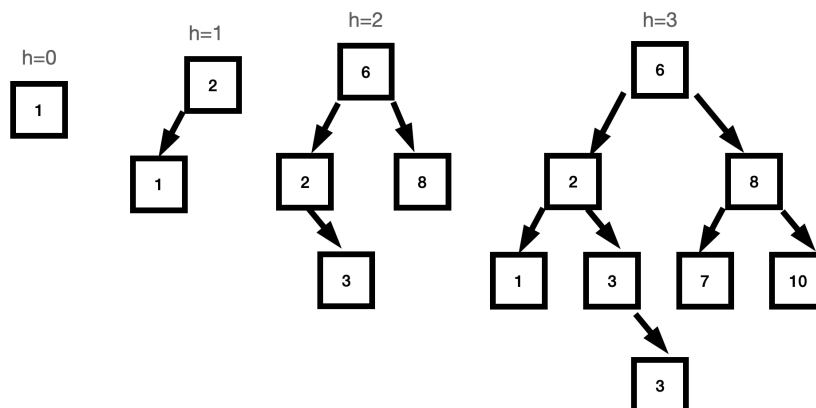


Figura 12.12: Exemplos de alturas de árvores.

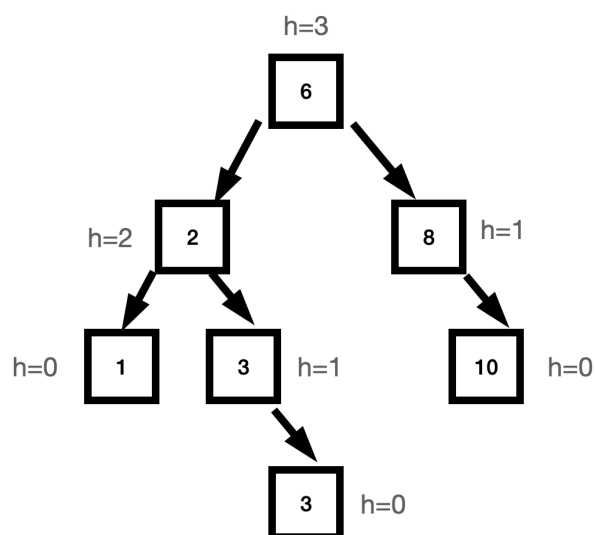


Figura 12.13: Altura de árvore é uma propriedade recursiva.

Ou seja, a altura de um nó é a altura máxima considerando-se os ramos à esquerda mais um. Veja Figura 12.14

Veja o código para o calculo da altura.

```

1 int altura(No *no){
2     if(no==NULL){
```

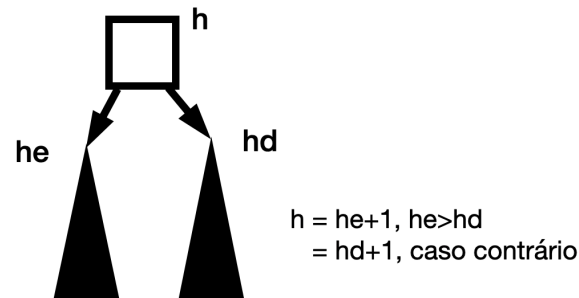


Figura 12.14: Altura de árvore em relação aos ramos à direita e à esquerda.

```

3         return -1;
4     }else{
5         int he = altura(no->esq);
6         int hd = altura(no->dir);
7         if (he>hd){
8             return he+1;
9         }else{
10            return hd+1;
11        }
12    }
13 }
```

Veja que novamente trata-se de um percurso em pós-ordem onde é calculada a altura de todos os nós. A altura de um nó sobe para quem chamou a função altura, é comparado com o outro ramo e o máximo somado de um é a altura do nó corrente. A complexidade é $O(n)$ onde n é o número de elementos da árvore.

Uma característica interessante das árvores é que, se a árvore tiver um bom balanceamento, a altura é $O(\lg n)$. Porém, para uma árvore degenerada, a altura pode atingir $O(n)$. Vamos fazer um exercício interessante:

12.3 Exercícios de implementação

Questão 144) Considere inserir n elementos aleatórios em uma árvore, onde cada elemento é um valor uniformemente distribuído entre 0 e $2n$. Faça isso para vários valores de n e descubra a altura da árvore, preenchendo a tabela abaixo:

n	h
1.000	
10.000	
50.000	
100.000	

Para o valor de 100.000 eu executei o código em minha máquina e obtive o seguinte valor para altura: 38. Veja que se a árvore tivesse balanceamento perfeito, a altura seria $\lfloor \lg n \rfloor$ que para 100.000 vale 16. Inserindo-se números aleatórios o valor foi pouco maior que $2 \lg n$ porém muito menor que n . Nestas condições, tanto a inserção quando a busca são realizadas de maneira rápida.

Por outro lado, se inseríssemos os valores na árvore em ordem crescente, a altura seria $n - 1$ e a árvore perderia sua eficiência. Existem árvores que mantem a propriedade da altura em $\lg n$ para qualquer ordem de inserção, como por exemplo, árvore B e árvore AVL. Neste curso iremos estudar árvore B.

Questão 145) Pense numa maneira de calcular a altura de uma árvore sem usar recursão.

Questão 146) Considere que uma árvore binária é balanceada se a diferença de altura entre o filho da direita e o filho da esquerda for 1 ou 0, para todo nó. Se existir um nó em que a altura da direita for diferente da altura da esquerda por mais de uma unidade é não balanceada. Faça um código que diz se uma árvore binária de busca é balanceada. Qual a complexidade do seu código.

Questão 147) Faça um programa que calcula o fator de balanceamento de uma árvore. Seja h a altura de um nó. O fator de balanceamento fb de um nó está definido abaixo:

$$fb(no) = |h(no \rightarrow esq) - h(no \rightarrow dir)|,$$

ou seja, é o módulo ou valor absoluto da diferença de altura dos ramos da direita e da

esquerda. O fator de balanceamento de uma árvore é o máximo fator considerando todos os nós. Qual a complexidade do seu algoritmo.

Questão 148) Faça uma função *conta* que conta o número de nós com valores iguais a uma dada chave de busca. Seu código deve rodar em $O(h)$ se a árvore for uma árvore de busca e em $O(n)$ se a árvore não for de busca. Veja a assinatura da função abaixo:

```
1 int conta(No *no, int valor){
2     //seu codigo vem aqui
3     return 0;
4 }
```

Questão 149) Responda às seguintes questões referente a árvore a seguir. Considere uma árvore ternária, com três filhos. Veja a nova estrutura nó.

```
1 typedef struct No {
2     int valor;
3     struct No * esq;
4     struct No * meio;
5     struct No * dir;
6 }No;
```

A árvore tem as seguintes propriedades no momento da inserção. Elementos menores que a raiz devem ser inseridos à esquerda. Elementos maiores que a raiz devem ser inseridos à direita e elementos iguais à raiz devem ser inseridos no meio.

1. Refaça o método para inserir um elemento em uma árvore seguindo esta propriedade. Faça tanto a versão iterativa quanto a versão recursiva.
2. Implemente um percurso com a seguinte característica. Imprime-se o nó raiz, de maneira recursiva, os ramos à direita, o ramo do meio e o ramo à esquerda, nesta ordem.
3. Imprima esta nova árvore em um arquivo XML. Crie uma árvore com muitos nós e navegue na estrutura XML para ver se a árvore está sendo impressa corretamente.

4. Faça a função que libera memória desta nova árvore.
5. Faça a função que calcula a altura e o tamanho desta árvore.

Considere agora uma árvore com a seguinte estrutura

```
1 typedef struct No {  
2     int valor;  
3     struct No * esq;  
4     struct No * dir;  
5     int size;  
6 }No;
```

Nesta árvore é guardado também o tamanho da árvore. Nestes casos, basta acessar este campo da estrutura para encontrar o valor correto. Entretanto o método de inserção deve ser melhorado. Ao inserir um novo elemento, o *size* deve ser atualizado.

Questão 150) Refaça a função *add* recursivo para que, após a inserção, o campo *no* $\rightarrow$ *size* esteja atualizado corretamente.

Questão 151) Faça o mesmo para a função *add* iterativo.

Veja agora mais uma versão da estrutura nó.

```
1 typedef struct No {  
2     int valor;  
3     struct No * esq;  
4     struct No * dir;  
5     int altura;  
6 }No;
```

Questão 152) Nesta versão fica guardada a altura da árvore dentro da estrutura. Refaça os métodos de inserção para que, após a inserção, o campo *no* $\rightarrow$ *altura* esteja guardando o valor correto da altura.

Referências Bibliográficas

- [Alman and Williams, 2021] Alman, J. and Williams, V. V. (2021). A refined laser method and faster matrix multiplication. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 522–539. SIAM.
- [Cormen et al., 2002] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2002). Algoritmos: teoria e prática. *Editora Campus*, 2:296.
- [Damas, 2006] Damas, L. (2006). *Linguagem C*. LTC.
- [Strassen et al., 1969] Strassen, V. et al. (1969). Gaussian elimination is not optimal. *Numerische mathematik*, 13(4):354–356.
- [Wikimedia Commons contributors, 2024a] Wikimedia Commons contributors (2024a). Binary tree v2 image. https://en.wikipedia.org/wiki/Binary_tree#/media/File:Binary_tree_v2.svg. [Accessed: October 12, 2024].
- [Wikimedia Commons contributors, 2024b] Wikimedia Commons contributors (2024b). Imagem de pastas de computador. https://pt.wikipedia.org/wiki/Diret%C3%B3rio_%28computa%C3%A7%C3%A3o%29#/media/Ficheiro:Pastas-computador.jpg. [Acessado em: 12 de outubro de 2024].
- [Wilson and Watkins, 1990] Wilson, R. J. and Watkins, J. J. (1990). *Graphs: an introductory approach: a first course in discrete mathematics*. Wiley New York.